

# Distributed Quantum Advantage in Locally Checkable Labeling Problems<sup>1</sup>

Alkida Balliu · Gran Sasso Science Institute

Filippo Casagrande · Gran Sasso Science Institute

Francesco d'Amore · Gran Sasso Science Institute

Massimo Equi · Aalto University

Barbara Keller · Aalto University

Henrik Lievonon · Aalto University

Dennis Olivetti · Gran Sasso Science Institute

Gustav Schmid · University of Freiburg

Jukka Suomela · Aalto University

**Abstract.** In this paper, we present the first known example of a locally checkable labeling problem (LCL) that admits asymptotic distributed quantum advantage in the LOCAL model of distributed computing: our problem can be solved in  $O(\log n)$  communication rounds in the quantum-LOCAL model, but it requires  $\Omega(\log n \cdot \log^{0.99} \log n)$  communication rounds in the classical randomized-LOCAL model.

We also show that distributed quantum advantage cannot be arbitrarily large: if an LCL problem can be solved in  $T(n)$  rounds in the quantum-LOCAL model, it can also be solved in  $\tilde{O}(\sqrt{nT(n)})$  rounds in the classical randomized-LOCAL model. In particular, an LCL problem that is strictly global classically is also almost-global in quantum-LOCAL.

This solves a major open question at the intersection of distributed graph algorithms and quantum computing. LCL problems [Naor and Stockmeyer, STOC 1993] have been extensively studied in the past decade and they are by now very well-understood in the classical LOCAL model, yet whether any of them admits a genuine quantum advantage has remained open. Coiteux-Roy et al. [STOC 2024] showed that for *some specific* LCL problems, *if* quantum helps, it cannot help by much. Akbari et al. [STOC 2025] showed that, for *some* LCLs, in *rooted trees*, quantum-LOCAL and randomized-LOCAL have the same power. Balliu et al. [STOC 2025] showed that quantum helps in solving locally checkable problems faster when the maximum degree of the graph is super-constant; however, this does not give any asymptotic quantum advantage for LCLs. The above-mentioned works repeatedly asked the key question of whether quantum helps for LCLs. We solve this open question by giving the first example of an LCL problem that admits a super-constant distributed quantum advantage, and by also giving the first result that puts limits on distributed quantum advantage for LCLs in general graphs.

Our second result also holds for  $T(n)$ -dependent probability distributions. As a corollary, if there exists a *finitely dependent distribution* over valid labelings of some LCL problem  $\Pi$ , then the same problem  $\Pi$  can also be solved in  $\tilde{O}(\sqrt{n})$  rounds in the classical randomized-LOCAL and deterministic-LOCAL models. That is, finitely dependent distributions cannot exist for global LCL problems.

---

<sup>1</sup>This work was originally presented at SODA 2026 [BCd+26]

# 1 Introduction

Does quantum computation and communication help with solving graph problems in the distributed setting? We study this question in the usual LOCAL model of distributed computing, more precisely comparing these two settings (see Section 2 for precise definitions):

- **Randomized-LOCAL model:** Each node of the input graph is a classical computer (that can store an arbitrary number of classical bits), and each edge is a classical communication channel (that can transmit an arbitrary number of classical bits per communication round). Each node is initialized with its own independent random bit string.
- **Quantum-LOCAL model:** Each node of the input graph is a quantum computer (that can store an arbitrary number of qubits), and each edge is a quantum communication channel (that can transmit an arbitrary number of qubits per communication round). Each node is initialized with its own unentangled qubits.

Given a graph problem  $\Pi$  (e.g. graph coloring), we say that it can be solved in  $T(n)$  rounds if there is a distributed algorithm  $A$  such that in any  $n$ -node graph after  $T(n)$  communication rounds each node terminates and outputs its own part of the solution (e.g. its own color).

## 1.1 Prior work on distributed quantum advantage

It is not at all obvious if quantum-LOCAL could possibly admit any advantage over randomized-LOCAL; after all, we did not put any limits on the amount of local computation, local storage, or number of bits communicated per round. Could it be the case that any quantum-LOCAL algorithm can be simulated with a classical algorithm in the same number of rounds (at least asymptotically), if we just encode the local state of qubits with an exponentially larger number of classical bits?

Surprisingly, this turns out not to be the case: Le Gall, Nishimura, and Rosmanis [LNR19] show that there are graph problems that can be solved in  $O(1)$  rounds in quantum-LOCAL but that require  $\Omega(n)$  rounds in classical randomized-LOCAL. However, the problem studied in [LNR19] is very different from problems commonly studied in the theory of distributed graph algorithms. In particular, their problem has an inherently *global* specification.

## 1.2 Prior work on LCL problems

In recent years, a large body of literature has focused on **locally checkable labeling problems**, or LCLs in brief, first introduced by Naor and Stockmeyer [NS95]. These are graph problems in which valid solutions can be specified by listing a finite set of valid labeled neighborhoods. LCLs strike a balance between being broad enough so that they contain a large number of interesting graph problems and being narrow enough so that it is possible to prove strong theorems that apply to all LCL problems, see e.g. [NS95, CP19, CKP19, BFH+16, BHK+18, BBO+21, BBO+20, Suo20, DDL+23]. There are numerous results about LCL problems in the classical LOCAL model, yet we do not know if *all* of these results hold also in the quantum-LOCAL model. Hence, this is the key question that we study: **do any LCL problems admit a distributed quantum advantage?**

There are several papers that have so far delivered mainly negative results: we have learned about cases in which quantum cannot help, at least not much [CDG+24, DKL+24, ACd+25, BCC+25], and we have also learned about barriers for studying such questions [ACd+25]. The main exception is the very recent work [BBC+25] that showed the following separation result: there is a family of LCL problems  $\text{GHZ}(\Delta)$  parameterized by maximum degree  $\Delta$  such that  $\text{GHZ}(\Delta)$  can be solved in  $O(1)$  rounds in quantum-LOCAL but it requires  $\Omega(\Delta)$  rounds in randomized-LOCAL. However,

this does not yield a super-constant separation for any fixed LCL problem: for any fixed  $\Delta$ , problem  $\text{GHZ}(\Delta)$  is an LCL that can be solved in  $O(1)$  rounds (albeit with very different constants) in both quantum-LOCAL and randomized-LOCAL.

Is it possible to exhibit a single LCL problem  $\Pi$  such that  $\Pi$  is solvable in  $T(n)$  rounds in quantum-LOCAL but requires  $\omega(T(n))$  rounds in classical models? This is a **key open question** that has been mentioned repeatedly in the literature, see e.g. [CDG+24, ACd+25, BBC+25, Suo24].

### 1.3 Contribution 1: first LCL problem with a quantum advantage

In this work we show that the answer is yes: quantum-LOCAL is strictly stronger than randomized-LOCAL for LCL problems. More precisely, in Sections 3 and 5 we prove:

**Theorem 1.1.** *There is an LCL problem  $\Pi$  such that the round complexity of  $\Pi$  is  $O(\log n)$  in quantum-LOCAL but  $\Omega(\log n \cdot \log^{0.99} \log n)$  in randomized-LOCAL.*

This brings us both good news and bad news: The good news is that this demonstrates that distributed quantum computing indeed helps even when restricted to the family of LCL problems. The bad news is that there is now no longer hope that we could prove a theorem that shows that quantum-LOCAL and randomized-LOCAL are asymptotically equally strong for LCL problems, enabling us to lift a large body of prior work from classical models to the quantum model—to fully characterize an LCL problem, we may need to separately classify its locality in deterministic-LOCAL, randomized-LOCAL, and quantum-LOCAL.

### 1.4 Contribution 2: limits on quantum advantage for LCLs

While Theorem 1.1 resolves the open question, the separation that we have between the classical and quantum models is tiny, especially if we compare this with the non-LCL problem from [LNR19] that admits an  $O(1)$ -round quantum algorithm and requires  $\Omega(n)$  rounds in classical models. Could we demonstrate such a separation with LCL problems?

In this work we show that the answer is no: there is no LCL with a linear-in- $n$  gap between classical and quantum round complexities. Formally, in Section 4 we show:

**Theorem 1.2.** *Let  $\Pi$  be any LCL problem that can be solved in  $T(n)$  rounds in quantum-LOCAL. Then  $\Pi$  can be solved in  $O(\sqrt{nT(n)} \text{poly log } n)$  rounds in randomized-LOCAL.*

In particular, if the classical round complexity is  $\Omega(n)$ , i.e., the problem is inherently global, then also in the quantum-LOCAL model we need  $\tilde{\Omega}(n)$  rounds (here we are using  $\tilde{\Omega}$  and  $\tilde{O}$  to hide polylogarithmic factors).

The main open question after this work is narrowing down the gap between Theorems 1.1 and 1.2: can we construct LCLs where quantum-LOCAL helps more than some doubly-logarithmic factors? Currently, there is a larger gap between deterministic-LOCAL and randomized-LOCAL than randomized-LOCAL and quantum-LOCAL, which seems counterintuitive: is access to classical randomness already almost as good as the ability to manipulate qubits?

While we do not know yet if Theorem 1.2 is tight, in Section 1.6 we will see that there is a direct generalization of Theorem 1.2 to stronger (super-quantum) models, and there the result turns out to be close to the best possible, in the sense that we may have a gap of  $\tilde{\Omega}(\sqrt{n})$  between classical models and super-quantum models. Hence to strengthen the claim of Theorem 1.2, we need techniques that do *not* generalize far beyond quantum-LOCAL.

## 1.5 Key ideas in the proof of Theorem 1.1

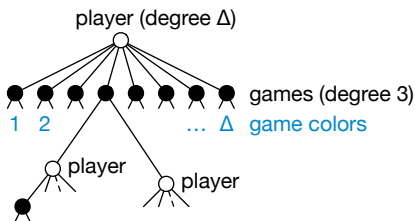
In the proof of Theorem 1.1, the key novel ingredient is the observation that the  $\text{GHZ}(\Delta)$  problem from [BBC+25] can be *linearized* (with high-degree nodes replaced by paths in a way that preserves local checkability), and then further *padded* with the technique from [BBO+20] to construct a genuine LCL problem that admits a quantum advantage. We summarize here the key elements of the proof.

**LCL problems.** As our goal is to construct an LCL problem that exhibits distributed quantum advantages, let us recall what kind of entities LCL problems are. We will present a formal definition in Section 2, but the following informal description suffices for now: An LCL problem  $\Pi$  can be specified by listing a **finite set** of valid labeled radius- $r$  neighborhoods  $\mathcal{N}$  for some constant  $r$ . The task is to label nodes and/or edges of the input graph  $G$ , and a solution is feasible if for every node  $v$ , the radius- $r$  neighborhood of  $v$  is isomorphic to one of the neighborhoods in  $\mathcal{N}$ .

In particular, this implies that the set of node labels and edge labels has to be also finite, and  $G$  must be a bounded-degree graph in order to admit a solution to  $\Pi$ . This leads to another equivalent way of characterizing LCL problems: they are graph problems for bounded-degree graphs, where the task is to label nodes and/or edges with labels from a finite alphabet, and the validity of a solution can be verified in a distributed manner by checking all constant-radius local neighborhoods.

The restrictions may sound at first technical, but this exact definition originally introduced by [NS95] has been tremendously successful: this is a problem family that is now very well-understood especially for the classical LOCAL model. What we seek to demonstrate is that inside this well-understood yet restrictive family there is indeed a problem  $\Pi$  that admits a super-constant quantum advantage.

**Starting point: the iterated GHZ problem.** Let us next recall the key elements of the problem family  $\text{GHZ}(\Delta)$  from [BBC+25]. Here  $\text{GHZ}(\Delta)$  is a family of LCL problems, parameterized by  $\Delta$ . The problem family is defined so that hard instances are bipartite graphs, where one part consists of white nodes of degree  $\Delta$  and the other part consists of black nodes of degree 3. White nodes represent *players* and black nodes represent *games*—more precisely, they are instances of the GHZ game [GHZ89, Mer90], which is a 3-party game where players can win the game without communication if they hold entangled qubits. The games are labeled with colors  $1, 2, \dots, \Delta$ , so that each player is adjacent to one game of each color.



Crucially, for each player, the input that they have in game of color  $i$  is the output they got from game of color  $i - 1$ . The key idea is that for a classical algorithm to win in a color- $i$  game, it first has to know the outputs of color- $(i - 1)$  games, and spend additional communication rounds after that to coordinate a feasible solution for the color- $i$  game. Such a strategy is inherently sequential, taking  $\Omega(\Delta)$  rounds, and it can be shown with the round elimination technique [Bra19] that this is indeed the best that a classical algorithm can do (at least in neighborhoods that are tree-like). However, a quantum algorithm can do much better: we can use one communication round to establish a set of

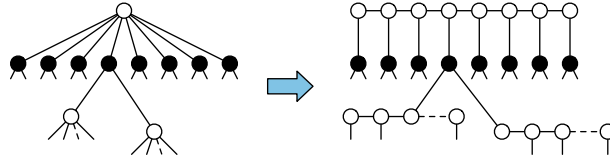
shared qubits for each game, and then the players can do the rest locally, by applying appropriate measurements to the shared qubits; this is  $O(1)$  rounds independently of  $\Delta$ .

At this point it is also good to recall that in the classical LOCAL model, time and distances are interchangeable: a  $T$ -round algorithm is equivalent to a function that maps radius- $T$  neighborhoods to local outputs (and vice versa). So the key point is this: any classical algorithm that solves  $\text{GHZ}(\Delta)$  has to see up to distance  $\Omega(\Delta)$ , in neighborhoods that locally look like a two-colored tree in which all white nodes have degree  $\Delta$  and black nodes have degree 3 (we will call it a  $(\Delta, 3)$ -biregular tree).

Now  $\text{GHZ}(\Delta)$  is a well-defined graph problem even if we plug in e.g.  $\Delta = \sqrt{n}$ , but then we cannot claim that any classical algorithm for solving it requires  $\Omega(\sqrt{n})$  rounds, simply because we can no longer construct a hard instance that would be a  $(\sqrt{n}, 3)$ -biregular tree of depth  $\sqrt{n}$  with at most  $n$  nodes, which certainly does not exist. However, a more modest choice of  $\Delta \approx \log n / \log \log n$  makes sense and results in a graph problem that admits an  $O(1)$ -round quantum algorithm but requires  $\Omega(\log n / \log \log n)$  rounds for any classical deterministic algorithm (the case of classical *randomized* algorithms is more involved, but let us put it aside for now for the purposes of this informal overview). However, this is **not an LCL problem**. It is some labeling problem in which validity of a solution is locally verifiable, but it cannot be described with any finite set of valid neighborhoods; in particular, we will need super-constant degrees in the input graph.

Yet this is a useful source of inspiration for the present work. We aim at engineering a graph problem  $\Pi$  that captures the spirit of “what would happen if we take  $\text{GHZ}(\Delta)$  and plug in something like  $\Delta \approx \log n / \log \log n$ ,” and at the same time  $\Pi$  is a genuine LCL problem in the strict sense of [NS95], without any cheating.

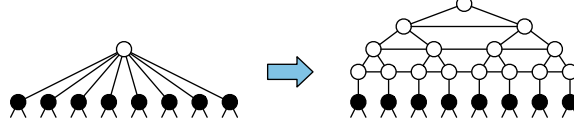
**Linearizing the iterated GHZ problem.** The key new observation is that  $\text{GHZ}(\Delta)$  happens to have a very convenient property: for each player, the set of valid labels on the incident edge number  $i$  only depends on the labels of the incident edge number  $i - 1$ . In particular, there is a finite automaton that processes the labels of the incident edges in the order specified by the colors of the games and determines if a solution is valid from the player’s perspective. This makes it possible to **linearize** the problem so that we can replace a player of degree  $\Delta$  with a path of  $\Delta$  nodes. This way we can (1) eliminate high-degree nodes, (2) preserve the property that we have a finite set of labels, and (3) preserve the property that the feasibility of a solution can be verified locally.



Therefore we can define an LCL problem  $\Pi_1$ , where all nodes have low degrees, and a path with  $\Delta$  white nodes behaves as if it was a degree- $\Delta$  player in the original  $\text{GHZ}(\Delta)$  problem. However, while doing this translation, we lose all control over the degrees of the players. What if the adversary constructs an input where there is a white path of length  $\Theta(n)$ ? Even a quantum algorithm would have to spend  $\Theta(n)$  rounds to just emulate the behavior of a single  $\Theta(n)$ -degree player, and hence there is no room for quantum advantage (as *any* problem can be solved in  $O(n)$  rounds by brute force).

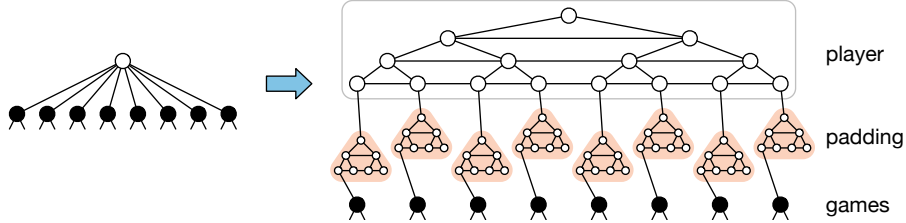
We circumvent this challenge with a commonly-used solution: instead of a path we use a tree-like construction (a balanced binary tree where each level forms a path). In particular, this way the white nodes that represent a  $\Delta$ -degree player will be within distance  $O(\log \Delta)$  from each other, and even in the worst case the overhead of simulating one such node is  $O(\log n)$ . We can furthermore

add additional rules that ensure that if the adversary presents a graph that does not represent a valid instance, we can nevertheless produce a valid labeling in  $O(\log n)$  rounds; let us call the resulting LCL problem  $\Pi_2$ .



**Padding.** Unfortunately,  $\Pi_2$  as constructed above does not admit any distributed quantum advantage. To construct a hard instance for classical algorithms, we would need to construct a  $(\Delta, r)$ -biregular tree of depth  $\Omega(\Delta)$ , and then replace each degree- $\Delta$  node with our tree-like construction. But we will have to use  $\Delta = O(\log n / \log \log n)$  so that we do not run out of nodes. To construct a worst-case instance, we would now take an  $O(\Delta)$ -deep  $(\Delta, r)$ -biregular tree and replace each white degree- $\Delta$  node with a tree-like construction of diameter  $O(\log \Delta)$ . This replacement process increases the diameter of the graph from  $O(\Delta)$  to  $O(\Delta) \cdot O(\log \Delta)$ . But the diameter of the entire graph would be still bounded by  $O(\Delta) \cdot O(\log \Delta) = O(\log n)$ , and a classical algorithm could solve it trivially in  $O(\log n)$  rounds by gathering everything. On the other hand, a quantum algorithm needs  $\Omega(\log n)$  rounds to merely simulate one player in the worst case, so we gained nothing.

We circumvent this issue by borrowing the idea of **padding** from [BBO+20]. In essence, we replace each edge between a player (white node) and a game (black node) with another tree-like construction, where the adversary gets to choose the height of the tree.



It turns out that this is enough to construct an LCL problem  $\Pi$  that exhibits a distributed quantum advantage! In essence, the reason for this is as follows:

- Quantum algorithms: In the worst case the adversary can add padding of depth  $O(\log n)$ , and the total overhead that we have in simulating the original  $O(1)$ -round quantum algorithm is  $O(\log n)$  rounds to simulate the activities of the players plus  $O(\log n)$  rounds to transmit information across a padded edge, in total  $O(\log n)$  rounds.
- Classical algorithms: The adversary can select a suitable super-constant  $\Delta$  and use trees of height  $O(\log \Delta)$  to represent players and trees of height  $O(\log n)$  for padding. We can start with a  $\text{GHZ}(\Delta)$ -instance with e.g.  $\Theta(n^{1/3})$  players, and we can then afford to use e.g.  $\Theta(n^{1/3})$  nodes for each tree that we use in padding. This results in an instance in which the algorithm has to see beyond  $\Delta$  padded edges, in total up to distance  $\Omega(\Delta \log n)$ , which is worse than the complexity of the quantum algorithm for a super-constant  $\Delta$ .

This is the essence of the construction. We give a more formal high-level overview in Section 3 and the technical details are postponed to Section 5.

**Generalizations and future work.** We note here that exactly the same framework is applicable to a wide range of problems beyond  $\text{GHZ}(\Delta)$ , as we are only making essential use of the fact that the validity of a solution for high-degree nodes can be verified with a finite automaton that processes edges in a sequential order, and hence we can linearize the problem. In particular, the widely-studied maximal matching problem has the same property.

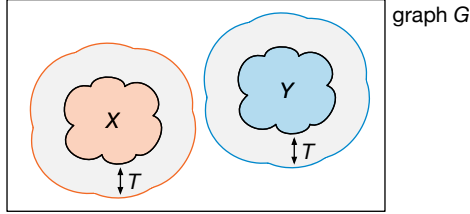
Therefore we expect that the same construction will find use in engineering LCL problems that separate other models. For example, the maximal matching problem would be a good candidate for proving a separation between the quantum-LOCAL model and the SLOCAL model [GKM17].

## 1.6 Key ideas in the proof of Theorem 1.2

To prove Theorem 1.2, we first take a step from distributed quantum algorithms to bounded-dependence distributions, and then observe that while such distributions *cannot* be efficiently sampled in the randomized-LOCAL model, we can nevertheless do a combination of *partial sampling* and *brute-force completion* to solve the same LCL problem.

**Bounded-dependence distributions.** The statement of Theorem 1.2 refers to the quantum-LOCAL model, but it will be much more convenient to work in the bounded-dependence model [ACd+25], which is a generalization of finitely-dependent distributions [HL16, HHL18, Hol24].

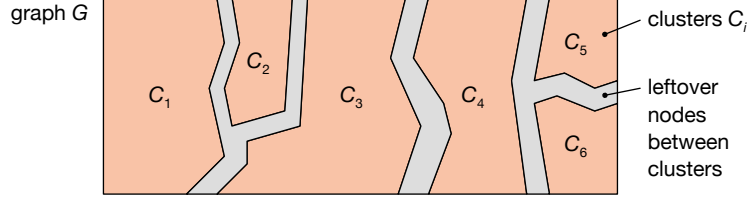
Informally, a  $T$ -dependent distribution is a probability distribution  $P$  over labelings of some graph  $G$  where the following holds: if we choose two sets of nodes  $X$  and  $Y$  such that their  $T$ -radius neighborhoods do not overlap, then  $P$  restricted to  $X$  is independent of  $P$  restricted to  $Y$ .



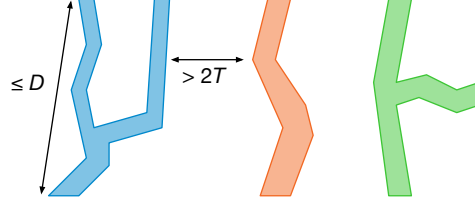
While it is easy to see that the output of a  $T$ -round randomized-LOCAL algorithm results in a  $T$ -dependent distribution, it also holds that the output of a  $T$ -round quantum-LOCAL algorithm results in a  $T$ -dependent distribution. Hence it suffices to show that if we have a  $T$ -dependent distribution  $P$  over valid solutions of some LCL problem  $\Pi$ , we can construct a randomized-LOCAL algorithm that solves the same problem in  $\tilde{O}(\sqrt{nT})$  rounds.

**Sampling is hard.** It would be tempting to now design a randomized-LOCAL algorithm that produces a sample from distribution  $P$ . However, this is *not* possible. Indeed, if we could sample efficiently from any  $T$ -dependent distribution, we would also have a classical algorithm that efficiently simulates the distributed quantum algorithm from [LNR19], which is known to require  $\Omega(n)$  rounds.

**Partial sampling is easy.** We will exploit the clustering algorithm from [CDG+24]. In particular, with it we can partition nodes into well-separated clusters where the diameter of each cluster is  $\tilde{O}(\sqrt{nT})$ , the number of unclustered leftover nodes is  $O(\sqrt{n/T})$ , and clusters can be constructed in  $\tilde{O}(\sqrt{nT})$  rounds in the randomized-LOCAL model.

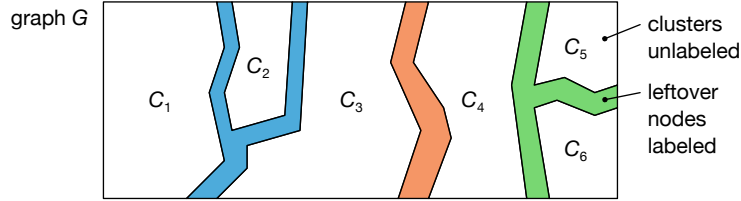


We will split the leftover nodes into components that are at distance  $\Omega(T)$  from each others; in the worst case all leftover nodes are in the same component and it will have diameter  $D = O(\sqrt{nT})$ .



Now if  $X$  and  $Y$  are two components of leftover nodes, they are sufficiently far from each other so that  $P$  restricted to  $X$  is independent of  $P$  restricted to  $Y$ . Hence we can sample from  $P$  independently in each such component, and the joint distribution will be a sample from  $P$  restricted to all leftover nodes.

To recap, so far we have used  $\tilde{O}(\sqrt{nT})$  rounds to compute a clustering, and another  $O(\sqrt{nT})$  rounds to sample from  $P$  in each component of leftover nodes. All leftover nodes have now committed to an output.



**Completeness helps us next.** Now it would be tempting to switch to the clusters and locally sample from  $P$  conditioned on what has already been sampled, with the hope of globally getting a sample from  $P$ . But as we discussed above, this *cannot work*, as it would let us simulate any quantum algorithms too efficiently in the classical models.

This is the point in which we will have to use the assumption that  $P$  is an output distribution that solves the LCL problem  $\Pi$  with high probability. In the previous step, we have produced a sample  $S$  from  $P$  restricted to the leftover nodes. Equivalently, we can imagine that  $S$  is constructed by the following process: we have first sampled a labeling  $S^*$  of the entire graph from  $P$ , and then discarded the labels inside the clusters (keeping only the values of the leftover nodes). Now with high probability  $S^*$  is a valid solution to  $\Pi$ , so with high probability our partial solution  $S$  can be **extended** to some globally valid solution of  $\Pi$ . Furthermore, we can find such a completion independently for each cluster, e.g., simply by brute force: collect the entire cluster and the partial solution at its boundaries and find a valid completion. Here we exploit the fact that  $\Pi$  is a locally checkable problem: any completion that is locally valid in each cluster is also globally valid. As the clusters had diameter  $\tilde{O}(\sqrt{nT})$ , this step takes also  $\tilde{O}(\sqrt{nT})$  rounds, and the total running time is also  $\tilde{O}(\sqrt{nT})$ .

Put together, given a  $T$ -round quantum-LOCAL algorithm for some LCL problem  $\Pi$ , we can construct a randomized-LOCAL algorithm that solves the same problem in  $\tilde{O}(\sqrt{nT})$  rounds. We will give the details of the proof in Section 4.

**Generalizations and future work.** As we took a route through bounded-dependence distributions, we immediately get a stronger result:  $T$ -dependent distributions over LCLs can be simulated in randomized-LOCAL in  $\tilde{O}(\sqrt{nT})$  rounds. In particular, this shows that there cannot exist an LCL that admits a finitely dependent distribution (i.e.,  $O(1)$ -dependent distribution) but requires  $\Omega(n)$  rounds to solve with classical algorithms—previously this was only known in trees [ACd+25, DKL+24].

Our general proof strategy can be pushed further, towards stronger models. It works directly in the *component-wise version* of the online-LOCAL model [AEL+23, ACd+25]. Furthermore, online-LOCAL algorithms with locality  $O(1)$  can be turned into component-wise algorithms [ACd+25], and hence we also get a result that puts limits on the gap between the online-LOCAL and classical LOCAL models. Our simulation result is also close to the best possible, as a much stronger result would contradict with the known bounds for the problem of 3-coloring bipartite graphs, which is a problem that can be solved with locality  $O(\log n)$  in (component-wise) online-LOCAL yet requires  $\Omega(\sqrt{n})$  rounds in the classical randomized-LOCAL model [AEL+23, CDG+24, ACd+25, CMN+23, BHK+17]. We will discuss the extension of our result to component-wise online-LOCAL more in Section A.

## 2 Preliminaries

**Graphs.** We work with simple undirected graphs unless otherwise specified. Let  $G = (V, E)$  be any graph. If the set of nodes  $V$  and the set of edges  $E$  are not specified, we refer to them by the notation  $V(G)$  and  $E(G)$ , respectively. For any subset of nodes  $A \subseteq V$ , the subgraph of  $G$  induced by  $A$  is denoted by  $G[A]$ . The distance between any two nodes  $u, v \in V$  is the number of edges composing a shortest path between  $u$  and  $v$ , and is denoted by  $\text{dist}_G(u, v)$ . The notion of distance can be easily extended to subsets of nodes: Given any node  $u \in V$  and any two subsets  $A, B \subseteq V$ , the distance between  $u$  and  $A$  is  $\text{dist}_G(u, A) = \min_{v \in A} \{\text{dist}_G(u, v)\}$  and the distance between  $A$  and  $B$  is  $\text{dist}_G(A, B) = \min_{u \in A, v \in B} \{\text{dist}_G(u, v)\}$ . When the graph is clear from the context, we omit the suffix and write only  $\text{dist}()$  instead of  $\text{dist}_G()$ .

For any non-negative integer  $T$ , the radius- $T$  (closed) neighborhood of a node  $u$  in a graph  $G$  is the set  $\mathcal{N}_T[u] = \{v \in V \mid \text{dist}(u, v) \leq T\}$ . More generally, the radius- $T$  neighborhood of any subset of node  $A \subseteq V$  is  $\mathcal{N}_T[A] = \cup_{u \in A} \mathcal{N}_T[u]$ .

We adopt the nomenclature *half-edge* to refer to any pair  $(v, e)$  where  $v \in V$  and  $e \in E$  is an edge incident to  $v$ . In order to be able to define the class of problems we consider, we need to define what a labeled graph is.

**Definition 2.1** (Labeled graph). Let  $\mathcal{V}$  and  $\mathcal{E}$  be sets of labels. A graph  $G = (V, E)$  is said to be  $(\mathcal{V}, \mathcal{E})$ -labeled if the following statements are satisfied:

1. Each node  $v \in V$  is assigned a label from  $\mathcal{V}$ ;
2. Each half-edge  $(v, e) \in V \times E$  satisfying  $v \in e$  is assigned a label from  $\mathcal{E}$ .

We also define the notion of *centered graph*, that is, a graph with a distinguished node that we call the *center*.

**Definition 2.2** (Centered graph). Let  $H = (V, E)$  be any graph, and let  $v_H \in V_H$  be any node of  $H$ . The pair  $(H, v_H)$  is said to be a *centered graph* and  $v_H$  is said to be the *center* of  $(H, v_H)$ .

We will focus on problems that ask to produce labelings over graphs that satisfy some constraints in all radius- $r$  neighborhoods of any node. We remind the reader that the *eccentricity* of a node  $v$  of a graph  $H$  is the maximum distance between  $v$  and any other node of  $H$ .

**Definition 2.3** (Set of constraints). Let  $r, \Delta \in \mathbb{N}$  be constants,  $I$  a finite set of indices, and  $(\mathcal{V}, \mathcal{E})$  a tuple of finite label sets. Let  $\mathcal{C}$  be a finite set of centered graphs  $\{(H_i, v_{H_i})\}_{i \in I}$  where each  $H_i$  is a  $(\mathcal{V}, \mathcal{E})$ -labeled graph of degree at most  $\Delta$  and  $v_{H_i}$  has eccentricity at most  $r$ . Then,  $\mathcal{C}$  is said to be an  $(r, \Delta)$ -set of constraints over  $(\mathcal{V}, \mathcal{E})$ .

With the following definition, we explain what it means to satisfy a set of constraints.

**Definition 2.4** (Labeled graph satisfying a set of constraints). Let  $G = (V, E)$  be a  $(\mathcal{V}, \mathcal{E})$ -labeled graph for some finite sets of labels  $\mathcal{V}$  and  $\mathcal{E}$ . Let  $\mathcal{C}$  be an  $(r, \Delta)$ -set of constraints over  $(\mathcal{V}, \mathcal{E})$  for some finite  $r, \Delta \in \mathbb{N}$ . The graph  $G$  satisfies  $\mathcal{C}$  if the following statement is satisfied:

- For every node  $v \in V$ , the  $(\mathcal{V}, \mathcal{E})$ -labeled graph  $G[\mathcal{N}_r[v]]$  is such that  $(G[\mathcal{N}_r[v]], v) \in \mathcal{C}$ .

We are now ready to define the class of problems of interest.

**Definition 2.5** (Locally checkable labeling (LCL) problems). Let  $r, \Delta \in \mathbb{N}$  be constants. A locally checkable labeling (LCL) problem  $\Pi$  is a tuple  $(\mathcal{V}_{\text{in}}, \mathcal{E}_{\text{in}}, \mathcal{V}_{\text{out}}, \mathcal{E}_{\text{out}}, \mathcal{C})$  where  $\mathcal{V}_{\text{in}}, \mathcal{E}_{\text{in}}, \mathcal{V}_{\text{out}},$  and  $\mathcal{E}_{\text{out}}$  are finite label sets and  $\mathcal{C}$  is an  $(r, \Delta)$ -set of constraints over  $(\mathcal{V}_{\text{in}} \times \mathcal{V}_{\text{out}}, \mathcal{E}_{\text{in}} \times \mathcal{E}_{\text{out}})$ .

We now explain what it means to *solve* an LCL problem  $\Pi$ . We are given as input a  $(\mathcal{V}_{\text{in}}, \mathcal{E}_{\text{in}})$ -labeled graph  $G = (V, E)$ . For any node  $v \in V$  and half-edge  $(v, e) \in V \times E$ , let us denote the input label of  $v$  by  $\ell_{\text{in}}(v)$ , and the input label of  $(v, e)$  by  $\ell_{\text{in}}((v, e))$ . We want to produce an output labeling on  $G$  so that we obtain a  $(\mathcal{V}_{\text{out}}, \mathcal{E}_{\text{out}})$ -labeled graph. Let us denote by  $\ell_{\text{out}}(v)$  and by  $\ell_{\text{out}}((v, e))$  the output labels of  $v$  and  $(v, e)$ , respectively. Consider the  $(\mathcal{V}_{\text{in}} \times \mathcal{V}_{\text{out}}, \mathcal{E}_{\text{in}} \times \mathcal{E}_{\text{out}})$ -labeled graph  $G$  where the labeling is defined as follows: the label of each node  $v$  is  $\ell(v) = (\ell_{\text{in}}(v), \ell_{\text{out}}(v))$ , and the label of each half-edge  $(v, e)$  is  $\ell((v, e)) = (\ell_{\text{in}}((v, e)), \ell_{\text{out}}((v, e)))$ . Now, the  $(\mathcal{V}_{\text{in}} \times \mathcal{V}_{\text{out}}, \mathcal{E}_{\text{in}} \times \mathcal{E}_{\text{out}})$ -labeled graph  $G$  must satisfy the  $(r, \Delta)$ -set of constraints  $\mathcal{C}$  over  $(\mathcal{V}_{\text{in}} \times \mathcal{V}_{\text{out}}, \mathcal{E}_{\text{in}} \times \mathcal{E}_{\text{out}})$  according to Definition 2.4.

**The LOCAL model.** In the LOCAL model of computing, we are given a distributed system of  $n$  processors/nodes. The processors are connected through a communication network that is modeled by a graph  $G = (V, E)$ . Computation proceeds in synchronous rounds: in each round, nodes simultaneously exchange messages with their neighbors, perform some local computation, and update their states variables based on the received messages. Note that communication bandwidth is unconstrained (i.e., messages can be of any size) and nodes are capable of arbitrary local computation, and it is assumed that all processes are fault-free and messages cannot be corrupted. At the beginning of computation, all processors are identical and start with identical copies of the same state variables, except for a distinguished local variable  $x(v)$  which stores input data for node  $v$ . Input data for a node  $v$  encodes the number  $n$  of nodes in the network, a unique identifier from the set  $[n^c] = \{1, 2, \dots, n^c\}$  where  $c \geq 1$  is a fixed constant, and possible inputs defined by the problem of interest (we assume nodes store both input node labels and input half-edge labels). If computation is randomized,  $x(v)$  also encodes a private, infinite random bit string that is independent of the random bits of all other nodes. In this case, we refer to the model as the *randomized LOCAL model* as opposed to the *deterministic LOCAL model*. The goal of the computation is to assign to each node  $v$  an output label  $\ell_{\text{out}}(v)$  and computation ends when all nodes have decided on their output labels. The running time of an algorithm is the number of

communication rounds required to solve a problem. If computation is randomized, we also ask that the algorithm solves the problem of interest with probability at least  $1 - 1/\text{poly}(n)$ , where  $\text{poly } n$  is any polynomial function in  $n$ . Since computation and message size is unbounded, we can look at  $T$ -rounds LOCAL algorithms as functions mapping radius- $T$  neighborhoods of nodes to output labels (in the deterministic case) or probability distributions over output labels (in the randomized case). That is why we refer to the parameter  $T$  as the *locality* of the algorithm.

**The quantum-LOCAL model.** The quantum-LOCAL model of computing is similar to the deterministic LOCAL model above, where we replace the classical processors with quantum processors and the classical communication links with quantum communication links. More precisely, the quantum processors can manipulate local states consisting of an unbounded number of qubits with arbitrary unitary transformations, while the communication links are quantum communication channels, and the local outputs can be the result of any quantum measurement. As in the deterministic LOCAL model, adjacent nodes can exchange any number of qubits. We refer to the randomized LOCAL model of computing as *classical LOCAL*, as opposed to the quantum-LOCAL model of computing. Akin to the classical LOCAL model, also in quantum-LOCAL we ask that a problem is solved with probability at least  $1 - 1/\text{poly}(n)$ . This is just an intuitive definition of the model which is sufficient for our purposes, but the interested reader can find a formal definition in [GKM09].

### 3 Quantum advantage: high level ideas

In this section, we provide a high-level explanation of how the LCL problem  $\Pi$  that exhibits quantum advantage is defined, and we explain why its quantum and classical complexities differ. All the formal details are deferred to Section 5.

**Proper instances.** In Section 1.5, we explained how we would like a graph to be in order to have an instance that is easy for a quantum algorithm but hard for any classical algorithm. We call the family of graphs explained in Section 1.5 *proper instances*, i.e., these graphs are graphs that can be obtained by starting from an arbitrary (not necessarily connected) graph and then replacing each node, and each edge, with a tree-like construction.

**LCLs must be promise-free.** However, a standard LCL must be defined on *any* graph, that is, it must not require the promise that the given input graph satisfies some specific conditions. That is, our LCL problem  $\Pi$  must be well-defined even in graphs that do not look at all like proper instances. In particular, we want a problem that, on any graph, even on those that look completely different from a proper instance, should still be solvable in  $O(\log n)$  rounds by a quantum algorithm.

**Overview of the problem  $\Pi$ .** On a high level, our (promise-free) LCL problem  $\Pi$  will be defined such that it requires solving two problems, called  $\Pi^{\text{badGraph}}$  and  $\Pi^{\text{promise}}$ . More in detail, in  $O(\log n)$  classical deterministic rounds, nodes can produce a labeling that marks *invalid* parts of the graph, that is, parts of the graph that do not look like subgraphs of a proper instance. On the other hand, the problem will be defined such that marking as invalid the parts of the graph that are actually valid is not possible. We will formally define this labeling as an LCL called  $\Pi^{\text{badGraph}}$ . Then, the problem  $\Pi$  will require to solve some other problem  $\Pi^{\text{promise}}$  in parts of the graph that are not marked.

**How we define  $\Pi^{\text{promise}}$ .** The problem  $\Pi^{\text{promise}}$  will satisfy the following properties.

- It is an LCL *with promise*, that is, it is guaranteed that the input graph is a proper instance.
- In proper instances, the complexity is  $O(\log n)$  for quantum algorithms but  $\omega(\log n)$  for classical randomized algorithms.

**How we define  $\Pi^{\text{badGraph}}$ .** The problem  $\Pi^{\text{badGraph}}$  will satisfy the following properties.

- This problem is *promise-free*, that is, any input graph is allowed.
- There is a special output label for  $\Pi^{\text{badGraph}}$  called  $\perp$ .
- In  $O(\log n)$  classical deterministic rounds it is possible to produce a solution for  $\Pi^{\text{badGraph}}$  such that the subgraph induced by nodes labeled  $\perp$  is a proper instance.

**How we define  $\Pi$ .** Our LCL problem  $\Pi$  will then be defined as follows.

- Nodes need to solve  $\Pi^{\text{badGraph}}$ .
- On the subgraph induced by nodes that output  $\perp$  for  $\Pi^{\text{badGraph}}$ , nodes need to solve  $\Pi^{\text{promise}}$ .

**Quantum upper bound.** Then, we will show that our problem  $\Pi$  can be solved in  $O(\log n)$  quantum rounds, as follows.

- Nodes spend  $O(\log n)$  classical deterministic rounds to mark invalid parts of the graph, guaranteeing that the subgraph induced by nodes labeled  $\perp$  is a proper instance.
- Nodes spend  $O(\log n)$  quantum rounds to solve  $\Pi^{\text{promise}}$  in the subgraph induced by nodes labeled  $\perp$ .

**Classical lower bound.** Finally, we will show that our problem  $\Pi$  requires  $\omega(\log n)$  classical randomized rounds, as follows.

- As a lower bound graph, we consider a graph that is a proper instance. This implies that, on this graph, any solution for  $\Pi^{\text{badGraph}}$  must label all nodes  $\perp$ .
- The (proper) instance is constructed as follows. Let  $G$  be a lower bound graph for  $\text{GHZ}(\Delta)$ , where  $\Delta$  is an appropriately chosen degree, as a function of  $n$ . We replace nodes and edges with tree-like constructions, as follows.
  - Tree-like constructions used to replace *nodes* will have an appropriate height to guarantee that they have  $\Delta$  leaves.
  - Tree-like constructions used to replace *edges* will have height  $\Theta(\log n)$ .
- We show that if  $\Pi$  can be solved in  $O(\log n)$  randomized classical rounds on these graphs, then we reach a contradiction with the lower bound for  $\text{GHZ}(\Delta)$ . For this purpose, we will exploit the fact that, informally, each edge has been stretched by a  $\Theta(\log n)$  factor.

### 3.1 The tree-like gadget

A basic building block for our construction is a structure called *tree-like gadget*, that has already been used in different ways in several works related to LCLs (see, e.g., [BBO+20, BGK+25, BCM+21]). An example of tree-like gadget is shown in Figure 1. In Section 5.1, we will formally define tree-like gadgets and state their properties.

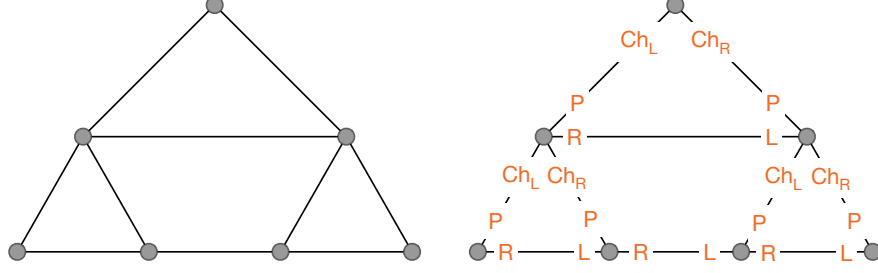


Figure 1: On the left, a tree-like gadget. On the right, a tree-like gadget labeled with labels from  $\Sigma^{\text{tree}}$  such that the constraints  $C^{\text{tree}}$  are satisfied.

**Local checkability of tree-like gadgets.** Informally, a tree-like gadget is a perfect binary tree in which all nodes that are at the same depth are connected via a path. It is known from previous work that this structure is *locally checkable*, that is, there exist a set of labels  $\Sigma^{\text{tree}}$  and a set of constraints  $C^{\text{tree}}$  (checkable by inspecting the  $O(1)$ -radius neighborhood of each node) satisfying the following properties.

- Any tree-like gadget can be labeled with labels from  $\Sigma^{\text{tree}}$  such that the constraints  $C^{\text{tree}}$  are satisfied on all nodes.
- For any connected graph that is not a tree-like gadget, and for any labeling from  $\Sigma^{\text{tree}}$ , there exists at least one node for which the constraints  $C^{\text{tree}}$  are not satisfied.

**The LCL problem  $\Pi^{\text{badTree}}$ .** While local checkability is a very useful property, in order to define our problem we need an even stronger property: if there is an error *somewhere in the graph* we want *all nodes to be able to quickly detect the error*. In previous works, it is shown that tree-like gadgets indeed satisfy this stronger property. More in detail, there exists an LCL problem  $\Pi^{\text{badTree}}$  satisfying the following properties.

- There is a special output label for  $\Pi^{\text{badTree}}$ , called  $\perp$ .
- For any connected graph  $G$ , there exists a solution for  $\Pi^{\text{badTree}}$  satisfying that, if  $G$  is not a tree-like gadget, then no node of  $G$  is labeled  $\perp$ .
- Such a solution can be computed in  $O(\log n)$  deterministic classical rounds.

On a high level, the problem  $\Pi^{\text{badTree}}$  is defined as follows.

- Labels from  $\Sigma^{\text{tree}}$  are an input for the problem  $\Pi^{\text{badTree}}$ .
- Nodes that do not satisfy the constraints  $C^{\text{tree}}$  can directly output an error.
- It is possible to prove that, if there is an error somewhere, all other nodes must see an error within distance  $O(\log n)$ , and  $\Pi^{\text{badTree}}$  allows outputting *pointers* to prove that there is an error somewhere.
- The constraints on the output of  $\Pi^{\text{badTree}}$  are carefully defined so that, on tree-like gadgets that are properly labeled with labels from  $\Sigma^{\text{tree}}$  (i.e., the constraints  $C^{\text{tree}}$  are satisfied on all nodes), *no cheating is allowed*, that is, it is not possible to output pointers in some carefully crafted way that gives a valid output for  $\Pi^{\text{badTree}}$ .

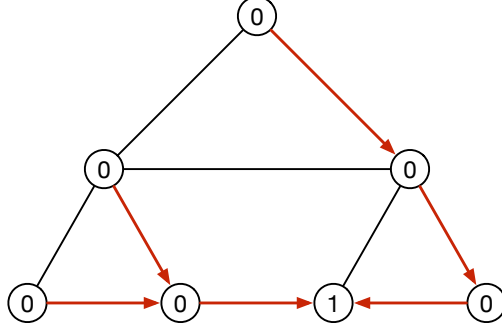


Figure 2: An example of solution of  $\Pi^{\text{badTree}}$  in which, since one node is marked (i.e., it has input 1), all nodes output something different from  $\perp$  (i.e., they all output *pointers*). Informally, the problem  $\Pi^{\text{badTree}}$  allows outputting pointers, and in order to not allow cheating (i.e., by creating local cycles with pointers), some specific sequences of pointers are forbidden in the definition of  $\Pi^{\text{badTree}}$ . For example, a pointer going up or down is not allowed to appear after a pointer going left or right, and pointing to the left child is always forbidden.

- Nodes that do not output errors or pointers must output  $\perp$ .

For reasons that will become clear later, nodes will have an additional input for  $\Pi^{\text{badTree}}$ , which is either 0, i.e., unmarked nodes, or 1, i.e., marked nodes, and  $\Pi^{\text{badTree}}$  is defined such that marked nodes are always allowed to output an error. An example of valid solution of  $\Pi^{\text{badtree}}$  is depicted in Figure 2.

**How we will use  $\Pi^{\text{badTree}}$ .** We will define a more complex gadget, called *octopus*, which consists of many tree-like structures connected in a specific way. We will define a problem  $\Pi^{\text{badOctopus}}$  that allows to mark invalid octopus gadgets. In the definition of  $\Pi^{\text{badOctopus}}$ , we will use  $\Pi^{\text{badTree}}$  as a subproblem. Informally, our final problem  $\Pi$  will satisfy that, if the adversary really wants nodes to solve  $\Pi^{\text{promise}}$  (i.e., to make the problem non-trivial), then it needs to properly label tree-like gadgets and octopus gadgets, so that the output for the problems  $\Pi^{\text{badTree}}$  and  $\Pi^{\text{badOctopus}}$  needs to be  $\perp$  everywhere.

### 3.2 The octopus gadget

In Section 5.2, we will formally define the notion of *octopus gadget*, and we will prove some useful properties of this object. Informally, an octopus gadget is a graph that can be obtained as follows. Let  $\Delta$  be an integer power of 2. Start with a tree-like gadget  $G_0$  that has  $\Delta$  leaves, and with additional  $\Delta$  tree-like gadgets  $G_1, \dots, G_\Delta$ . For each  $1 \leq i \leq \Delta$ , add an edge between the  $i$ -th leaf of the gadget  $G_0$  and the root of the gadget  $G_i$ . The tree-like gadget  $G_0$  is called *head* gadget, while all the other tree-like gadgets are called *port* gadgets. The formal definition of octopus gadget in Section 5.2 will be more general: to each leaf of the head gadget we will connect either 1 or 2 port gadgets. In this way, we will be able to handle also cases in which  $\Delta$  is not a power of 2. An example of octopus gadget is depicted in Figure 3.

**Local checkability of octopus gadgets.** We will prove that, similarly to the case of tree-like gadgets, also octopus gadgets are locally checkable. More in detail, we will prove that there exists a set of input labels  $\Sigma^{\text{octopus}}$  and a set of constraints  $C^{\text{octopus}}$  over these labels satisfying the following properties.

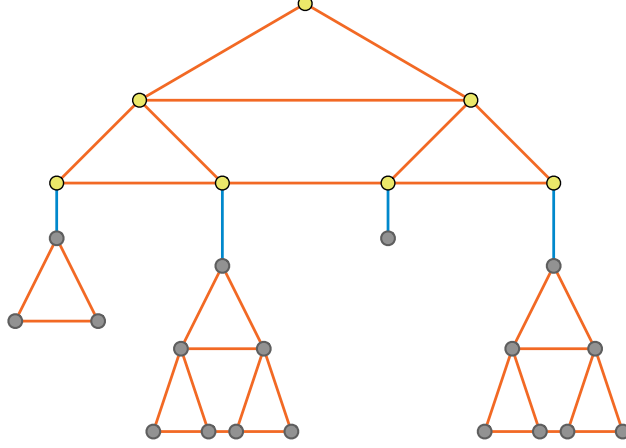


Figure 3: An example of octopus gadget. The connected component induced by yellow nodes and their incident orange edges is the head gadget. Each connected component induced by gray nodes and their incident orange edges is a port gadget. Blue edges properly connect these tree-like gadgets to create a proper octopus gadget. Nodes and edges will be labeled such that this structure is locally checkable. For example, blue edges will have special labels to denote that they do not belong to the tree-like gadgets.

- Any octopus gadget can be labeled with labels from  $\Sigma^{\text{octopus}}$  such that the constraints  $C^{\text{octopus}}$  are satisfied on all nodes.
- For any connected graph that is not an octopus gadget, and for any labeling from  $\Sigma^{\text{octopus}}$ , there exists at least one node for which the constraints  $C^{\text{octopus}}$  are not satisfied.

In order to define  $\Sigma^{\text{octopus}}$  and  $C^{\text{octopus}}$ , we will extend  $\Sigma^{\text{tree}}$  and  $C^{\text{tree}}$ , as follows.

- Each edge will be additionally labeled to denote whether the edge is part of a tree-like gadget, i.e., *internal edge*, or if it is an edge connecting different tree-like gadgets, i.e., *external edge*.
- Each node will be additionally labeled to denote whether it is part of the head gadget or if it is part of a port gadget.
- $C^{\text{octopus}}$  will be defined such that, in the subgraph induced by internal edges, the constraints  $C^{\text{tree}}$  must hold, and such that nodes belonging to the same tree-like gadget must either be all labeled head or all labeled port.
- $C^{\text{octopus}}$  will contain additional constraints to make sure that external edges are properly connected.

**The LCL problem  $\Pi^{\text{badOctopus}}$ .** As in the case of tree-like gadgets, we need the stronger property that, if there is some error somewhere, then all nodes are able to prove this fact in  $O(\log n)$  deterministic classical rounds, such that no cheating is allowed (i.e., in properly labeled octopus gadgets, the output must be  $\perp$  everywhere). For this purpose, we define an LCL problem  $\Pi^{\text{badOctopus}}$ . On a high level, this problem is defined by requiring nodes to solve  $\Pi^{\text{badTree}}$  *multiple times*, each time having potentially more *marked* nodes (and hence allowed to directly output *error*). More in detail, we define  $\Pi^{\text{badOctopus}}$  as follows.

- Nodes that do not satisfy some of the constraints of  $C^{\text{octopus}}$  are *marked*.

- Nodes need to solve  $\Pi^{\text{badTree}}$  in the subgraph induced by internal edges. Observe that it could be the case that some node is in a valid tree-like gadget, but some constraint of  $C^{\text{Octopus}}$  is not satisfied on the node (e.g., because some external edge is wrongly connected). In this case, since the node is marked, it is allowed to produce an error when solving  $\Pi^{\text{badTree}}$ . Let  $O_1$  be the output labeling produced by the nodes at this point.
- Nodes that are labeled to be part of a *head* gadget need to solve  $\Pi^{\text{badTree}}$  *again*. This time, however, a node is marked also if it has an incident external edge connecting it to a node that did not output  $\perp$  in  $O_1$ . Let  $O_2$  be the output labeling produced by the nodes at this point.
- Nodes that are labeled to be part of a *port* gadget need to solve  $\Pi^{\text{badTree}}$  *again*. Similarly as before, this time, a node is marked also if it has an incident external edge connecting it to a node that did not output  $\perp$  in  $O_2$ .

Thanks to our triple usage of  $\Pi^{\text{badTree}}$ , we will be able to handle bad cases like the following. Assume that there is a broken port gadget connected to a valid head gadget, and all other port gadgets connected to the head gadget are valid. In this case, we want  $\Pi^{\text{badOctopus}}$  to allow a solution in which no node outputs  $\perp$ . Indeed,  $\Pi^{\text{badOctopus}}$  allows the following solution.

- In the first instance of  $\Pi^{\text{badTree}}$ , all nodes part of the broken port gadget can output something different from  $\perp$ , while all other nodes output  $\perp$ .
- In the second instance of  $\Pi^{\text{badTree}}$ , the node of the head gadget that is connected to the broken gadget will be marked, and hence all nodes of the head gadget can output something different from  $\perp$ .
- In the third instance of  $\Pi^{\text{badTree}}$ , all the roots of the port gadgets will be marked, and hence all nodes can output something different from  $\perp$ .

In fact, we will prove that in *all* graphs that are not a valid octopus gadget, nodes can compute a solution in  $O(\log n)$  deterministic classical rounds such that no node outputs  $\perp$ , while in valid octopus gadgets, we still maintain the property that the only allowed output is  $\perp$  everywhere. Figure 4 shows an example of a solution of  $\Pi^{\text{badOctopus}}$  in a broken octopus gadget.

Similarly as in the case of  $\Pi^{\text{badTree}}$ , also in the case of  $\Pi^{\text{badOctopus}}$  nodes will have an additional input for  $\Pi^{\text{badOctopus}}$ , which is either 0, i.e., unmarked nodes, or 1, i.e., marked nodes, and  $\Pi^{\text{badOctopus}}$  is defined such that nodes that are marked for  $\Pi^{\text{badOctopus}}$  are also marked for  $\Pi^{\text{badTree}}$ . We will use this fact later when defining  $\Pi^{\text{badGraph}}$ .

### 3.3 The family of proper instances

In Section 5.3, we will formally define the family of *proper instances*. Informally, a proper instance is a graph  $G'$  that can be obtained as follows. Let  $G = (U, V, E)$  be a bipartite graph that is not necessarily connected and that could contain parallel edges. Assume each node  $u \in U$  has a degree that is an integer power of 2 (we will get rid of this assumption, and only require  $u$  to not have degree 0, in Section 5.3). For each node  $u \in U$ , we put an octopus gadget  $g(u)$  in  $G'$ , and for each node  $v \in V$ , we put a node  $f(v)$  in  $G'$ . These latter types of nodes are called *inter-octopus nodes*. Then, let  $\{u, v\}$  be the  $i$ -th edge incident to  $u$ , for some given order. We add an edge in  $G'$  between the left-most leaf of the  $i$ -th port gadget of  $g(u)$  and  $f(v)$ . We call these edges *inter-octopus edges*. Figure 5 shows an example of a proper instance.

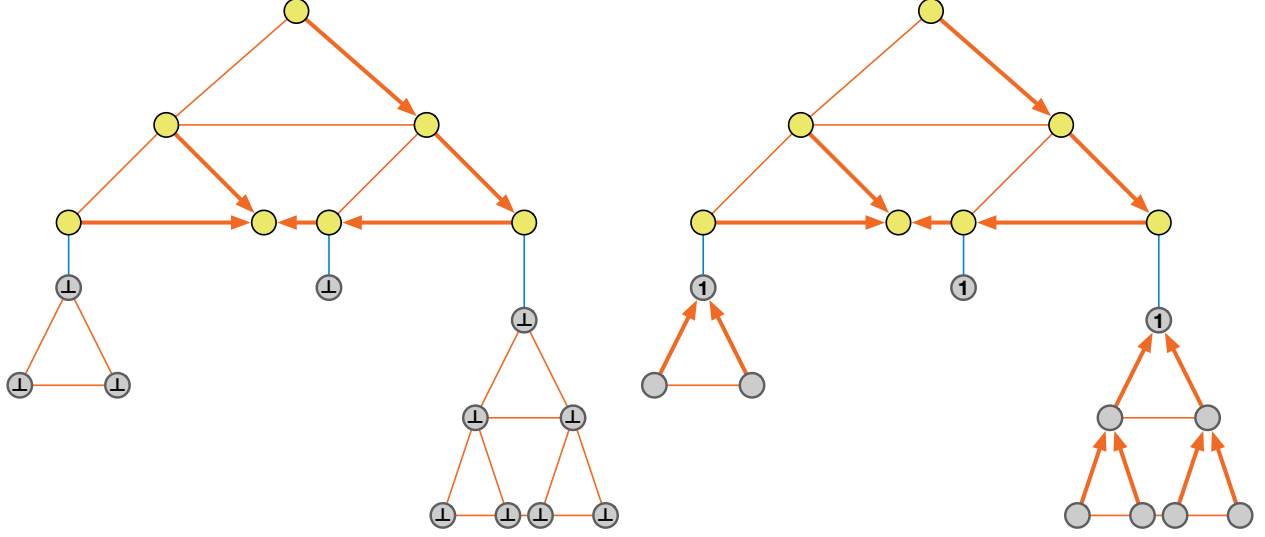


Figure 4: This graph contains a yellow node that has no children (i.e., no incident edges labeled  $\text{Ch}_L$  and  $\text{Ch}_R$ ) and is missing a blue edge, i.e., this graph is not a valid octopus gadget. On the left, the output of the nodes in the first instance of  $\Pi^{\text{badTree}}$  is shown, where port gadgets do not see any error and output  $\perp$ , but the nodes that are part of the broken head gadget prove that there is some error by using pointers. On the right, the output of the nodes in the next instance of  $\Pi^{\text{badTree}}$  is shown, where now the roots of the port gadgets are marked, and hence all the nodes in the port gadgets can now prove that there is an error.

**Local checkability of proper instances.** Similarly as in the case of tree-like gadgets and octopus gadgets, we will define a set of input labels  $\Sigma^{\text{proper}}$  and a set of constraints  $C^{\text{proper}}$  satisfying the following properties.

- Any proper instance can be labeled with labels from  $\Sigma^{\text{proper}}$  such that the constraints  $C^{\text{proper}}$  are satisfied on all nodes.
- For any connected graph that is not a proper instance, and for any labeling from  $\Sigma^{\text{proper}}$ , there exists at least one node for which the constraints  $C^{\text{proper}}$  are not satisfied.

Similarly as in the case of octopus gadgets, in order to define  $\Sigma^{\text{proper}}$  and  $C^{\text{proper}}$ , we will extend  $\Sigma^{\text{octopus}}$  and  $C^{\text{octopus}}$ . For example, we will use additional labels for labeling inter-octopus edges. These edges will have additional constraints, to ensure that they are properly connected to octopus gadgets and to inter-octopus nodes.

**The LCL problem  $\Pi^{\text{badGraph}}$ .** Similarly as in the case of tree-like gadgets and octopus gadgets, we would like nodes to be able to quickly prove that the graph is not a proper instance, if that is the case. Unfortunately, this seems to be not possible. However, a weaker guarantee is sufficient. We will define a problem  $\Pi^{\text{badGraph}}$  satisfying the following properties.

- There is a special output label for  $\Pi^{\text{badGraph}}$ , called  $\perp$ .
- For any graph  $G$ , there exists a solution for  $\Pi^{\text{badGraph}}$  satisfying that the subgraph induced by nodes labeled  $\perp$  is a proper instance.
- Such a solution can be computed in  $O(\log n)$  deterministic classical rounds.

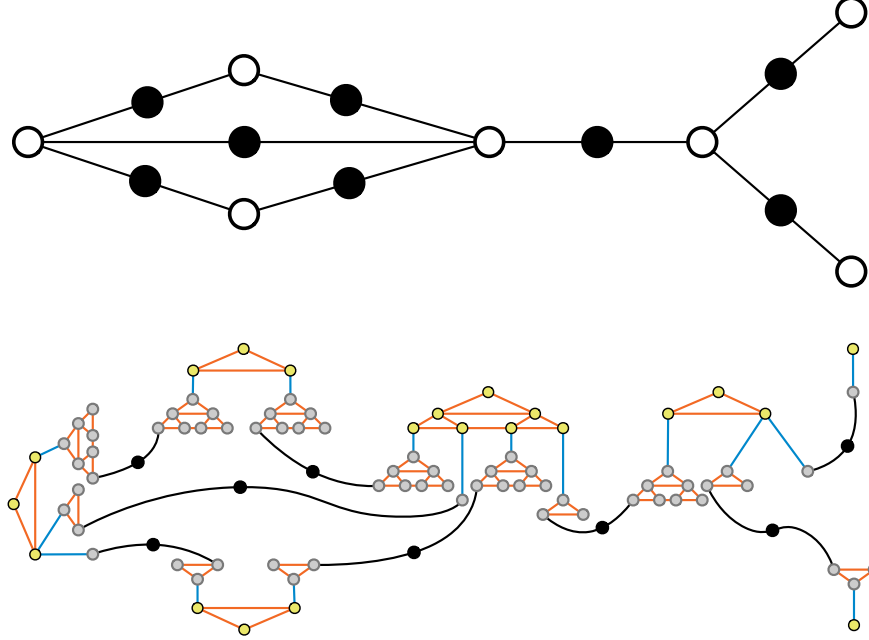


Figure 5: A graph  $G$  (above) and a possible proper instance  $G'$  (below) that can be constructed starting from  $G$ . Black nodes in  $G'$  correspond to inter-octopus nodes, and black edges correspond to inter-octopus edges.

This problem will be defined by first marking nodes that do not satisfy  $C^{\text{proper}}$ , and then requiring the nodes to solve  $\Pi^{\text{badOctopus}}$  on the subgraph induced by nodes that are not inter-octopus. Moreover, there will be a special output allowed for inter-octopus nodes that do not satisfy  $C^{\text{proper}}$ .

### 3.4 The LCL problem $\Pi$

We will formally define our LCL problem  $\Pi$  in Section 5.5. Informally, it is defined as follows.

- The input given to  $\Pi$  becomes an input for the problem  $\Pi^{\text{badGraph}}$ .
- Nodes need to solve  $\Pi^{\text{badGraph}}$ .
- Let  $G'$  be the subgraph induced by the nodes outputting  $\perp$ . On  $G'$ , nodes need to solve  $\Pi^{\text{promise}}$ .

In other words, on a high level, we use  $\Pi^{\text{badGraph}}$  to transform the promise-problem  $\Pi^{\text{promise}}$  into a promise-free LCL problem  $\Pi$ . That is, all the machinery that we have defined until this point are what we use to make  $\Pi^{\text{promise}}$  promise-free. More in detail, everything is defined so that on proper instances, the only valid solution for  $\Pi^{\text{badGraph}}$  is the one assigning  $\perp$  to all nodes, and hence it is required to solve  $\Pi^{\text{promise}}$  on all nodes. Hence, a worst-case instance for  $\Pi^{\text{promise}}$  can be translated into a worst-case instance for  $\Pi$ .

In order to make  $\Pi^{\text{proper}}$  promise-free, we pay a cost: it takes  $O(\log n)$  rounds to mark invalid parts of the graph, and hence to restrict to a subgraph that is a proper instance. Since this cost is still lower than the randomized classical complexity of  $\Pi^{\text{promise}}$ , we are still able to prove a promise-free separation between quantum and classical randomized.

### 3.5 The problem $\Pi^{\text{promise}}$

Before giving an overview of the problem  $\Pi^{\text{promise}}$ , we first discuss a family of problems that can be used, as a starting point, in order to define  $\Pi^{\text{promise}}$ . In fact, our construction does not only work for a specific problem, but it works for an entire *family* of problems that we call *linearizable* problems. A linearizable problem  $\Pi^{\text{linearizable}} = (\Sigma, (F, L, P), B)$  is defined as follows.

**Definition 3.1** (Linearizable problem). Let  $H$  be a hypergraph, and let  $G$  be its bipartite incidence graph. Let the nodes of  $G$  corresponding to the nodes of  $H$  be called *white* nodes, and let the nodes of  $G$  corresponding to the hyperedges of  $H$  be called *black* nodes.

- The task is to label each edge of  $G$  with a label from some finite set  $\Sigma$ .
- There is a list of allowed *black node configurations*  $B$ , which is a list of multisets of labels from  $\Sigma$  that describes valid labelings of edges incident on a black node. We say that a black node *satisfies the black constraint* if its incident edges are labeled in a valid way. It is assumed that the rank of  $H$ , and hence the maximum degree of black nodes, is a constant.
- Constraints on white nodes are described as a triple  $(F, L, P)$ , where  $F$  (which stands for *first*) and  $L$  (which stands for *last*) are finite sets of labels, and  $P$  (which stands for *pairs*) is a finite set of ordered pairs of labels. In this formalism, it is assumed that an ordering on the incident edges of a white node is given, and it is required that:
  - The first edge is labeled with a label from  $F$ ;
  - The last edge is labeled with a label from  $L$ ;
  - Each pair of consecutive edges must be labeled with a pair of labels from  $P$ .

We say that a white node *satisfies the white constraint* if its incident edges are labeled in a valid way.

When solving a linearizable problem in the distributed setting, it is assumed that each node knows whether it is white or black.

An example of a definition of a problem using this formalism is shown in Figure 6. The useful feature of this family of problems is that, even if the labels of a node are laid down on a (possibly very long) path, it is still possible to locally check for correctness. This property will be crucial for defining  $\Pi^{\text{promise}}$ . Moreover, as discussed in Section 1, the *family* of problems  $\text{GHZ}(\Delta)$  can be described as a *single* linearizable problem.

We are now ready to discuss  $\Pi^{\text{promise}}$ . Let  $\Pi^{\text{linearizable}} = (\Sigma, (F, L, P), B)$  be a linearizable problem. The problem  $\Pi^{\text{promise}}$  is defined as a function of the chosen problem  $\Pi^{\text{linearizable}}$ , as follows. While in Section 5.4 we will handle the case in which each leaf of the head gadgets is connected to either 1 or 2 port gadgets, in the following we consider the simpler setting in which each leaf of the head gadgets is connected to a single port gadget.

- Each port gadget of our construction corresponds to an edge of  $G$ . We require all nodes of each port gadget to output a label of  $\Pi^{\text{linearizable}}$ , with the requirement that all nodes of a port gadget are labeled with the same label.
- In our construction, nodes corresponding to black nodes of  $G$  are marked in a special way, and hence they know to be nodes corresponding to black nodes. Let  $b$  be one such node, and let  $M$  be the multiset containing the output labels of the neighboring nodes (which are nodes belonging to port gadgets) of  $b$ . We require  $M$  to be a configuration allowed by the black constraint of  $\Pi^{\text{linearizable}}$ .

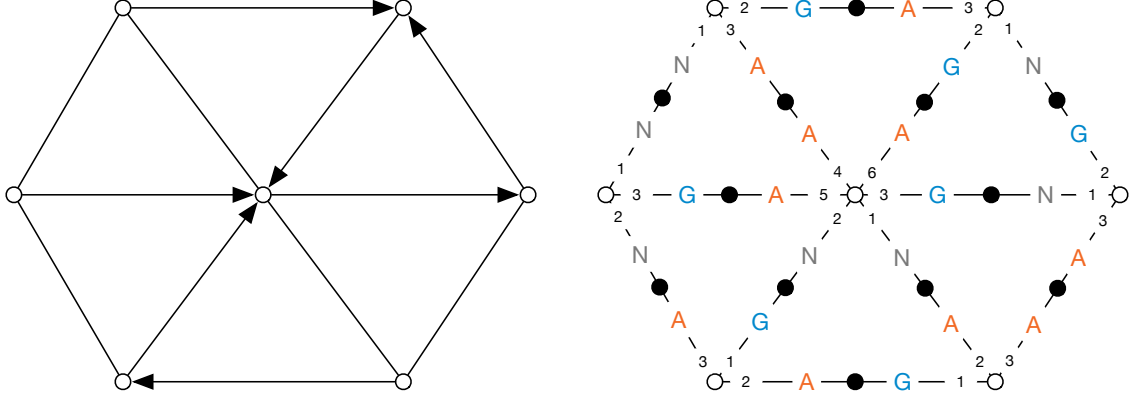


Figure 6: In this example we have hyperedges of rank 2, i.e., black nodes have degree exactly 2. We consider the problem of *edge grabbing*, where we need to orient some edges such that each white node has exactly one outgoing (or grabbed) edge. On the left, it is depicted an example of a solution for this problem. On the right, we show the same solution encoded as a solution for an equivalent linearizable problem. The label  $G$  stands for “grabbed”, the label  $N$  stands for “non grabbed”, the label  $A$  stands for “already grabbed some previous edge”. The constraints on black nodes are  $\{\{N, N\}, \{N, A\}, \{N, G\}, \{G, A\}, \{A, A\}\}$ , that is, we do not allow white nodes to grab the same edge (i.e., we forbid  $\{G, G\}$ ). The constraints on white nodes are determined by the triple  $(F, L, P)$ , where  $F = \{N, G\}$ ,  $L = \{G, A\}$ ,  $P = \{(N, N), (N, G), (G, A), (A, A)\}$ . For example, the configuration of the central white node, ordered according to the ordering of its incident edges, is  $(N, N, G, A, A, A)$ , where  $N \in F$ ,  $A \in L$ , and each consecutive pair of labels is in  $P$ .

- Thanks to the input of the problem  $\Pi$ , leaves of a head gadget know that they are leaves of a head gadget, and additionally, the left-most leaf  $v$  and the right-most leaf  $u$  know that they are, respectively, the left-most and the right-most leaves. Let  $f$  be the output label of the port node connected to  $v$ , let  $\ell$  be the output label of the port node connected to  $u$ , and let  $o_i$  be the output label of the port node connected to the  $i$ -th leaf. We require that  $f \in F$ , that  $\ell \in L$ , and that each pair  $(o_i, o_{i+1})$  is in  $P$ .

Thanks to these constraints, we have that, in a valid labeling, each octopus gadget encodes a labeling that satisfies the white constraint of  $\Pi^{\text{linearizable}}$ . We will give all the details about  $\Pi^{\text{promise}}$  in Section 5.4.

### 3.6 The complexity of $\Pi$

We now provide an overview on how we will prove lower and upper bounds for the problem  $\Pi$ . Since our goal is to prove a separation between quantum-LOCAL and classical randomized-LOCAL, our upper bound will be a quantum algorithm, while the lower bound will hold for classical randomized algorithms.

**Upper bound.** In the case of upper bounds, we will assume the existence of a quantum algorithm  $\mathcal{A}$  for  $\Pi^{\text{linearizable}}$  with some complexity  $T$  (for example, in the case of  $\text{GHZ}(\Delta)$ , we know that  $T = 1$ ), and we will show that, with  $O(\log n)$  overhead, it can be converted into an algorithm  $\mathcal{B}$  that solves  $\Pi$ . On a high level, algorithm  $\mathcal{B}$  works as follows.

- First, nodes spend  $O(\log n)$  deterministic classical rounds to solve  $\Pi^{\text{badGraph}}$ . The result will be a labeling satisfying that nodes labeled  $\perp$  induce a proper instance.

- Consider the graph  $G'$  obtained by contracting each octopus gadget into a single node. The graph  $G'$  is an instance of  $\Pi^{\text{linearizable}}$ , and each quantum communication round on  $G'$  can be simulated with  $O(\log n)$  overhead in the original graph. Hence, it is possible to solve  $\Pi^{\text{linearizable}}$  in the original graph in  $O(T \log n)$  quantum rounds.
- Once  $\Pi^{\text{linearizable}}$  is solved, nodes can spend  $O(\log n)$  additional rounds to produce a labeling that is valid for  $\Pi^{\text{promise}}$  on the subgraph induced by nodes labeled  $\perp$  for  $\Pi^{\text{badGraph}}$ .

Therefore, algorithm  $\mathcal{B}$  solves  $\Pi$  in  $O(T \log n)$  quantum rounds. The details are given in Section 5.6.

**Lower bound.** Suppose we know that the problem  $\Pi^{\text{linearizable}}$  requires  $\Omega(T)$  classical randomized rounds. We prove that, if  $\Pi$  can be solved in  $o(T \log n)$  classical randomized rounds, then  $\Pi^{\text{linearizable}}$  can be solved in  $o(T)$  classical randomized rounds, violating the lower bound. This result will be achieved by showing the following. Suppose for a contradiction that there exists an algorithm  $\mathcal{B}$  that solves  $\Pi$  in  $o(T \log n)$  classical randomized rounds, we construct an algorithm  $\mathcal{A}$  for  $\Pi^{\text{linearizable}}$  as follows.

- Let  $G$  be an instance of  $\Pi^{\text{linearizable}}$ . Nodes construct a virtual graph  $G'$ , which is an instance of  $\Pi$ , by imagining themselves to be a valid octopus gadget where each port gadget has height  $\Theta(\log n)$ , and imagining each (hyper)edge to be a black node connecting different gadgets.
- In this virtual graph, the only solution for  $\Pi^{\text{badGraph}}$  is the one assigning  $\perp$  to all nodes.
- By assumption, the problem  $\Pi^{\text{promise}}$  can be solved in  $T' = o(T \log n)$  rounds in  $G'$ , and the  $T'$ -radius neighborhood of each node in  $G'$  is contained in  $o(T)$ -radius neighborhood of each node in  $G$ . Hence, nodes can solve  $\Pi^{\text{promise}}$  by communicating for only  $o(T)$  rounds on  $G$ , and then obtain a solution for  $\Pi^{\text{linearizable}}$ .

There are some details that we need to take care of, like the fact that the number of nodes in  $G$  and  $G'$  are different, and we will show how to handle all these details in Section 5.7.

## 4 Upper bounding quantum advantage

In this section, we prove Theorem 1.2. We start by defining *T-dependent distributions* and the *bounded-dependence model* using *outcomes*. In Section 4.1, we follow the outline of Section 1.6 and provide and prove lemmas for each of the intermediate steps. We then combine these lemmas to prove Theorem 1.2.

**Preliminary definitions.** We start by introducing some key definitions. Let  $G$  and  $H$  be two graphs. We define the *union* of  $G$  and  $H$  to be the graph  $G \cup H = (V, E)$  where  $V = V(G) \cup V(H)$  and  $E = E(G) \cup E(H)$ . The *intersection* is similarly defined, with  $G \cap H = (V, E)$  where  $V = V(G) \cap V(H)$  and  $E = E(G) \cap E(H)$ . The *graph difference* is defined as  $G \setminus H = (V, E)$  where  $V = V(G)$  and  $E = E(G) \setminus E(H)$ ; note that here we only take the difference of the edge sets. We define the *ring neighborhood* between  $T_1$  and  $T_2$  of a node  $u \in V$  (for  $T_1 \leq T_2$ ) as  $\mathcal{N}_{T_2}^{T_1}[u] = \mathcal{N}_{T_2}[u] \setminus \mathcal{N}_{T_1}[u]$ . We use an analogous notation for subsets of nodes. Consider any node  $u \in V$  (or any subset  $S \subseteq V$ ): with an abuse of notation, we define the *open induced subgraph* as the graph  $\mathring{G}[\mathcal{N}_T[u]] = G[\mathcal{N}_T[u]] \setminus G[\mathcal{N}_T^{T-1}[u]]$  (or  $\mathring{G}[\mathcal{N}_T[S]] = G[\mathcal{N}_T[S]] \setminus G[\mathcal{N}_T^{T-1}[S]]$ ). In practice, in this definition we are removing from the classical notion of neighborhood the edges that connect nodes that are at distance  $T$  from  $u$  (or  $S$ ) as the graph that  $u$  (or  $S$ ) *sees* by moving  $T$  hops away does not include them.

For any function  $f : A \rightarrow B$  and any subset  $A' \subseteq A$ , we denote by  $f \upharpoonright_{A'}$  the function  $g : A' \rightarrow B$  such that  $f(x) = g(x)$  for all  $x \in A'$ . For any input distributed network  $(G, x)$  to any problem, where  $x(v)$  encodes input data for all  $v \in V(G)$ , and any subset of nodes  $S \subseteq V(G)$ , the *radius- $T$  view* of  $S$  is  $\mathcal{V}_T(S) = (\dot{G}[\mathcal{N}_T[S]], x \upharpoonright_{\mathcal{N}_T[S]})$ . Basically,  $\mathcal{V}_T(S)$  includes everything that can be *seen* by the nodes in  $S$  with  $T$  rounds of communication, including input data (degree, ports, identifiers and input labels—if any, etc.). Suppose  $G$  has  $n$  nodes, and fix any subset of nodes  $S \subseteq V(G)$ . Given any two graphs  $G, H$  with inputs  $x_G, x_H$  and any two subset of nodes  $S_G \subseteq V(G)$  and  $S_H \subseteq V(H)$ , it is natural to define the notion of *isomorphism between views*. We say that  $\mathcal{V}_T(S_G)$  is isomorphic to  $\mathcal{V}_T(S_H)$  if there exists a function  $\varphi : V(G) \rightarrow V(H)$  such that the following holds:

1.  $\varphi \upharpoonright_{S_G}$  is an isomorphism between  $G[S_G]$  and  $H[S_H]$ ;
2.  $\varphi \upharpoonright_{\mathcal{N}_T[S_G]}$  is an isomorphism between  $\dot{G}[\mathcal{N}_T[S_G]]$  and  $\dot{H}[\mathcal{N}_T[S_H]]$ ;
3.  $x_G(u) = x_H(\varphi(u))$  for all  $u \in \mathcal{N}_T(S_G)$ .

Consider any  $T$ -round (deterministic or randomized) LOCAL algorithm  $\mathcal{A}$  run by the nodes of  $G$ , and any subset of nodes  $S \subseteq V(G)$ . Run  $\mathcal{A}$  on  $G$  for  $T$  rounds and look at the output distribution over  $S$ . Clearly, such distribution depends only on  $\mathcal{V}_T(S)$ , while everything that is outside  $\mathcal{V}_T(S)$  cannot influence the output distribution, even if some adversary changes the graphs and the inputs outside  $\mathcal{V}_T(S)$  (apart from the number of nodes, as this is an input to the algorithm). The *cause* of outputs in  $S$  *cannot be influenced* by any action of an adversary outside  $\mathcal{V}_T(S)$ .

This property is the so-called *no-signaling from the future* principle in physics, which states that no signals can be sent from the future to the past, and is equivalent to the *causality principle* [DCP16]. Such principle holds in *every physical distributed network* running any kind of synchronous distributed algorithm, including quantum ones.

To see how this principle formally translates in our setting, let us define the notion of *outcome*, which is some kind of generalization of an algorithm. Here, we restrict to finite sets of input and output labels since we focus on LCL problems. For a graph  $G$ , let us denote by  $\mathcal{H}(G) \subseteq V(G) \times V(G) \times E(G)$  the set containing all tuple of the form  $(v, (v, e))$  where  $(v, e) \in V(G) \times E(G)$  and  $(v, e)$  is a half-edge incident to  $v$ . Let us define the notion of *labeling function* for a graph  $G$  as follows: a function  $\text{out}_i : \mathcal{H}(G) \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}$  is a labeling function if, for two different half-edges  $(v, e), (v, e')$  of the same node  $v$ ,  $\text{out}(v, (v, e))[1] = \text{out}(v, (v, e'))[1]$ , where by  $\text{out}(v, (v, e))[1]$  we mean the first component of the tuple  $\text{out}(v, (v, e))$ . In words, the labeling function must assign to a node  $v$  only one possible label. For any node  $v$ , the label assigned to  $v$  is any  $\text{out}(v, (v, e))[1]$  for any half-edge  $(v, e)$  incident to  $v$ . The label assigned to a half-edge  $(v, e)$  is  $\text{out}(v, (v, e))[2]$ . With an abuse of notation, if  $v$  has no incident edges we still assume that the writing  $\text{out}(v)$  is well-defined and only assigns a label from  $\mathcal{V}_{\text{out}}$  to  $v$ .

**Definition 4.1** (Outcome). Let  $\mathcal{V}_{\text{in}}, \mathcal{E}_{\text{in}}, \mathcal{V}_{\text{out}}, \mathcal{E}_{\text{out}}$  be finite sets of labels. Let  $\mathcal{I}$  be the family of *all* input distributed networks  $(G, x)$ , where  $G$  is a  $(\mathcal{V}_{\text{in}}, \mathcal{E}_{\text{in}})$ -labeled graph and  $x$  encodes input data (identifiers, ports, possible random bits, input labels, etc.). An *outcome*  $O$  is a function that maps every input distributed network  $(G, x) \in \mathcal{I}$  to distribution over output labelings  $\{(\text{out}_i, p_i)\}_{i \in J}$ , where  $\{(\text{out}_i, p_i)\}_{i \in J}$  is defined as follows:  $J$  is a finite set of indices. The function  $\text{out}_i : \mathcal{H}(G) \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}$  is a labeling function that we call *output labeling function*. The value  $p_i$  is the probability that  $\text{out}_i$  occurs.

This definition is easily generalizable to the case of infinite label sets, but we avoid it for the sake of simplicity. Notice that all synchronous distributed algorithms yield outcomes: it is just the assignment of an output distribution (the one that is the result of the algorithmic procedure) to the input graph. We remark we have defined the domain set of outcomes to include *all possible graphs*.

This is not restrictive: Even classical (or quantum) algorithms can run on every possible graph. One can just introduce some garbage output label so that whenever a node running an algorithm needs to do something that is not well-defined (given its local view), it can just output the garbage label.

We say that an outcome  $O$  solves a problem  $\Pi$  over a family of graphs  $\mathcal{F}$  with probability  $q > 0$  if, for every  $G \in \mathcal{F}$  and every input data  $x$ , it holds that

$$\sum_{\substack{i \in I: \\ \text{out}_i \in \Pi(G, x)}} p_i \geq q.$$

Let  $O : (G, x) \mapsto \{(\text{out}_i, p_i)\}_{i \in I}$  be any outcome and fix an input  $(G, x)$ . Consider any subset of nodes  $S \subseteq V(G)$ . Let  $\mathcal{H}(G)[S]$  be the subset of  $\mathcal{H}(G)$  that contains all elements  $(v, (v, e)) \in \mathcal{H}(G)$  where  $v \in S$ . The restriction of the output distribution  $O(G, x) = \{(\text{out}_i : \mathcal{H}(G) \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}, p_i)\}_{i \in I}$  to  $S$  is the distribution  $\{(\text{out}_j : \mathcal{H}(G)[S] \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}, p'_j)\}_{j \in J}$  such that

$$p'_j = \sum_{\substack{i \in I: \\ \text{out}_j = \text{out}_i|_{\mathcal{H}(G)[S]}}} p_i,$$

and is denoted by  $O(G, x)[S]$  or  $\{(\text{out}_i, p_i)\}_{i \in I}[S]$ . We also say that two labeling distributions  $\{(\text{out}_i : \mathcal{H}(G) \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}, p_i)\}_{i \in I}, \{(\text{out}_j : \mathcal{H}(G') \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}, p_j)\}_{j \in J}$  over two graphs  $G, G'$  are isomorphic if there is an isomorphism  $\varphi : V(G) \rightarrow V(G')$  between  $G$  and  $G'$  such that  $\{(\text{out}_i : \mathcal{H}(G) \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}, p_i)\}_{i \in I} = \{(\text{out}_j \circ \varphi : \mathcal{H}(G) \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}, p_j)\}_{j \in J}$ .

We are now ready to define *non-signaling outcomes*.

**Definition 4.2** (Non-signaling outcome). Let  $O$  be any outcome. Suppose there exists a non-negative integer  $T \geq 0$  with the following property: For any two subsets  $S_G \subseteq V(G), S_H \subseteq V(H)$  such that  $\varphi : V(G) \rightarrow V(H)$  is an isomorphism between  $\mathcal{V}_T(S_G)$  and  $\mathcal{V}_T(S_H)$ , then the restrictions  $O(H, x_H)[S_H]$  and  $O(G, x_G)[S_G]$  are isomorphic under  $\varphi$ . In this case, we say that  $O$  is *non-signaling beyond distance  $T$*  or, alternatively, that  $O$  has locality  $T$ .

Running  $T$ -rounds classical or quantum-LOCAL algorithms without shared resources, we obtain the following property on the output labeling distribution: for every two subset of nodes  $A, B$  such that  $\text{dist}(A, B) > 2T$ , then the output distributions restricted to  $A$  and  $B$  are independent. Let us formalize this notion.

**Definition 4.3** ( $T$ -dependent distribution). Let  $\mathcal{V}_{\text{in}}, \mathcal{E}_{\text{in}}, \mathcal{V}_{\text{out}}, \mathcal{E}_{\text{out}}$  be finite sets of labels, and  $J$  a finite set of indices. Let  $\mathcal{I}$  be the family of *all* input distributed networks  $(G, x)$ , where  $G$  is a  $(\mathcal{V}_{\text{in}}, \mathcal{E}_{\text{in}})$ -labeled graph and  $x$  contains input data (identifiers, ports, possible random bits, etc.). An output labeling distribution  $\{(\text{out}_i : \mathcal{H}(G) \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}, p_i)\}_{i \in J}$  is  $T$ -dependent if, for any two subsets of nodes  $A, B \subseteq V(G)$  such that  $\text{dist}(A, B) > T$ , we have that  $\{(\text{out}_i, p_i)\}_{i \in J}[A]$  is independent of  $\{(\text{out}_i, p_i)\}_{i \in J}[B]$ .

We can think now of non-signaling outcomes that produce such distributions.

**Definition 4.4** (Bounded-dependent outcome). Let  $O$  be any outcome that is non-signaling beyond distance  $T$ . We say that  $O$  is bounded-dependent with locality  $T$  if for any input  $(G, x)$  we have that  $O(G, x)$  is  $2T$ -dependent. Furthermore, when  $T = O(1)$  (i.e., it does not depend on the input graph), we say that  $O(G, x)$  is a finitely-dependent distribution. Moreover, if for all inputs  $(G, x)$  it holds that  $O(G, x)$  does not depend on identifiers and port numbers, we say that  $O$  is *invariant under subgraph isomorphism*.

With the addition of this further property, we can define the *bounded-dependence model*, first formalized in [ACd+25].

**The bounded-dependence model.** The *bounded-dependence model* is a computational model that produces bounded-dependent outcomes. The required success probability when solving a problem must be at least  $1 - 1/\text{poly}(n)$ .

#### 4.1 Limits on quantum advantage

With these definitions, we are ready to prove Theorem 1.2. Formally, in this section, we prove the following theorem:

**Theorem 4.5.** *Let  $\Pi$  be an LCL problem with checking radius  $r$ . Let  $\mathcal{O}$  be a bounded-dependent outcome with locality  $T(n)$  of labelings that solve problem  $\Pi$  with probability  $p(n)$ . Then there exists a rand-LOCAL algorithm solving  $\Pi$  with locality  $O(\sqrt{nT(n)} \text{poly log } n)$  with probability at least  $p(n)$ .*

Like in the overview, we start by clustering the graph into small well-separated clusters. Formally, we mean the following:

**Definition 4.6** ( $(\varepsilon, d)$ -clustering). Given a graph  $G$ , an  $(\varepsilon, d)$ -clustering is a partition  $V(G) = D \cup S_1 \cup \dots \cup S_k$  meeting the following conditions:

- The distance between two nodes  $u \in S_i, v \in S_j$ , from different clusters  $i \neq j$ , is at least  $2$ .
- For every  $1 \leq i \leq k$ , the diameter  $\max_{u,v \in S_i} \text{dist}_G(u, v)$  of  $S_i$  is at most  $O(d)$ .
- $D$  contains at most  $|D| \leq \varepsilon |V(G)|$  vertices.

To actually compute such clustering, we use the following clustering algorithm by Coiteux-Roy et al. [CDG+24]:

**Theorem 4.7.** *For any  $0 < \varepsilon \leq 1$ , an  $(\varepsilon, \frac{\log n \cdot \log \log \log n}{\varepsilon})$ -clustering can be computed with locality*

$$O\left(\frac{\log(n)^2 \cdot \log(1/\varepsilon)}{\varepsilon}\right)$$

*rounds in the deterministic LOCAL model.*

By the definition of clustering, we only get that nodes in different clusters are mutually non-adjacent. However, to ensure that the labelings in each cluster are mutually independent given a labeling for the unclustered nodes, we require nodes of different clusters to be at distance at least  $r$ , where  $r$  denotes the checkability radius of our LCL problem  $\Pi$ . We can fix this by running the algorithm from Theorem 4.7 on the power graph  $G^r$ . By doing so and choosing  $\varepsilon = (1/\sqrt{nT(n)})$  we obtain the following corollary.

**Corollary 4.8.** *A  $(1/\sqrt{nT(n)}, \sqrt{nT(n)} \log^2 n)$ -clustering can be computed with locality*

$$O\left(\sqrt{nT(n)} \log^3(n)\right)$$

*in the deterministic LOCAL model. Additionally, any two nodes  $u, v$  from different clusters have distance at least  $r$  in  $G$  for any pre-chosen constant  $r$ .*

This clustering ensures that the clusters are well-separated enough such that their labelings can be completed independently given a labeling in the unclustered nodes  $D$ . Our next step is to produce a labeling on  $D$  by sampling for the bounded-dependence original distribution. To be able to do this, we need to partition  $D$  into well-separated partitions whose outputs are independent. Formally, we do the following:

**Lemma 4.9.** *For any subset of nodes  $D \subseteq V$  of size at most  $O(\sqrt{n/T(n)})$ , we can partition  $D$  into  $D_1, \dots, D_h$  such that*

- *for every  $1 \leq i \leq h$ , the diameter of  $D_i$  is  $O(\sqrt{nT(n)})$  in  $G$ , and*
- *for every  $1 \leq i < j \leq h$ , if  $u \in D_i$  and  $v \in D_j$  then  $\text{dist}_G(u, v) \geq 2T(n) + 1$ .*

*Moreover, this partitioning can be computed in the LOCAL model with locality  $O(\sqrt{nT(n)})$ .*

The latter condition allows us to sample the outputs for each partition independently as, by the definition of bounded-dependence distributions, their outputs are independent. The former condition makes sure that each partition has a low diameter and hence the sampling can be done with low locality. Intuitively, the partitioning is done by finding the connected components in the power graph  $G^{2T(n)}$ :

*Proof.* Each node in  $D$  iteratively runs the following exploration procedure: The node  $v$  looks at its radius- $2T(n)$  neighborhood in the original graph  $G$  to see if there are other nodes that belong to  $D$ . If not, the node stops as it has found all nodes of  $D$  that are within distance  $2T(n)$  of it, and hence all other nodes must belong to different partitions. Otherwise the node adds newly-found nodes to its own partition and repeats this exploration procedure from these newly-found nodes.

It is clear that this algorithm produces a partitioning that satisfies the latter condition. It remains to prove that the algorithm stops within  $O(\sqrt{nT(n)})$  steps. Since nodes only start another iteration if they find a new node from the set  $D$ , this procedure stops after at most  $|D|$  iterations. As each iteration has locality  $2T(n)$ , the total running time of the algorithm is bounded by

$$|D| \cdot 2T(n) = O(\sqrt{n/T(n)}) \cdot 2T(n) = O(\sqrt{nT(n)}).$$

This also implies that the diameter of each cluster is  $O(\sqrt{nT(n)})$ .  $\square$

We now have the nodes of graph  $G$  clustered into  $D, S_1, \dots, S_k$ , and nodes  $D$  partitioned into  $D_1, \dots, D_h$ . Let  $x$  be an input for  $G$  and consider a  $2T(n)$ -dependent labeling distribution  $O(G, x)$  over  $(G, x)$  that is given by a bounded-dependent outcome  $O$  with locality  $T(n)$ . We now show that a randomized LOCAL algorithm can sample a labeling for  $D$  from  $O(G, x)[D]$  if it is given an oracle to  $O$ .

**Lemma 4.10.** *Let  $O$  be a bounded-dependent outcome with locality  $T(n)$ . There exists a rand-LOCAL algorithm that runs on  $(G, x)$  with  $V(G)$  clustered as  $D, S_1, \dots, S_k$ ,  $D$  partitioned as  $D_1, \dots, D_h$  and an oracle to  $O$ , and outputs labelings for  $D$  that follow the distribution  $O(G, x)[D]$ . This algorithm has locality  $O(\max_i \text{diam}_G(D_i))$ .*

*Proof.* Since  $\text{dist}_G(D_i, D_j) > 2T(n)$  if  $i \neq j$ , for all  $1 \leq \ell \leq h$ , for any sequence  $1 \leq i_1 < \dots < i_\ell \leq h$ , the restrictions  $O(G, x)[D_{i_1}], \dots, O(G, x)[D_{i_\ell}]$  of  $O(G, x)$  to  $D_{i_1}, \dots, D_{i_\ell}$  are mutually independent by definition of  $2T(n)$ -dependent distribution. Let  $\text{out} : \mathcal{H}(G) \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}$  be any output labeling function. For a given subset of nodes  $S \subseteq V(G)$ , let us denote the probability that  $O(G, x)$  actually yields the output  $\text{out}$  on  $\mathcal{H}(G)[S]$  by  $P_{\text{out}}(\mathcal{H}(G)[S])$ . For all  $1 \leq \ell \leq h$ , for any sequence  $1 \leq i_1 < \dots < i_\ell \leq h$ , we have

$$P_{\text{out}}(\mathcal{H}(G)[D_{i_1}]) \cdot \dots \cdot P_{\text{out}}(\mathcal{H}(G)[D_{i_\ell}]) = P_{\text{out}}(\mathcal{H}(G)[D_{i_1} \cup \dots \cup D_{i_\ell}]).$$

It follows that, for  $1 \leq i \leq h$ , a rand-LOCAL algorithm can gather the subgraph induced by  $D_i$  and sample a labeling from distribution  $O(G, x)[D_i]$  independently from the other  $D_j$ ,  $i \neq j$ . The equality above ensures that the final labeling obtained in this way for  $D$  is sampled with the same probability as if we sampled it from  $O(G, x)[D]$ . As the algorithm samples the labeling for each  $D_i$  independently, it is obvious that the locality of the algorithm is bounded by  $O(\max_i \text{diam}_G(D_i))$ .  $\square$

In other words, the output labels of nodes belonging to different partitions can be sampled independently while respecting the locality of the bounded-dependent distribution. This implies that, by sampling a function  $\text{out}_D : \mathcal{H}(G)[D] \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}$  from the bounded-dependent outcome  $O(G, x)[D]$ , we can first fix the output labels  $\text{out}_D(D)$ , and then label the remaining nodes in  $G$  by brute force. We first show that, if the original outcome  $O$  has success probability  $p$ , then there exists an extension of  $\text{out}_D$  to a labeling for  $\mathcal{H}(G)$  that is a solution for  $\Pi$  with probability at least  $p$ . Actually finding such a labeling for  $\mathcal{H}(G)$  by brute force will follow as a corollary.

**Lemma 4.11.** *Let  $O$  be a bounded-dependent outcome that solves the LCL problem  $\Pi$  on  $(G, x)$  with success probability  $p$ , and let  $\text{out}_D$  be a labeling for  $\mathcal{H}(G)[D]$  sampled from  $O(G, x)[D]$ . Then, with probability at least  $p$ , there exists at least one labeling  $\text{out}_G$  for  $\mathcal{H}(G)$  such that  $\text{out}_D$  is a restriction of  $\text{out}_G$  to  $\mathcal{H}(G)[D]$ , namely  $\text{out}_D = \text{out}_G \upharpoonright_{\mathcal{H}(G)[D]}$ , and  $\text{out}_G$  is a solution for  $\Pi$  on  $(G, x)$ .*

*Proof.* If we sample labeling  $\text{out}_D$  from  $O(G, x)[D]$ , we can be in three possible cases.

1. Labeling  $\text{out}_D$  for  $\mathcal{H}(G)$  is the restriction of some labeling  $\text{out}$  for  $\mathcal{H}(G)$ , and  $\text{out}$  solves  $\Pi$  in  $(G, x)$ . In this case we are guaranteed that at least the extension  $\text{out}_G = \text{out}$  exists.
2. Labeling  $\text{out}_D$  for  $\mathcal{H}(G)$  is the restriction of some labeling  $\text{out}$  for  $\mathcal{H}(G)$ ,  $\text{out}_D$  is a solution for  $\Pi$  on  $(\dot{G}[D], x \upharpoonright_D)$ , but  $\text{out}$  is not a solution for  $\Pi$  in  $(G, x)$ . Even if we cannot take the extension  $\text{out}_G = \text{out}$ , it might still be possible to find a different  $\text{out}'$  that does not belong to  $O(G, x)$  but that is a solution  $\Pi$  on  $(G, x)$  nonetheless.
3. Labeling  $\text{out}_D$  for  $\mathcal{H}(G)$  is the restriction of some labeling  $\text{out}$  for  $\mathcal{H}(G)$ , and  $\text{out}_D$  is not a solution for  $\Pi$  on  $(\dot{G}[D], x \upharpoonright_D)$ . Then, no correct extension is possible.

The first case occurs with probability  $p$ , while with probability  $1 - p$  either the second or the third occur. Since a correct extension of  $\text{out}_D$  could be possible even in the second case, it follows that the overall probability of finding a correct extension of  $\text{out}_D$  is at least  $p$ .  $\square$

**Corollary 4.12.** *Let  $\text{out}_D$  be a labeling for  $\mathcal{H}(G)[D]$  such that there exists a non-empty set of labelings  $L = \{\text{out}_j\}_{j \in J}$  for  $\mathcal{H}(G)$  where each  $\text{out}_j$  is a solution for  $\Pi$  on  $(G, x)$  and  $\text{out}_D$  is a restriction of  $\text{out}_j$  to  $\mathcal{H}(G)[D]$ , namely  $\text{out}_D = \text{out}_j \upharpoonright_{\mathcal{H}(G)[D]}$ . There exists a LOCAL algorithm that runs on  $(G, x)$  with  $V(G)$  clustered as  $D, S_1, \dots, S_k$  and outputs a labeling  $\text{out}_G \in L$ . This algorithm has locality  $O(\max_{1 \leq i \leq k} \{\text{diam}_G(S_i)\})$ .*

*Proof.* Every node in a given  $S_i$  gathers its radius- $O(\text{diam}_G(S_i))$  neighborhood, making sure that this radius is not smaller than the diameter of  $S_i$  plus  $r$ . Then we know that every node in  $S_i$  sees every other node in the same  $S_i$ , plus a frontier of depth  $r$  in  $D$ . We can then compute by brute force a valid solution to  $\Pi$  restricted to  $S_i$  by testing every possible labeling  $\text{out}_j : \mathcal{H}(G)[S_i] \rightarrow \mathcal{V}_{\text{out}} \times \mathcal{E}_{\text{out}}$ .

The procedure just described clearly has locality  $O(\text{diam}_G(S_i))$ , and it is always correct because  $\Pi$  has checkability radius  $r$ , and Corollary 4.8 guarantees that nodes in different clusters have distance at least  $r$  in  $G$ . Thus, the radius- $r$  neighborhood of any node in  $S_i$  can intersect only the same  $S_i$  and  $D$ . Since the labeling for  $D$  is already fixed by  $\text{out}_D$ , we can safely check whether an output labeling solves  $\Pi$  in  $S_i$ .  $\square$

We can now prove Theorem 4.5:

*Proof of Theorem 4.5:* Let  $\Pi$  be an LCL problem with checking radius  $r$  and that admits a bounded-dependence distribution with locality  $T(n)$  and success probability  $p(n)$ . We construct a rand-LOCAL algorithm that solves problem  $\Pi$  with locality  $O(\sqrt{nT(n)} \text{poly log } n)$ . The algorithm works in four steps:

1. It first computes a  $(1/\sqrt{nT(n)}, \sqrt{nT(n)} \log^2 n)$ -clustering  $D \cup S_1, \cup \dots, S_k$ . By Corollary 4.8, this can be done with locality  $O(\sqrt{nT(n)} \log^3(n))$ .
2. The algorithm then partitions the unclustered nodes  $D$  into partitions  $D_1, \dots, D_p$  such that each partition has distance at least  $2T(n)$  to every other partition. By Lemma 4.9, this can be done with locality  $O(\sqrt{nT(n)})$ .
3. The algorithm then samples a labeling for each partition  $D_i$  independently. This too can be done with locality  $O(\sqrt{nT(n)})$  thanks to Lemma 4.10 as each partition has diameter  $O(\sqrt{nT(n)})$  and partitions are at distance at least  $2T(n)$  from each other, which means that their distributions are independent.
4. Finally, the algorithm completes the labeling for each cluster independently. This succeeds with probability at least  $p(n)$  by Lemma 4.11, and the brute force completion of Corollary 4.12 takes locality  $O(\sqrt{nT(n)} \log^2 n)$  as this is the bound on the diameter of each cluster.

The locality of the algorithm is dominated by the first step. In total, the algorithm has locality  $O(\sqrt{nT(n)} \log^3 n)$ .

The only part in this algorithm where we use randomness is in step 3 when sampling from the bounded-dependence distribution. By Lemma 4.11, we sample a labeling that is extendable to a labeling for the whole graph with probability at least  $p(n)$ . As the rest of the algorithm is deterministic, the algorithm succeeds with probability at least  $p(n)$ .  $\square$

Finally, we note that Theorem 4.5 can also be derandomized e.g. by using techniques by Ghaffari, Harris, and Kuhn [GHK18] combined with a modern network decomposition [GGH+23], assuming that the original bounded-dependence distribution succeeds with high enough probability. This gives us the following result:

**Corollary 4.13.** *Let  $\Pi$  be an LCL problem with checking radius  $r$ . Let  $\mathcal{O}$  be a bounded-dependent outcome with locality  $T(n)$  of labelings that solve problem  $\Pi$  with probability strictly larger than  $1 - \frac{1}{n}$ . Then there exists a det-LOCAL algorithm solving  $\Pi$  with locality  $O(\sqrt{nT(n)} \text{poly log } n)$ .*

## 5 Quantum advantage: technical details

**Preliminary definitions.** For any  $(\mathcal{V}, \mathcal{E})$ -labeled graph  $G = (V, E)$ , and for any node  $v \in V$  and any edge  $e \in E$ , we denote by  $L_v(e)$  the label of the half-edge  $(v, e)$ . We also want to be able to refer to the endpoint of a path that starts from some node  $v$  and follows a given sequence  $L_1, \dots, L_k$  of labels over half-edges. We define the function  $f$  as follows: Consider any node  $v$  and any sequence of edge labels  $L_1, \dots, L_k \in \mathcal{E}$ . Let  $P = (v_1, \dots, v_{k+1})$  be a path in  $G$  such that  $v_1 = v$  and, for any edge  $e = \{v_i, v_{i+1}\}$ , the half-edge  $(v_i, e)$  is labeled with  $L_i$ . Then, we set  $f(v, L_1, \dots, L_k) = v_{k+1}$  if the path  $P$  exists and is unique, and  $\perp$  otherwise.

### 5.1 Tree-like gadget

In order to define our LCL problem, we will use, as a basic building block, an object called *tree-like gadget*, which has already been used in different works related to LCLs [BBO+20, BGK+25, BCM+21]. An example of tree-like gadget is shown in Figure 1. We report here the definition provided in [BBO+20].

**Definition 5.1** (Tree-like gadget [BBO+20]). A graph  $G$  is a *tree-like gadget* of height  $\ell$  if it is possible to assign coordinates  $(l_u, k_u)$  to each node  $u \in G$ , where

- $0 \leq l_u < \ell$  denotes the depth of  $u$  in the tree, and
- $0 \leq k_u < 2^{l_u}$  denotes the position of  $u$  (according to some order) in layer  $l_u$ ,

such that there is an edge connecting two nodes  $u, v \in G$  with coordinates  $(l_u, k_u)$  and  $(l_v, k_v)$  if and only if:

- $l_u = l_v$  and  $|k_u - k_v| = 1$ , or
- $l_v = l_u - 1$  and  $k_v = \lfloor \frac{k_u}{2} \rfloor$ , or
- $l_u = l_v - 1$  and  $k_u = \lfloor \frac{k_v}{2} \rfloor$ .

A useful property of this gadget is that it can be made *locally checkable*, in the sense that there exists a set of labels  $\mathcal{E}^{\text{tree}}$  that can be assigned to each node-edge pair, and a constraint  $C^{\text{tree}}$  over the labels  $\mathcal{E}^{\text{tree}}$ , satisfying the following.

**Lemma 5.2** ([BBO+20]). *Let  $G$  be a graph that is labeled with labels in  $\mathcal{E}^{\text{tree}}$  such that  $C^{\text{tree}}$  is satisfied for all nodes in  $G$ . Then, each connected component of  $G$  is a tree-like gadget.*

**Lemma 5.3** ([BBO+20]). *Each tree-like gadget graph  $G$  can be labeled with labels in  $\mathcal{E}^{\text{tree}}$  such that  $C^{\text{tree}}$  is satisfied for all nodes in  $G$ .*

The set of labels  $\mathcal{E}^{\text{tree}}$  is defined in [BCM+21] as  $\mathcal{E}^{\text{tree}} = \{\text{L}, \text{R}, \text{P}, \text{Ch}_\text{L}, \text{Ch}_\text{R}\}$  (the labels stand for “left”, “right”, “parent”, “left child”, and “right child”, respectively). The set of local constraints  $\mathcal{C}^{\text{tree}}$  is defined in [BCM+21] as follows.

*The constraints  $\mathcal{C}^{\text{tree}}$  of [BCM+21]*

1. For any two edges  $e, e'$  incident to a node  $u$ , it must hold that  $L_u(e) \neq L_u(e')$ ;
2. For each edge  $e = \{u, v\}$ , if  $L_u(e) = \text{L}$ , then  $L_v(e) = \text{R}$ , and vice versa;
3. For each edge  $e = \{u, v\}$ , if  $L_u(e) = \text{P}$ , then  $L_v(e) \in \{\text{Ch}_\text{L}, \text{Ch}_\text{R}\}$ , and vice versa;
4. If a node  $u$  has an incident edge  $e = \{u, v\}$  with label  $L_u(e) = \text{P}$  such that  $L_v(e) = \text{Ch}_\text{L}$ , then  $f(u, \text{P}, \text{Ch}_\text{R}, \text{L}) = u$ ;
5. If a node  $u$  has an incident edge  $e = \{u, v\}$  with label  $L_u(e) = \text{P}$  such that  $L_v(e) = \text{Ch}_\text{R}$ , if  $u$  has an incident edge labeled  $\text{R}$ , then  $f(u, \text{P}, \text{R}, \text{Ch}_\text{L}, \text{L}) = u$ .
6. If a node has an incident half-edge labeled  $\text{Ch}_\text{L}$ , then it must also have an incident half-edge labeled  $\text{Ch}_\text{R}$ , and vice versa;
7. A node does not have an incident half-edge labeled  $\text{P}$  if and only if it has no incident half-edges labeled  $\text{L}$  or  $\text{R}$ ;
8. If a node  $u$  does not have an incident edge  $e$  with label  $L_u(e) \in \{\text{Ch}_\text{L}, \text{Ch}_\text{R}\}$ , then neither do nodes  $f(u, \text{L})$  and  $f(u, \text{R})$  (if they exist);
9. If a node  $u$  has an incident edge  $e = \{u, v\}$  with label  $L_u(e) = \text{P}$  such that  $L_v(e) = \text{Ch}_\text{R}$  (resp.  $L_v(e) = \text{Ch}_\text{L}$ ), then  $u$  has an incident edge labeled  $\text{R}$  (resp.  $\text{L}$ ) if and only if  $f(u, \text{P})$  has an incident edge labeled  $\text{R}$  (resp.  $\text{L}$ ).

An example of tree-like gadget labeled from labels in  $\mathcal{E}^{\text{tree}}$  such that  $C^{\text{tree}}$  is satisfied for all nodes is shown in Figure 1.

Moreover, in [BGK+25], it is shown that it is easy for the nodes to *prove* that the graph in which they are is an invalid tree-like gadget. More formally, in [BGK+25], it is defined an LCL problem  $\Pi^{\text{badtree}}$  satisfying the following. Nodes have input 0 or 1, and nodes with input 1 are called *marked*. Node-edge pairs have an input label from  $\mathcal{E}^{\text{tree}}$ . Recall that a graph with such an input is called  $(\{0, 1\}, \mathcal{E}^{\text{tree}})$ -labeled graph. The set  $\mathcal{V}^{\text{badTree}}$  of outputs labels for  $\Pi^{\text{badtree}}$  contains a special label called  $\perp$ , and in this problem there are only node output labels (hence, there are no half-edge output labels).

**Lemma 5.4** ([BGK+25]). *There exists an LCL problem  $\Pi^{\text{badtree}}$  satisfying the following properties.*

- *Let  $G$  be a  $(\{0, 1\}, \mathcal{E}^{\text{tree}})$ -labeled graph where  $C^{\text{tree}}$  is satisfied on all nodes and all nodes are not marked. Then, the only valid solution for  $\Pi^{\text{badtree}}$  is the one assigning  $\perp$  to all nodes.*
- *Let  $G$  be a connected  $(\{0, 1\}, \mathcal{E}^{\text{tree}})$ -labeled graph where either  $C^{\text{tree}}$  is not satisfied on at least one node, or there is at least one marked node. Then, there exists a solution for  $\Pi^{\text{badtree}}$  where all nodes produce an output different from  $\perp$ . Moreover, such a solution can be computed in  $O(\log n)$  deterministic rounds in the LOCAL model.*

On a high level, we will later use  $\Pi^{\text{badtree}}$  as a black box, and we will mark nodes that witness some local errors that are not related to the tree-like gadget itself. In this way, we will be able to use  $\Pi^{\text{badtree}}$  to also prove errors that are unrelated to the tree-like gadget itself. An example of valid solution of  $\Pi^{\text{badtree}}$  is depicted in Figure 2.

## 5.2 Octopus gadget

In this section, we formally define the notion of *octopus gadget*, then we prove that an octopus gadget is locally checkable, and finally we define the LCL problem  $\Pi^{\text{badOctopus}}$  and prove some properties about it.

**Definition 5.5** (Octopus gadget). Let  $x \geq 1$  be a natural number, and  $\eta = (\eta_0, \dots, \eta_{2^{x-1}-1})$  a vector of  $2^{x-1}$  entries in  $\{1, 2\}$ . Let  $W = \{w_{(i,j)}\}_{(i,j) \in I}$  be a family of positive integer weights, where  $I$  is the set containing all pairs  $(i, j)$  satisfying  $(i, j) \in \{0, 1, \dots, 2^{x-1} - 1\} \times \{1, 2\}$  and  $j \leq \eta_i$ .

A graph  $G = (V, E)$  is an  $(x, \eta, W)$ -octopus gadget if there exists a labeling  $\lambda : V \rightarrow \mathcal{L} = I \cup \{\text{root}\}$  of the nodes of  $G$  such that the following holds.

1. For each element  $y \in \mathcal{L}$ , let  $G_y$  be the subgraph of  $G$  induced by nodes labeled with  $y$ . Then, for all  $y \in \mathcal{L}$ ,  $G_y$  must be a tree-like gadget according to Definition 5.1.
2. For all  $y, z \in \mathcal{L}$  such that  $y \neq z$ ,  $G_y$  and  $G_z$  must be disjoint.
3.  $G_{\text{root}}$  has height  $x$  and, for all  $(i, j) \in I$ ,  $G_{(i,j)}$  has height  $w_{(i,j)} \in W$ .
4. For all  $(i, j) \in I$ , there is an edge connecting the node of  $G_{(i,j)}$  that has coordinates  $(0, 0)$  with the node of  $G_{\text{root}}$  that has coordinates  $(x - 1, i)$ .

$G_{\text{root}}$  is called the *head-gadget* and, for all  $(i, j) \in I$ ,  $G_{(i,j)}$  is called a *port-gadget*.

**Local checkability.** While Definition 5.5 gives the definition of octopus gadget from a *global* perspective, we now define a finite set of labels and a set of local constraints satisfying that a connected graph is a properly labeled octopus gadget if and only if these *local* constraints are satisfied by all nodes.

We first define the sets of labels  $\mathcal{V}^{\text{octopus}}$  and  $\mathcal{E}^{\text{octopus}}$ , and then we define a set of local constraints  $\mathcal{C}^{\text{octopus}}$  over these labels. We will prove that a connected graph  $G$  can be  $(\mathcal{V}^{\text{octopus}}, \mathcal{E}^{\text{octopus}})$ -labeled such that the constraints  $\mathcal{C}^{\text{octopus}}$  are satisfied on all nodes if and only if  $G$  is an  $(x, \eta, W)$ -octopus gadget, for some  $x, \eta$ , and  $W$ .

The sets of labels are defined as follows:

$$\begin{aligned}\mathcal{V}^{\text{octopus}} &= \{\text{head}, \text{port}\}, \\ \mathcal{E}^{\text{octopus}} &= \{\text{hplink}_1, \text{hplink}_2, \text{phlink}\} \cup \mathcal{E}^{\text{tree}},\end{aligned}$$

where  $\text{hplink}$  stands for “head-port link” and  $\text{phlink}$  stands for “port-head link”, respectively. Recall that the labels in  $\mathcal{V}^{\text{octopus}}$  are *node* labels, while  $\mathcal{E}^{\text{octopus}}$  are labels for *node-edge pairs*.

We now define the set of constraints  $\mathcal{C}^{\text{octopus}}$ . An edge  $e = \{u, v\}$  is called *internal* if and only if  $\{L_u(e), L_v(e)\} \cap \{\text{hplink}_1, \text{hplink}_2, \text{phlink}\} = \emptyset$ , and *external* otherwise.

*The constraints  $\mathcal{C}^{\text{octopus}}$*

0. In the subgraph induced by internal edges, all the constraints in  $\mathcal{C}^{\text{tree}}$  must be satisfied.
1. For each edge  $e = \{u, v\}$ , it must hold that nodes  $u$  and  $v$  are both labeled **head**, or both labeled **port**, if and only if  $e$  is an internal edge.
2. For each edge  $e = \{u, v\}$ , if  $L_u(e) = \text{hplink}_j$  for some  $j \in \{1, 2\}$ , then  $L_v(e) = \text{phlink}$ , and vice versa.
3. If a node  $v$  is labeled with **head** and  $e$  is an edge incident to  $v$ , then  $L_v(e) \neq \text{phlink}$ .
4. If a node  $v$  is labeled with **port** and  $e$  is an edge incident to  $v$ , then  $L_v(e) \neq \text{hplink}_j$  for all  $j \in \{1, 2\}$ .
5. A node  $v$  has an incident edge labeled  $\text{hplink}_j$  for some  $j \in \{1, 2\}$  if and only if  $v$  is labeled with **head** and it has no incident edge  $e$  satisfying  $L_v(e) \in \{\text{Ch}_L, \text{Ch}_R\}$ .
6. A node  $v$  has an incident edge labeled  $\text{phlink}$  if and only if  $v$  is labeled with **port** and it has no incident edge  $e$  satisfying  $L_v(e) = \text{P}$ .
7. A node cannot have more than one incident edge labeled with  $\text{phlink}$ ,  $\text{hplink}_1$ , and  $\text{hplink}_2$ .
8. If a node  $v$  has an incident edge  $e$  such that  $L_v(e) = \text{hplink}_2$ , then it has another incident edge  $e'$  such that  $L_v(e') = \text{hplink}_1$ .

**Lemma 5.6.** *Let  $G$  be a connected graph that is  $(\mathcal{V}^{\text{octopus}}, \mathcal{E}^{\text{octopus}})$ -labeled such that  $\mathcal{C}^{\text{octopus}}$  is satisfied at all nodes. Then,  $G$  is an octopus gadget.*

*Proof.* In the following, by labels of an edge  $e = \{u, v\}$  we denote the labels  $L_v(e)$  and  $L_u(e)$ . By constraint 2, for each edge  $e$  it holds that the labels of  $e$  are either both in  $\{\text{hplink}_1, \text{hplink}_2, \text{phlink}\}$ , or both in  $\mathcal{E}^{\text{tree}}$ . Hence, internal edges have only labels from  $\mathcal{E}^{\text{tree}}$  and external edges have only labels from  $\{\text{hplink}_1, \text{hplink}_2, \text{phlink}\}$ . Moreover, by constraint 2, an external edge has exactly two labels from  $\{\text{hplink}_1, \text{hplink}_2, \text{phlink}\}$ , and these two labels cannot be  $\{\text{hplink}_1, \text{hplink}_2\}$ .

By constraint 0 and Lemma 5.2, each connected component in the subgraph induced by internal edges is a tree-like gadget (possibly, a graph without edges). Moreover, for each connected component, by constraint 1, it must hold that all nodes in the component are either all labeled **head**, or all labeled **port**.

Summarizing what we have showed so far, each connected component in the subgraph induced by internal edges is either a tree-like gadget where all nodes are labeled **head** or a tree-like gadget where all nodes are labeled **port**, and external edges have labels  $\{\mathbf{hp}_{\text{link}_1}, \mathbf{ph}_{\text{link}}\}$  or labels  $\{\mathbf{hp}_{\text{link}_2}, \mathbf{ph}_{\text{link}}\}$ . Hence, we only need to prove that external edges are properly connected.

Suppose  $G$  is non-empty. Let  $v$  be a node in  $G$ . We prove that the connected component  $G'$  (in  $G$ , including external edges) containing  $v$  is an octopus gadget.

Let  $H$  be the connected component containing  $v$  in the graph induced by internal edges. As discussed,  $H$  is a tree-like gadget. We start by proving that, in  $G'$ , there must be a tree-like gadget where all nodes are labeled **head**. If  $v$  itself is labeled **head**, we are done. If  $v$  is marked **port**, then by constraint 6 the node  $u$  in the tree-like gadget  $H$  that has no incident half-edge labeled **P** (i.e., the node with coordinates  $(0,0)$ ) must have an edge  $e = \{u, z\}$  satisfying  $L_u(e) = \mathbf{ph}_{\text{link}}$ . By constraint 2,  $L_z(e) = \mathbf{hp}_{\text{link}_j}$  for some  $j \in \{1, 2\}$ , and by constraint 4 no node labeled **port** can have incident edges with label  $\mathbf{hp}_{\text{link}_j}$  for any  $j \in \{1, 2\}$ . Hence,  $z$  must be a node labeled **head**.

Let  $H_h$  be the connected component containing  $z$  in the graph induced by internal edges.  $H_h$  must be a tree-like gadget. By constraint 5, only nodes in the last layer of  $H_h$  can have an incident edge labeled  $\mathbf{hp}_{\text{link}_j}$  for some  $j \in \{1, 2\}$ , and such nodes must have at least one such edge. By constraint 7, each of these nodes cannot have other edges labeled with  $\mathbf{hp}_{\text{link}_j}$ . Let  $e = \{w_h, w_p\}$  be an arbitrary such edge, where  $L_{w_h}(e) = \mathbf{hp}_{\text{link}_j}$  for some  $j \in \{1, 2\}$ . By constraint 2, it must hold that  $L_{w_p}(e) = \mathbf{ph}_{\text{link}}$ . Moreover, by constraint 7, node  $w_p$  cannot have additional incident edges labeled  $\mathbf{ph}_{\text{link}}$ . By constraint 3, node  $w_p$  must be labeled **port** and by constraints 6, 0, and 1, node  $w_p$  must be the root of a tree-like gadget  $H_p$  where all nodes are labeled **port**. By constraint 6, no other node of  $H_p$  can have edges labeled  $\mathbf{ph}_{\text{link}}$ . Furthermore, by constraint 8, each node in the last layer of  $H_p$  must have an incident edge labeled  $\mathbf{hp}_{\text{link}_1}$  if it has an incident edge labeled  $\mathbf{hp}_{\text{link}_2}$ .

We thus get that, for each head-gadget  $H_h$ , there is at least one port-gadget  $H_p$  connected to each leaf of  $H_h$  and at most two of them, and that  $H_p$  is connected to exactly one head-gadget via its root. Hence,  $G'$  is an octopus gadget. Let  $x$  be the height of the tree-like gadget  $H_h$ . Now, we assign the label **root** to all nodes in  $H_h$ , and the label  $(i, j)$  to all nodes in the port-gadget  $H_p$  if  $H_p$  is connected to  $H_h$  via the leaf  $v$  of  $H_h$  having coordinates  $(x-1, i)$  through the edge  $e$  such that  $L_v(e) = \mathbf{hp}_{\text{link}_j}$ . By this assignment, we have shown that  $G'$  satisfies Definition 5.5.

Finally, since  $G$  is connected, then  $G' = G$ . □

**Lemma 5.7.** *Let  $G$  be an octopus gadget. Then,  $G$  can be  $(\mathcal{V}^{\text{octopus}}, \mathcal{E}^{\text{octopus}})$ -labeled such that  $\mathcal{C}^{\text{octopus}}$  is satisfied at all nodes.*

*Proof.* Let  $G = (V, E)$  be an octopus gadget. We can assign the labels in this way.

- We assign the label **head** to all the nodes of the head-gadget.
- We assign the label **port** to all the nodes of the port-gadgets.
- By Lemma 5.3, we can assign labels to each tree-like gadget such that the constraints of  $\mathcal{C}^{\text{tree}}$  are satisfied.
- Let  $e = \{u, v\}$  be an edge connecting a head-gadget to a port-gadget (i.e., the edges of point 4 of Definition 5.5), where  $u$  is the node in the head-gadget. If  $v$  has the label  $(i, j) \in I$  in Definition 5.5, we set  $L_u(e) = \mathbf{hp}_{\text{link}_j}$  and  $L_v(e) = \mathbf{ph}_{\text{link}}$ .

By construction, all the constraints of  $\mathcal{C}^{\text{octopus}}$  are satisfied. □

**The LCL problem  $\Pi^{\text{badOctopus}}$ .** We now define an LCL problem  $\Pi^{\text{badOctopus}}$  that, on a high level, allows the nodes to prove that the graph in which they are is not an octopus gadget. Similarly as in the case of  $\Pi^{\text{badTree}}$ , nodes also receive a binary input, where nodes that receive 1 are called *marked*. Again, we will use this input later, to mark nodes that witness errors that may be unrelated to the octopus gadget itself. On a high level, the problem will satisfy the following properties.

- Nodes receive as input whether they are marked or not.
- There are two possible types of output: a node can either produce an empty output ( $\perp$ ), or it can output an error. In the latter case, a node needs to also output a locally checkable proof of the fact that the graph is not an octopus gadget or that it contains at least one marked node.
- If the octopus gadget is valid and it does not contain any marked node, the constraint of the problem are defined such that the only valid solution to the problem  $\Pi^{\text{badOctopus}}$  is the one where all nodes output  $\perp$ .
- Otherwise, if the graph is not an octopus gadget or it contains at least one marked node, *all* nodes are able to spend  $O(\log n)$  round in the LOCAL model to produce a proof of this fact.

We now give a formal definition of the LCL problem  $\Pi^{\text{badOctopus}}$ . The inputs are defined as follows.

- Each node  $v$  receives an input pair  $(m_v, g_v)$  from the set  $\{0, 1\} \times \mathcal{V}^{\text{octopus}}$ , that is, each node receives a pair, where the first element  $m_v$  denotes whether the node is marked or not, while the second element  $g_v$  of the pair is an element from the previously described set  $\mathcal{V}^{\text{octopus}}$ .
- Each half-edge receives an input from the previously described set  $\mathcal{E}^{\text{octopus}}$ .

Let  $\mathcal{V}^{\text{badTree}}$  be the node output labels of the LCL problem  $\Pi^{\text{badTree}}$ . The possible output labels  $\mathcal{V}^{\text{badTree}}$  are the following.

- The label  $\perp$ , which represent an empty output.
- A pair  $(\text{Error}, \mathcal{M})$ , where  $\mathcal{M}$  is a triple, and each element of the triple is a label from  $\mathcal{V}^{\text{badTree}}$ .

We denote the pairs  $(\text{Error}, \mathcal{M})$  as *error outputs*. On a high level, the constraints  $\mathcal{C}^{\text{badOctopus}}$  are defined such that the output  $\perp$  is always allowed, while the constraints on the error outputs are defined such that these outputs encode a proof of the fact that there is an error in the graph. Informally, the error output triples will be used as follows.

- If the *first* element of a triple of some node  $v$  is not  $\perp$ , it means that, in the connected component of the subgraph induced by internal edges that contains  $v$  there is some error.
- If the *second* element of a triple of some node  $v$  is not  $\perp$ , it means that, even if the connected component of  $v$  may be correct, node  $v$  is in a head gadget connected to at least one broken port gadget.
- If the *third* element of a triple of some node  $v$  is not  $\perp$ , it means that, even if the connected component of  $v$  may be correct, and even if  $v$  is in a correct port gadget connected to a correct head gadget, node  $v$  is in a port gadget connected to a head gadget that is connected to at least one broken port gadget.

In order to allow this labeling, we allow more and more nodes to be marked, as follows.

- For the first instance of  $\Pi^{\text{badTree}}$ , only marked nodes and nodes that have already witnessed some error are marked.
- For the second instance, additionally, nodes in head gadgets that are neighbors of nodes of port gadgets that output an error in the previous instance are marked.
- For the third instance, additionally, nodes in port gadgets that are neighbors of nodes of head gadgets that output an error in the previous instance are marked.

More formally, the constraints  $\mathcal{C}^{\text{badOctopus}}$  are defined as follows.

*The constraints  $\mathcal{C}^{\text{badOctopus}}$*

- A node can always output  $\perp$ .
- If a node  $v$  outputs  $(\text{Error}, (x_{v,1}, x_{v,2}, x_{v,3}))$ , then the following must hold.
  1. The triple  $(x_{v,1}, x_{v,2}, x_{v,3})$  must be different from  $(\perp, \perp, \perp)$ .
  2. Consider the labeling of the graph obtained by labeling each node  $v$  with the label  $g_v \in \mathcal{V}^{\text{octopus}}$ , and each half-edge  $(v, e)$  with the label  $L_v(e) \in \mathcal{E}^{\text{octopus}}$  (i.e., on the nodes we take the second element of their input pair, while on the edges we take the original input half-edge label). For each node  $v$ , let  $i_{v,1} := 1$  if, according to this labeling, node  $v$  does not satisfy the constraints  $\mathcal{C}^{\text{octopus}}$  or if  $m_v = 1$ , and let  $i_{v,1} := 0$  otherwise.
  3. Consider the labeling of the subgraph induced by internal edges obtained by input labeling each node  $v$  with  $i_{v,1}$  and output labeling each node  $v$  with  $x_{v,1}$ . This labeling must satisfy the constraints of  $\Pi^{\text{badTree}}$ .
  4. Let  $i_{v,2} := 1$  if  $i_{v,1} = 1$  or if  $v$  is a node such that  $g_v = \text{head}$  and such that there exists an edge  $e = \{u, v\}$  satisfying  $L_v(e) = \text{hp}_{\text{link}j}$  for some  $j \in \{1, 2\}$  and  $x_{u,1} \neq \perp$ . Let  $i_{v,2} := 0$  otherwise.
  5. Consider the labeling of the subgraph induced by internal edges obtained by input labeling each node  $v$  with  $i_{v,2}$  and output labeling each node  $v$  with  $x_{v,2}$ . This labeling must satisfy the constraints of  $\Pi^{\text{badTree}}$ .
  6. Let  $i_{v,3} := 1$  if  $i_{v,2} = 1$  or if  $v$  is a node such that  $g_v = \text{port}$  and such that there exists an edge  $e = \{u, v\}$  satisfying  $L_v(e) = \text{ph}_{\text{link}}$  and  $x_{u,2} \neq \perp$ . Let  $i_{v,3} := 0$  otherwise.
  7. Consider the labeling of the subgraph induced by internal edges obtained by input labeling each node  $v$  with  $i_{v,3}$  and output labeling each node  $v$  with  $x_{v,3}$ . This labeling must satisfy the constraints of  $\Pi^{\text{badTree}}$ .

**Lemma 5.8.** *Let  $G$  be a  $(\{0, 1\} \times \mathcal{V}^{\text{octopus}}, \mathcal{E}^{\text{octopus}})$ -labeled graph where  $\mathcal{C}^{\text{octopus}}$  is satisfied for all nodes and all nodes are not marked. Then, the only valid solution for  $\Pi^{\text{badOctopus}}$  is the one assigning  $\perp$  to all nodes.*

*Proof.* For a contradiction, assume that some node  $v$  in  $G$  outputs  $(\text{Error}, (x_{v,1}, x_{v,2}, x_{v,3}))$  for some labels  $x_{v,1}, x_{v,2}, x_{v,3}$ . By the definition of  $i_{v,1}$  in constraint 2, for all nodes  $v$  it must hold that  $i_{v,1} = 0$ , and by constraint 3 and Lemma 5.4 we thus get that  $x_{v,1} = \perp$ . Similarly, by the definition of  $i_{v,2}$  in constraint 4, for all nodes  $v$  it must hold that  $i_{v,2} = 0$ , and by constraint 5 and Lemma 5.4 we thus get that  $x_{v,2} = \perp$ . Again, by the definition of  $i_{v,3}$  in constraint 6, for all nodes  $v$  it must

hold that  $i_{v,3} = 0$ , and by constraint 7 and Lemma 5.4 we thus get that  $x_{v,3} = \perp$ . The claim then follows by the fact that constraint 1 forbids the triple  $(\perp, \perp, \perp)$ .  $\square$

**Lemma 5.9.** *Let  $G$  be a connected  $(\{0,1\} \times \mathcal{V}^{\text{Octopus}}, \mathcal{E}^{\text{Octopus}})$ -labeled graph where either  $\mathcal{C}^{\text{Octopus}}$  is not satisfied on at least one node, or there is at least one marked node. Then, there exists a solution for  $\Pi^{\text{badOctopus}}$  where all nodes produce an output different from  $\perp$ . Moreover, such a solution can be computed in  $O(\log n)$  deterministic rounds in the LOCAL model.*

*Proof.* In order to produce the claimed solution, we will use, as a black box, the algorithm for  $\Pi^{\text{badTree}}$  reported in Lemma 5.4. The algorithm for solving  $\Pi^{\text{badOctopus}}$  works as follows. Each node  $v$  spends  $O(1)$  rounds to compute  $i_{v,1}$ . By Lemma 5.4, nodes can spend  $O(\log n)$  deterministic classical rounds to compute a solution for  $\Pi^{\text{badTree}}$ , where  $i_{v,1}$  is the input of  $v$  for the problem, and, if in the subgraph induced by internal edges, in the connected component containing  $v$ , there is at least one marked node or a node not satisfying  $\mathcal{C}^{\text{tree}}$ , then  $v$  outputs a label different from  $\perp$ . For each node  $v$ , let  $x_{v,1}$  be this output.

Then, each node  $v$  spends  $O(1)$  rounds to compute  $i_{v,2}$ , and again, as before, nodes spend  $O(\log n)$  rounds to compute a solution for  $\Pi^{\text{badTree}}$ , where this time the input of node  $v$  is  $i_{v,2}$ .

Finally, each node  $v$  spends  $O(1)$  rounds to compute  $i_{v,3}$  and spends  $O(\log n)$  rounds to compute a solution for  $\Pi^{\text{badTree}}$ , where the input of node  $v$  is now  $i_{v,3}$ . If a node  $v$  satisfies  $x_{v,1} = x_{v,2} = x_{v,3}$ , then it outputs  $\perp$ . Otherwise, it outputs  $(\text{Error}, (x_{v,1}, x_{v,2}, x_{v,3}))$ .

The correctness of the output directly follows from the definition of  $\mathcal{C}^{\text{badOctopus}}$ , and the runtime is clearly upper bounded by  $O(\log n)$  deterministic classical rounds. We now prove that, if  $G$  contains at least one marked node, or at least one node not satisfying  $\mathcal{C}^{\text{Octopus}}$ , then all nodes output a label different from  $\perp$ .

Let  $\hat{G}$  be the graph obtained by contracting each connected component induced by internal edges into a single node. If there are multiple edges between different connected components, we have parallel edges in  $\hat{G}$ . By the definition of the algorithm, it holds that all nodes  $v$  of  $G$  corresponding to the same node of  $\hat{G}$ , in the first output of  $\Pi^{\text{badTree}}$  (i.e., the values  $x_{v,1}$ ), either all output  $\perp$  or all output something different. Let us label  $E$  (which stands for *error*) each node of  $\hat{G}$  satisfying that all its nodes output something different from  $\perp$ . We label all the other nodes either  $H$  (*head*) or  $P$  (*port*), as follows. If a node of  $\hat{G}$  is not labeled  $E$ , then either all its nodes of  $G$  have input head, or all its nodes of  $G$  have input port. In the former case, we label the node  $H$ , while in the latter case we label the node  $P$ .

By Lemma 5.6, at least one node of  $\hat{G}$  is labeled  $E$ . Moreover, by the definition of  $\mathcal{C}^{\text{Octopus}}$ , the graph  $\hat{G}$  must satisfy the following properties.

- Each node labeled  $P$  must have degree 1.
- Nodes labeled  $P$  must form an independent set.
- Nodes labeled  $H$  must form an independent set.

We thus get that each connected component of  $\hat{G}$  induced by nodes not labeled  $E$  must form stars centered at nodes labeled  $H$ . Moreover, we observe that each node labeled  $H$  must have at least one neighbor labeled  $E$ , since otherwise we would get that, in  $\hat{G}$ , there is a connected component not containing  $E$ , which in  $G$  corresponds to a valid octopus gadget. By the definition of the algorithm, we thus get that, in each connected component corresponding to a node labeled  $H$ , at least one node is marked when solving  $\Pi^{\text{badTree}}$  for the second time. Let us update the labeling of  $\hat{G}$  by changing nodes labeled  $H$  into nodes labeled  $E$  if their connected component contains nodes not outputting  $\perp$ . We get that, after this update, no node of  $\hat{G}$  is labeled  $H$ . Hence, all nodes of  $\hat{G}$  labeled  $P$  have a neighbor labeled  $E$ . By the definition of the algorithm, we thus get that, in each connected

component corresponding to a node labeled  $P$ , at least one node is marked when solving  $\Pi^{\text{badTree}}$  for the third time. Let us update the labeling of  $\hat{G}$  by changing nodes labeled  $P$  into nodes labeled  $E$  if their connected component contains nodes not outputting  $\perp$ . We get that, after this update, no node of  $\hat{G}$  is labeled  $P$ . Hence, all nodes of  $\hat{G}$  are labeled  $E$ , and hence no node of  $G$  outputs  $\perp$ .  $\square$

### 5.3 The family of proper instances

In Section 3.3, we informally defined the family of graphs called *proper instances* that we will use to prove our main result. We now provide a formal definition of such graphs. The main ingredient is the octopus gadget defined in Definition 5.5.

**Definition 5.10** (Proper instance). Let  $G = (V, E)$  be a graph. We say that  $G$  is a *proper instance* if there exists a node labeling function  $\lambda : V \rightarrow \{\text{intra-octopus}, \text{inter-octopus}\}$  with the following properties.

1. Every connected component in the subgraph induced by nodes labeled **intra-octopus** is an octopus gadget (according to Definition 5.5).
2. The subgraph induced by nodes labeled **inter-octopus** does not contain any edge.
3. A node  $v$  labeled **intra-octopus** is connected to a node labeled **inter-octopus** if and only if  $v$  has coordinates  $(w - 1, 0)$  in the port-gadget  $P$  containing  $v$ , where  $w$  is the height of  $P$  (that is,  $v$  is the left-most leaf of the port-gadget containing  $v$ ).

**Local checkability.** While Definition 5.10 gives the definition of proper instances from a *global* perspective, we now define a finite set of labels and a set of local constraints satisfying that a connected graph is a properly labeled valid instance if and only if these *local* constraints are satisfied by all nodes.

We first define the sets of labels  $\mathcal{V}^{\text{proper}}$  and  $\mathcal{E}^{\text{proper}}$ , and then we define a set of local constraints  $\mathcal{C}^{\text{proper}}$  over these labels. We will prove that a connected graph  $G$  can be  $(\mathcal{V}^{\text{proper}}, \mathcal{E}^{\text{proper}})$ -labeled such that the constraints  $\mathcal{C}^{\text{proper}}$  are satisfied on all nodes if and only if  $G$  is a proper instance.

The sets of labels are defined as follows. The node labels are defined as  $\mathcal{V}^{\text{proper}} = \mathcal{V}^{\text{octopus}} \cup \{\text{inter-octopus}\}$ , where **inter-octopus** stands for *inter-octopus node*. The half-edge labels are defined as  $\mathcal{E}^{\text{proper}} = \mathcal{E}^{\text{octopus}} \cup \{\text{pi}_{\text{link}}, \text{ip}_{\text{link}}\}$ , where **pi<sub>link</sub>** stands for “port–inter-octopus link” and **ip<sub>link</sub>** stands for “inter-octopus–port link”.

We now define the set of constraints  $\mathcal{C}^{\text{proper}}$ . An edge  $e = \{u, v\}$  is called *intra-octopus* if and only if  $\{L_u(e), L_v(e)\} \cap \{\text{pi}_{\text{link}}, \text{ip}_{\text{link}}\} = \emptyset$ , and *inter-octopus* otherwise.

*The constraints  $\mathcal{C}^{\text{proper}}$*

0. In the subgraph induced by intra-octopus edges, all the constraints in  $\mathcal{C}^{\text{octopus}}$  must be satisfied.
1. For each edge  $e = \{u, v\}$ , if  $L_u(e) = \text{pi}_{\text{link}}$ , then  $L_v(e) = \text{ip}_{\text{link}}$ , and vice versa.
2. Each node  $v$  has an incident edge labeled **pi<sub>link</sub>** if and only if  $v$  is labeled with **port** and it has no incident edge  $e$  satisfying  $L_v(e) \in \{\text{Ch}_L, \text{Ch}_R, L\}$ .
3. Each node  $v$  labeled **inter-octopus** has degree at least 1 and only incident half-edges labeled **ip<sub>link</sub>**. Moreover, nodes not labeled **inter-octopus** have no incident half-edges labeled **ip<sub>link</sub>**.

**Lemma 5.11.** *Let  $G$  be any non-empty connected graph that is  $(\mathcal{V}^{\text{proper}}, \mathcal{E}^{\text{proper}})$ -labeled such that  $\mathcal{C}^{\text{proper}}$  is satisfied at all nodes. Then,  $G$  is a proper instance according to Definition 5.10.*

*Proof.* We start by proving that there must be at least one node not labeled **inter-octopus**. Since the graph  $G$  is non-empty, it contains at least some node  $v$ , which is either labeled **inter-octopus** or not. In the latter case we are done. In the former case, by constraint 3,  $v$  has an incident edge  $e = \{u, v\}$  such that  $L_v(e) = \text{ip}_{\text{link}}$ . By constraints 1 and 2,  $u$  is labeled with **port** and  $L_u(e) = \text{pi}_{\text{link}}$ . Hence, a node not labeled **inter-octopus** exists.

We thus know that the subgraph  $H_{\text{Oct}}$  induced by nodes not labeled **inter-octopus** is non-empty, and, by constraint 0,  $H_{\text{Oct}}$  is such that each connected component is an octopus gadget.

By constraint 2, each **port** node  $u$  that is the left-most leaf of its gadget must have an incident edge  $\{u, u'\}$  such that  $L_u(e) = \text{pi}_{\text{link}}$ , and no other node is allowed to have such an incident edge. By constraint 1 and 3,  $u'$  must be labeled **inter-octopus** and  $L_{u'}(e) = \text{ip}_{\text{link}}$ . This in particular implies that the set  $V(H_{\text{Ext}}) = V(G) \setminus V(H_{\text{Oct}})$  is non-empty.

By constraint 3, all edges incident to nodes in  $V(H_{\text{Ext}})$  are labeled  $\text{ip}_{\text{link}}$ . Moreover, by constraint 3, no nodes in  $H_{\text{Oct}}$  have incident half-edges labeled  $\text{ip}_{\text{link}}$ . By constraint 1, each edge  $e = \{u, v\}$  incident to an **inter-octopus** node  $u$ , must satisfy  $L_v(e) = \text{pi}_{\text{link}}$ , and, by constraint 2,  $v$  must be a left-most leaf of a port gadget. Summarizing, we obtained the following.

- Each connected component in the subgraph induced by intra-octopus edges is an octopus gadget.
- Only left-most leaves of port gadgets have incident inter-octopus edges.
- Inter-octopus edges must have an endpoint that is a left-most leaf of a port gadget and an endpoint that is an inter-octopus node.

Hence,  $G$  is a proper instance. □

**Lemma 5.12.** *Let  $G$  be a proper instance as defined in Definition 5.10. Then, there exists a  $(\mathcal{V}^{\text{proper}}, \mathcal{E}^{\text{proper}})$ -labeling of  $G$  that satisfies the constraints in  $\mathcal{C}^{\text{proper}}$  at all nodes.*

*Proof.* We properly label all nodes and edges in octopus gadgets respecting  $\mathcal{C}^{\text{octopus}}$  as in Lemma 5.7. To all nodes that are outside octopus gadgets, we assign the label **inter-octopus**. To all edges  $e$  that connect a node  $u$  labeled with **inter-octopus** to a node  $v$  labeled with **port**, we assign labels so that  $L_u(e) = \text{ip}_{\text{link}}$  and  $L_v(e) = \text{pi}_{\text{link}}$ . All constraints are satisfied by construction. □

**The LCL problem  $\Pi^{\text{badGraph}}$ .** We now define an LCL problem  $\Pi^{\text{badGraph}}$  that, on a high level, allows the nodes to prove that the graph in which they are is not a proper instance labeled in a valid way. Differently from the cases of  $\Pi^{\text{badTree}}$  and  $\Pi^{\text{badOctopus}}$ , we cannot guarantee the property that, if the graph is invalid, then nodes can spend  $O(\log n)$  rounds to produce a solution in which no node outputs  $\perp$ . This time, the problem  $\Pi^{\text{badGraph}}$  will satisfy the following weaker guarantee, which will be sufficient for our purposes: nodes can spend  $O(\log n)$  rounds to produce a solution in which the graph induced by nodes outputting  $\perp$  is a proper instance. On a high level, the problem will satisfy the following properties.

- There are three possible types of output: a node can either produce an empty output ( $\perp$ ), or it can output an error, and there are two types of errors. In the latter case, a node needs to also output a locally checkable proof of the fact that the graph is not a proper instance.
- If the graph is a proper instance, the constraints of the problem are defined such that the only valid solution to the problem  $\Pi^{\text{badGraph}}$  is the one where all nodes output  $\perp$ .

- Otherwise, if the graph is not a proper instance, nodes are able to spend  $O(\log n)$  deterministic classical rounds to label invalid parts of the graph, such that the graph induced by nodes outputting  $\perp$  is a proper instance, and such that the labeling on the invalid parts of the graph encodes a proof that these parts are indeed invalid.

We now give a formal definition of the LCL problem  $\Pi^{\text{badGraph}}$ . The inputs are defined as follows.

- Each node  $v$  receives as input a label  $g_v \in \mathcal{V}^{\text{proper}}$ .
- Each half-edge receives as input a label from  $\mathcal{E}^{\text{proper}}$ .

The possible node output labels  $\mathcal{V}^{\text{badGraph}}$  for the problem  $\Pi^{\text{badGraph}}$  are the following.

- The empty output  $\perp$ .
- A pair  $(\text{Error}_{\text{intra}}, x_v)$ , where  $x_v \in \mathcal{V}^{\text{badOctopus}}$ . This label will be allowed only on intra-octopus nodes.
- The labels  $\text{Error}_{\text{inter},1}$  and  $\text{Error}_{\text{inter},2}$ . These labels will be allowed only on inter-octopus nodes.

We call *error outputs* all labels that are different from  $\perp$ . Informally, the label  $\text{Error}_{\text{inter},1}$  will be used by inter-octopus nodes that do not satisfy the constraints  $\mathcal{C}^{\text{proper}}$ , while  $\text{Error}_{\text{inter},2}$  will be used by inter-octopus nodes satisfying that *all* their intra-octopus neighbors output errors. On a high level, these labels will be used as follows. Inter-octopus nodes that are not properly connected to intra-octopus nodes output  $\text{Error}_{\text{inter},1}$ . Then, intra-octopus nodes connected to inter-octopus nodes that are outputting an error are marked. Then, nodes solve  $\Pi^{\text{badOctopus}}$  in the subgraph induced by intra-octopus nodes. Finally, inter-octopus nodes output  $\text{Error}_{\text{inter},2}$  if they satisfy the following two properties: they did not output  $\text{Error}_{\text{inter},1}$  previously, and all their neighbors (which are intra-octopus nodes) output an error. We now define the constraints  $\mathcal{C}^{\text{badGraph}}$ .

*The constraints  $\mathcal{C}^{\text{badGraph}}$*

- A node can always output  $\perp$ .
- A node  $v$  can output  $\text{Error}_{\text{inter},1}$  only if  $g_v = \text{inter-octopus}$  and it does not satisfy the constraints  $\mathcal{C}^{\text{proper}}$ .
- If a node  $v$  outputs  $(\text{Error}_{\text{intra}}, x_v)$ , then the following must hold.
  1.  $x_v$  must be different from  $\perp$ .
  2. Consider the labeling of the graph obtained by input labeling each node  $v$  with the pair  $(m_v, g_v)$ , where  $m_v := 1$  if  $v$  has a neighbor outputting  $\text{Error}_{\text{inter},1}$  and  $m_v := 0$  otherwise, and output labeling each node  $v$  with  $x_v$ . On node  $v$  the constraints  $\mathcal{C}^{\text{badOctopus}}$  must be satisfied. That is,  $x_v$  is a valid solution for  $\Pi^{\text{badOctopus}}$  when we mark nodes that are incident to inter-octopus nodes that output  $\text{Error}_{\text{inter},1}$ .
- A node  $v$  can output  $\text{Error}_{\text{inter},2}$  only if  $g_v = \text{inter-octopus}$  and all its neighbors do not output  $\perp$ .

**Lemma 5.13.** *Let  $G$  be a  $(\mathcal{V}^{\text{proper}}, \mathcal{E}^{\text{proper}})$ -labeled graph where  $\mathcal{C}^{\text{proper}}$  is satisfied for all nodes. Then, the only valid solution for  $\Pi^{\text{badGraph}}$  is the one assigning  $\perp$  to all nodes.*

*Proof.* Since  $\mathcal{C}^{\text{proper}}$  is satisfied at all nodes, an inter-octopus node cannot output  $\text{Error}_{\text{inter},1}$ . This implies that the input for  $\Pi^{\text{badOctopus}}$  satisfies  $m_v = 0$  for all intra-octopus nodes  $v$ . Since each connected component of the subgraph induced by intra-octopus nodes is a properly labeled octopus gadget, intra-octopus nodes must output  $\perp$ . This implies that inter-octopus nodes cannot output  $\text{Error}_{\text{inter},2}$ . We thus get that all nodes must output  $\perp$ .  $\square$

**Lemma 5.14.** *Let  $G$  be a  $(\mathcal{V}^{\text{proper}}, \mathcal{E}^{\text{proper}})$ -labeled graph. There exists a solution for  $\Pi^{\text{badGraph}}$  where each connected component induced by nodes outputting  $\perp$  is a proper instance. Moreover, such a solution can be computed in  $O(\log n)$  classical deterministic rounds.*

*Proof.* In order to produce the claimed solution, we will use the algorithm for  $\Pi^{\text{badOctopus}}$  reported in the proof of the Lemma 5.9. The algorithm for solving  $\Pi^{\text{badGraph}}$  works as follows.

Each node  $v$  satisfying  $g_v = \text{inter-octopus}$  spends  $O(1)$  rounds to check whether it satisfies  $\mathcal{C}^{\text{proper}}$ . If these constraints are not satisfied,  $v$  outputs  $\text{Error}_{\text{inter},1}$ . Then, each intra-octopus node  $v$  spends  $O(1)$  rounds to compute  $m_v$ . By Lemma 5.9, intra-octopus nodes can spend  $O(\log n)$  deterministic classical rounds to compute a solution for  $\Pi^{\text{badOctopus}}$ , where  $(m_v, g_v)$  is the input of node  $v$  for the problem. Such a solution satisfies that the following property: if in the subgraph induced by intra-octopus edges, in the connected component containing  $v$ , there is at least one marked node or a node not satisfying  $\mathcal{C}^{\text{octopus}}$ , then  $v$  outputs a label different from  $\perp$ . For each node  $v$ , let  $x_v$  be this output. If  $x_v \neq \perp$ , then  $v$  outputs  $(\text{Error}_{\text{intra}}, x_v)$ , otherwise  $v$  outputs  $\perp$ . Finally, each inter-octopus node that did not output  $\text{Error}_{\text{inter},1}$  spends  $O(1)$  rounds to check if at least one neighbor output some error in the previous phase, and in that case it outputs  $\text{Error}_{\text{inter},1}$ .

The solution clearly satisfies the constraints of  $\Pi^{\text{badGraph}}$ , and the runtime is clearly  $O(\log n)$ . We now prove that the subgraph induced by nodes outputting  $\perp$  is a proper instance.

Let  $G'$  be the subgraph induced by nodes outputting  $\perp$ . By the definition of the algorithm, each connected component of the subgraph of  $G'$  induced by intra-octopus nodes must be a valid octopus gadget. Moreover, the inter-octopus nodes incident to intra-octopus gadgets of  $G'$  must not be labeled  $\text{Error}_{\text{inter},1}$ , since otherwise their neighbors  $v$  in octopus gadgets would have satisfied  $m_v = 1$  and hence output something different from  $\perp$ . Also, inter-octopus nodes incident to intra-octopus gadgets of  $G'$  cannot be labeled  $\text{Error}_{\text{inter},2}$ , since they have at least one intra-octopus neighbor outputting  $\perp$ . We thus get that inter-octopus nodes that are neighbors of intra-octopus gadgets of  $G'$  are all labeled  $\perp$ . Additionally, all left-most leaves of octopus ports must have an inter-octopus neighbor, since otherwise they would have output an error. We thus get that, in  $G'$ ,  $\mathcal{C}^{\text{proper}}$  is satisfied on all nodes, and by Lemma 5.11 this implies that  $G'$  is a proper instance.  $\square$

## 5.4 The problem $\Pi^{\text{promise}}$

In this section, we define the problem  $\Pi^{\text{promise}}$  as a function of a given problem  $\Pi^{\text{linearizable}} = (\Sigma, (F, L, P), B)$  (see Definition 3.1 for the definition of linearizable problems). As mentioned in Section 3.5, the problem  $\Pi$  will be defined such that it is required to solve  $\Pi^{\text{badGraph}}$  on the whole graph, and it is required to solve  $\Pi^{\text{promise}}$  on the subgraph induced by nodes that output  $\perp$  for  $\Pi^{\text{badGraph}}$ .

Recall that the problem  $\Pi^{\text{badGraph}}$  allows to output  $\perp$  on all nodes, even on graphs that are not proper instances. If an algorithm produces such an output, then it is required to solve  $\Pi^{\text{promise}}$  on a graph that is not a proper instance. However, when proving a lower bound for  $\Pi$ , we will use a proper instance, and when proving an upper bound for  $\Pi$ , we will define an algorithm satisfying that the subgraph induced by nodes outputting  $\perp$  for  $\Pi^{\text{badGraph}}$  is always a proper instance. In other words, it does not matter how  $\Pi^{\text{promise}}$  is defined on graphs that are not proper instances, as it will not affect the lower and the upper bounds that we prove. For this reason, we will define the constraints  $\mathcal{C}^{\text{promise}}$  of  $\Pi^{\text{promise}}$  under the assumption that the given graph  $G$  is a proper instance labeled such that all nodes satisfy  $\mathcal{C}^{\text{proper}}$ . This will make the definition easier to read. Then, the definition of  $\Pi^{\text{promise}}$  is lifted to any arbitrary graph as follows.

- Let  $r = O(1)$  be the distance that each node  $v$  needs to inspect to check whether it satisfies  $\mathcal{C}^{\text{promise}}$  (on a proper instance).

- If all nodes that are within distance  $r$  from  $v$  (including  $v$ ) satisfy  $\mathcal{C}^{\text{proper}}$ , then  $v$  needs to satisfy  $\mathcal{C}^{\text{promise}}$ .
- Otherwise, any output is considered incorrect.

In this way, whenever a node  $v$  needs to satisfy  $\mathcal{C}^{\text{promise}}$ , the radius- $r$  neighborhood of  $v$  is a subgraph of a proper instance, and hence the constraints of  $\mathcal{C}^{\text{promise}}$  are well-defined.

Let us proceed with some nomenclature. If  $v$  is labeled **head** and  $L_v(e) \notin \{\text{Ch}_L, \text{Ch}_R\}$  for all  $e$  incident to  $v$ , then we call  $v$  *head-leaf*. If  $v$  is labeled **port** and  $L_v(e) \neq \text{P}$  for all  $e$  incident to  $v$ , then we call  $v$  *port-root*. For each head-leaf  $v$ , let  $\eta(v) = |\{e : L_v(e) \in \{\text{hp}_{\text{link}_1}, \text{hp}_{\text{link}_2}\}\}|$  count the number of port gadgets that are connected to  $v$  (which are either 1 or 2).

We are now ready to define  $\Pi^{\text{promise}}$ . The node input set for  $\Pi^{\text{promise}}$  is  $\mathcal{V}_{\text{in}}^{\text{promise}} = \mathcal{V}^{\text{proper}}$ , while the node-edge input set for  $\Pi^{\text{promise}}$  is  $\mathcal{E}_{\text{in}}^{\text{promise}} = \mathcal{E}^{\text{proper}}$ . The node output set is  $\mathcal{V}_{\text{out}}^{\text{promise}} = \Sigma \cup \{\perp\}$ . Let us denote with  $\text{out}(v)$  the output of a node  $v$  for  $\Pi^{\text{promise}}$ . The constraints  $\mathcal{C}^{\text{promise}}$  of  $\Pi^{\text{promise}}$  are defined as follows.

*The constraints  $\mathcal{C}^{\text{promise}}$*

1. If  $v$  is labeled **port**, then it must hold that  $\text{out}(v) \in \Sigma$ .
2. If  $v$  is not labeled **port**, then it must hold that  $\text{out}(v) = \perp$ .
3. If a node  $v$  is labeled **port**, then, for each neighbor  $u$  of  $v$  that is also labeled **port**, it must hold that  $\text{out}(v) = \text{out}(u)$ .
4. If  $v$  is a head-leaf node, and  $L_v(e) \neq \text{L}$  for all  $e$  incident to  $v$ , then it must hold that  $\text{out}(f(v, \text{hp}_{\text{link}_1})) \in F$ .
5. If  $v$  is a head-leaf node, and  $L_v(e) \neq \text{R}$  for all  $e$  incident to  $v$ , then:
  - if  $\eta(v) = 1$ , then it must hold that  $\text{out}(f(v, \text{hp}_{\text{link}_1})) \in L$ ;
  - if  $\eta(v) = 2$ , then it must hold that  $\text{out}(f(v, \text{hp}_{\text{link}_2})) \in L$ .
6. If  $v$  is a head-leaf node satisfying that  $\eta(v) = 2$ , then it must hold that  $(\text{out}(f(v, \text{hp}_{\text{link}_1})), \text{out}(f(v, \text{hp}_{\text{link}_2}))) \in P$ .
7. If  $v$  is a head-leaf node satisfying that  $\eta(v) = j$  for some  $j \in \{1, 2\}$ , and  $f(v, \text{R}) = u \neq \perp$ , then it must hold that  $(\text{out}(f(v, \text{hp}_{\text{link}_j})), \text{out}(f(u, \text{hp}_{\text{link}_1}))) \in P$ .
8. If  $v$  is labeled **inter-octopus** and  $\{u_1, u_2, \dots, u_k\}$  is the set of all nodes adjacent to  $v$ , then it must hold that  $\{\text{out}(u_1), \text{out}(u_2), \dots, \text{out}(u_k)\} \in B$ .

We now prove some facts about the relation between  $\Pi^{\text{linearizable}}$  and  $\Pi^{\text{promise}}$ . As informally explained in Section 3.3, a proper instance can be obtained by replacing the nodes belonging to one side of a bipartite graph with octopus gadgets. We now provide a formal definition of this construction.

**Definition 5.15** (Lift of a bipartite graph). Let  $G = (W \cup B, E)$  be a bipartite graph that is not necessarily connected and that may contain parallel edges. A *lift* of  $G$  is a pair  $(L, \text{compression})$ , where  $L = (V', E')$  is a graph that is a proper instance (according to Definition 5.10), and  $\text{compression} : V' \rightarrow W \cup B \cup E$  is a function, with the following properties.

1. For each node  $v$  of any head gadget,  $\text{compression}(v) \in W$ .

2. If  $u, v \in V'$  are nodes of the same head gadget, then  $\text{compression}(u) = \text{compression}(v)$ .
3. If  $u, v \in V'$  are nodes of different head gadgets, then  $\text{compression}(u) \neq \text{compression}(v)$ .
4. For any node  $v$  of any port gadget,  $\text{compression}(v) \in E$ .
5. If  $u, v \in V'$  are nodes of the same port gadget, then  $\text{compression}(u) = \text{compression}(v)$ .
6. If  $u, v \in V'$  are nodes of different port gadgets, then  $\text{compression}(u) \neq \text{compression}(v)$ .
7. Let  $u \in V'$  be a node of a port gadget  $P$ , and let  $v \in V'$  be a node of a head gadget  $H$ . The edge  $\text{compression}(u)$  is incident to the node  $\text{compression}(v)$  if and only if  $P$  is connected to  $H$ .
8. If  $v \in V'$  is an inter-octopus node, then  $\text{compression}(v) \in B$ .
9. Let  $u \in V'$  be a node of a port gadget  $P$ , and let  $v \in V'$  be an inter-octopus node. The edge  $\text{compression}(u)$  is incident to the node  $\text{compression}(v)$  if and only if  $P$  is connected to  $v$ .

Furthermore, for a proper instance  $L$  and a bipartite graph  $G$ , if there exists a function  $\text{compression}$  for which  $(L, \text{compression})$  is a lift of  $G$ , then we say that  $L$  can be compressed to  $G$ .

In the following, for a bipartite graph  $G = (W \cup B, E)$ , by *ordering assigned to the edges of  $G$*  we denote a function  $\tau : w \mapsto \sigma_w$  that takes as input a node  $w \in W$  and returns a function  $\sigma_w$ , where the function  $\sigma_w$  gives an ordering of the incident half-edges of  $w$ . That is,  $\tau$  assigns, to each node  $w$ , an injective function  $\sigma_w$  mapping each edge incident to  $w$  to an integer in  $\{1, \dots, \deg(w)\}$ , where  $\deg(w)$  is the degree of  $w$ .

**Observation 5.16.** *Let  $G = (W \cup B, E)$  be a bipartite graph, and let  $(L = (V', E'), \text{compression})$  be a lift of  $G$  that is  $(\mathcal{V}^{\text{proper}}, \mathcal{E}^{\text{proper}})$ -labeled such that  $\mathcal{C}^{\text{proper}}$  is satisfied on all nodes. Then, the function  $\text{compression} : V' \rightarrow W \cup B \cup E$  uniquely defines, for each node  $w \in W$ , the ordering  $\sigma_w$  of the edges incident to  $w$ .*

*Proof.* For each node  $w \in W$ , let  $G_w$  be the octopus gadget whose nodes map either to  $w$  or to its incident edges according to the function  $\text{compression}$ . We now define an ordering of the port gadgets of  $G_w$  which will automatically induce an ordering of the edges incident to  $w$ , and hence an ordering assigned to the edges of  $G$ . Assume that  $G_w$  has height  $h$ . Consider two port gadgets  $P$  and  $P'$  of  $G_w$ , and let  $r_P$  and  $r_{P'}$  be their root nodes, respectively. Furthermore, let  $v_P$  and  $v_{P'}$  be the head-leaf nodes of  $G_w$  that are connected to  $r_P$  and  $r_{P'}$ , respectively. Assume that  $v_P = (h-1, i)$  and  $v_{P'} = (h-1, j)$  for some  $i, j \in \{0, \dots, 2^{h-1}\}$ . We say that  $P < P'$  if and only if  $i < j$  or, if  $i = j$ ,  $L_{v_P}(\{v_P, r_P\}) = \text{hp}_{\text{link}_1}$  and  $L_{v_{P'}}(\{v_{P'}, r_{P'}\}) = \text{hp}_{\text{link}_2}$ . Let  $P_i$  be the  $i$ -th port of  $G_w$  according to this ordering, and let  $u$  be an arbitrary node of  $P_i$ . The  $i$ -th edge of  $w$  is the edge  $\text{compression}(u)$ .  $\square$

In the definition of  $\Pi^{\text{linearizable}}$ , the given bipartite graph  $G = (W \cup B, E)$  comes with a local ordering  $\sigma_w$  of the edges of each node  $w \in W$ . We can then think of lifts that “respect” such an ordering.

**Definition 5.17** (Edge-order preserving lift). Let  $G = (W \cup B, E)$  be a bipartite graph that is not necessarily connected and that may contain parallel edges, and let  $\tau : w \mapsto \sigma_w$  be an ordering assigned to the edges of  $G$ . Let  $(L, \text{compression})$  be a lift of  $G$ , where  $L$  is  $(\mathcal{V}^{\text{proper}}, \mathcal{E}^{\text{proper}})$ -labeled. Let  $\tau' : w \mapsto \sigma'_w$  be a function assigning to each  $w \in W$  the edge ordering defined via  $\text{compression}$  according to Observation 5.16. We say that  $(L, \text{compression})$  is edge-order preserving with respect to  $\tau$  if  $\sigma'_w = \sigma_w$  for all  $w \in W$ .

**Lemma 5.18.** *For each bipartite graph  $G = (W \cup B, E)$  which comes with an ordering  $\tau : w \in W \mapsto \sigma_w$  assigned to its edges, and for each integer  $h$ , there exists a lift  $(L, \text{compression})$  that is  $(\mathcal{V}^{\text{proper}}, \mathcal{E}^{\text{proper}})$ -labeled such that all nodes satisfy  $\mathcal{C}^{\text{proper}}$  where all port gadgets of  $L$  have height  $h$ , and such that  $(L, \text{compression})$  is edge-order preserving with respect to  $\tau$ . Moreover, for each proper instance  $L$ , there exists a bipartite graph  $G$  such that  $L$  can be compressed to  $G$ .*

*Proof.* We first prove that for each bipartite graph  $G = (W \cup B, E)$  with an edge-ordering  $\tau$  and for each integer  $h$ , there exists a lift  $(L, \text{compression})$  where all port gadgets of  $L$  have height  $h$  and that is edge-order preserving w.r.t.  $\tau$ . Recall that to each node  $w \in W$  is assigned an ordering  $\sigma_w$  of its incident edges.

For each node  $w \in W$ , we create an  $(x_w, \eta_w, W_w)$ -octopus gadget  $G_w$ , where the parameters  $x_w$ ,  $\eta_w$ , and  $W_w$  are chosen as follows.

- $x_w = 2^{\lfloor \log_2(\deg(w)) \rfloor} + 1$ .
- $\eta_w \in \{1, 2\}^{x_w - 1}$  is a vector satisfying that  $\sum \eta_w(i) = \deg(w)$ . Such a vector exists, since  $(x_w - 1) \leq \deg(w) \leq 2(x_w - 1)$ .
- All elements of  $W_w$  are equal to  $h$ .

Let  $E_w = \{e_{(w,1)}, \dots, e_{(w,\deg(w))}\}$  be the set of edges that are incident to  $w$ , ordered according to  $\sigma_w$ . For each  $1 \leq i \leq \deg(w)$ , let  $P_{e_{(w,i)}}$  be the  $i$ -th port gadget connected to the octopus gadget  $G_w$  (according to the natural order defined in the proof of Observation 5.16). For each black node  $b \in B$ , we create a node  $v_b$  that is an inter-octopus node. If  $b$  is connected to  $w$  through some edge  $e_{(w,i)}$ , then  $v_b$  is connected to the left-most leaf of  $P_{e_{(w,i)}}$ . It is trivial to see that  $L$  admits a compression function to  $G$  with the properties given in Definition 5.15.

We now prove that each proper instance  $L = (V, E)$  can be compressed to some bipartite graph  $G = (W \cup B, E')$ . We construct  $G$  as follows. The set of white nodes  $W$  contains a vertex  $v_H$  for each head gadget  $H$ . The set of black nodes  $B$  contains a node  $u_{v_{\text{inter-octopus}}}$  for each inter-octopus node  $v_{\text{inter-octopus}}$ . Then,  $v_H$  is connected to  $u_{v_{\text{inter-octopus}}}$  through an edge if and only if  $H$  is connected to a port gadget  $P$  that is, in turn, connected to  $v_{\text{inter-octopus}}$ .  $\square$

**Lemma 5.19.** *Let  $\Pi^{\text{promise}}$  be the problem defined as a function of a linearizable problem  $\Pi^{\text{linearizable}}$  as described in Section 5.4. Let  $L = (V', E')$  be a proper instance, and let  $G = (W \cup B, E)$  be the bipartite graph to which  $L$  can be compressed to, according to Lemma 5.18. For each node  $w \in W$ , let  $\sigma_w$  be the ordering defined in Observation 5.16.*

*Then, there is a bijective function that maps solutions of  $\Pi^{\text{promise}}$  on  $L$  to solutions of  $\Pi^{\text{linearizable}}$  on  $G$  (with the aforementioned edge ordering).*

*Proof.* Let  $\text{compression} : V' \rightarrow W \cup B \cup E$  be the function that compresses  $L$  to  $G$ . First, suppose that we are given a solution for  $\Pi^{\text{promise}}$  on  $L$ . Consider any white node  $w$  of  $G$  and its set of incident edges  $e_{(w,1)}, \dots, e_{(w,\deg(w))}$  ordered according to  $\sigma_w$ . Consider an arbitrary node  $v$  of a port gadget  $P$  such that  $\text{compression}(v) = e_{(w,i)}$ . Then, the output of  $w$  on the edge  $e_{(w,i)}$  is defined to be the same as the output of  $v$ . This labeling clearly satisfies the constraints of  $\Pi^{\text{linearizable}}$  under the ordering of the edges defined by  $\sigma = \{\sigma_w \mid w \in W\}$ .

Now, suppose we are given a solution for  $\Pi^{\text{linearizable}}$  on  $G$ . Consider any white node  $w$  of  $G$  and its set of incident edges  $e_{(w,1)}, \dots, e_{(w,\deg(w))}$ , ordered according to  $\sigma_w$ . For all nodes  $v \in \text{compression}^{-1}(e_{(w,i)})$ , we define the output of  $v$  to be the same as the output of  $w$  on  $e_{(w,i)}$ . For all other nodes  $v$  of  $G$ , we define the output of  $v$  to be  $\perp$ . This labeling clearly satisfies the constraints of  $\Pi^{\text{promise}}$ .  $\square$

## 5.5 The LCL problem $\Pi$

We are now ready to define our problem  $\Pi$ , as a function of the given problem  $\Pi^{\text{promise}}$ . The input labels of  $\Pi$  are the same as the input labels  $\mathcal{V}^{\text{proper}}$  and  $\mathcal{E}^{\text{proper}}$  of  $\Pi^{\text{badGraph}}$ . The set  $\mathcal{V}^\Pi$  of output labels contains the following labels.

- $(\text{badGraph}, x)$ , for all  $x \in \mathcal{V}^{\text{badGraph}} \setminus \{\perp\}$ .
- $(\text{promise}, x)$ , for all  $x \in \mathcal{V}_{\text{out}}^{\text{promise}}$ .

The constraints  $\mathcal{C}^\Pi$  are defined as follows.

*The constraints  $\mathcal{C}^\Pi$*

- Consider the graph labeling in which the output labels are changed as follows: nodes labeled  $(\text{promise}, x)$  become labeled  $\perp$ , while nodes labeled  $(\text{badGraph}, x)$  become labeled  $x$ . Then, this labeling must satisfy the constraint  $\mathcal{C}^{\text{badGraph}}$ .
- Let  $G'$  be the subgraph induced by nodes outputting  $(\text{promise}, x)$ , for some  $x$ . Consider the labeling of  $G'$  in which nodes labeled  $(\text{promise}, x)$  become labeled  $x$ . Then, all nodes of  $G'$ , in the subgraph  $G'$ , must satisfy  $\mathcal{C}^{\text{promise}}$ .

In other words, if nodes output a  $(\text{badGraph}, \cdot)$  label, then this labeling must be a valid solution for  $\Pi^{\text{badGraph}}$ . This allows nodes to mark invalid parts of the graph. Then, in the parts that are not marked as invalid, nodes need to solve  $\Pi^{\text{promise}}$ .

## 5.6 Upper bound in quantum-LOCAL

In this section, we prove an upper bound on the quantum complexity of  $\Pi$  as a function of the quantum complexity of  $\Pi^{\text{linearizable}}$ . More in detail, we prove the following.

**Lemma 5.20.** *Let  $T(n)$  be an upper bound on the quantum complexity of  $\Pi^{\text{linearizable}}$ , that holds also if the given graph contains parallel edges. Then, the quantum complexity of  $\Pi$  is upper bounded by  $O(T(n) \log n)$ .*

*Proof.* Let  $G$  be the graph in which we need to solve  $\Pi$ . The algorithm is composed of two parts, a classical deterministic part and a quantum part. The first part uses Lemma 5.14 to compute a solution for  $\Pi^{\text{badGraph}}$  such that the nodes outputting  $\perp$  induce a proper instance. This requires  $O(\log n)$  classical deterministic rounds. For each  $v$ , let  $x_v$  be its output for  $\Pi^{\text{badGraph}}$ . Each node  $v$  satisfying  $x_v \neq \perp$  outputs  $(\text{badGraph}, x_v)$  for  $\Pi$ . Note that the graph  $G'$  induced by nodes that still did not output anything for  $\Pi$  is a proper instance.

The second part of the algorithm works as follows. Nodes that have already produced an output for  $\Pi$  do nothing. The other nodes simulate the quantum algorithm  $\mathcal{A}$  for  $\Pi^{\text{linearizable}}$  as follows. The goal is to simulate  $\mathcal{A}$  in the graph obtained by contracting each octopus gadget of  $G'$  into a single node. Let  $\hat{G}$  be this graph. Observe that this is the graph that  $G$  can be compressed to, according to Lemma 5.18. For each octopus gadget  $\nu$ , the root of its head gadget is the node responsible for storing the quantum state of the node in  $\hat{G}$  corresponding to  $\nu$ . The inter-octopus nodes are unaffected and simulate themselves in  $\hat{G}$ . Since the diameter of a valid octopus gadget is clearly upper bounded by  $O(\log n)$ , we get that the communication between nodes of  $\hat{G}$  can be simulated in  $O(\log n)$  quantum rounds in  $G'$ . Hence, after  $O(T(n) \log n)$ , all the roots of the head gadget computed a solution for  $\Pi^{\text{linearizable}}$  in  $\hat{G}$ . With additional  $O(\log n)$  steps nodes can solve  $\Pi^{\text{promise}}$  in  $G'$ , according to the mapping defined in Lemma 5.19. For each node  $v$  in  $G'$ , let  $x_v$  be its output for  $\Pi^{\text{promise}}$ . Each node  $v$  in  $G'$  outputs  $(\text{promise}, x_v)$  for  $\Pi$ .

The output clearly satisfies the constraints  $\mathcal{C}^\Pi$ , and the runtime is clearly upper bounded by  $O(T(n) \log n)$ .  $\square$

## 5.7 Lower bound in LOCAL

In this section, we prove a lower bound on the classical randomized complexity of  $\Pi$  as a function of the classical randomized complexity of  $\Pi^{\text{linearizable}}$ . More in detail, we prove the following.

**Lemma 5.21.** *Let  $T(n)$  be a lower bound on the time required to solve  $\Pi^{\text{linearizable}}$  with a classical randomized algorithm that has failure probability at most  $1/n$ . Then, any classical randomized algorithm for  $\Pi$  with failure probability at most  $1/n$  requires  $\Omega(T(n^{1/3}) \log n)$  rounds.*

*Proof.* For a contradiction, assume that there exists an algorithm  $\mathcal{A}$  that solves  $\Pi$  in  $T'(n) = o(T(n^{1/3}) \log n)$  rounds with failure probability at most  $1/n$ . We show that we can construct an algorithm  $\mathcal{B}$  that solves  $\Pi^{\text{linearizable}}$  in  $o(T(n))$  with failure probability at most  $1/n$ , contradicting the hypothesis.

The algorithm  $\mathcal{B}$  works as follows. Let  $G$  be an instance of  $\Pi^{\text{linearizable}}$ . We construct an instance  $G'$  of  $\Pi$  by applying Lemma 5.18 to construct a lift of  $G$  with parameter  $h = \Theta(\log n)$ . On a high level, the algorithm  $\mathcal{B}$  will be defined such that the nodes of  $G$  simulate the execution of  $\mathcal{A}$  on  $G'$ .

Recall that  $G'$  is constructed as follows. Each hyperedge  $e$  of  $G$  is replaced with an inter-octopus node  $b_e$ . Each node  $v$  of degree  $d_v$  is replaced with an octopus gadget  $\nu_v$  that contains exactly  $d_v$  ports, such that each port gadget has height  $\Theta(\log n)$ , and such that the number of nodes of each port gadget is strictly less than  $n$ . For each node-hyperedge pair  $(u, e)$  of  $G$ , we add an edge to  $G'$  that connects the left-most leaf of the  $i$ -th port of  $\nu_v$  to  $b_e$ , assuming that  $e$  is the  $i$ -th edge incident to  $u$  according to the given ordering  $\sigma_u$ . We label  $G'$  such that it is a properly labeled valid instance. Observe that the labeling of the nodes of  $\nu_v$  can be computed by  $v$  without communication.

Since each port gadget has size strictly less than  $n$ , and since the maximum degree of  $G$  is  $n$ , we obtain that  $G'$  has at most  $n^3$  nodes. Observe that if two nodes  $(u, v)$  are neighbors in  $G$ , then the head gadgets of  $\nu_v$  and  $\nu_u$  are at distance  $\Theta(\log n)$  in  $G'$ . Hence, each node  $v$  of  $G$  can spend  $O(T'(n^3)/\log n)$  rounds of communication in  $G$  to gather its  $T'(n^3)$ -radius neighborhood in  $G'$ . Since the runtime of  $\mathcal{A}$  on instances of size at most  $n^3$  is upper bounded by  $T'(n^3)$ , and since  $G'$  has size at most  $n^3$ , then each node  $v$  is able to compute the output of  $\mathcal{A}$  for  $\Pi$  on  $G'$  without further communication.

Since  $G'$  is a properly labeled valid instance, all nodes of  $G'$  must output labels of type  $(\text{promise}, \cdot)$ . Hence,  $v$  can reconstruct a solution for  $\Pi^{\text{linearizable}}$  as a function of the output of the nodes in  $\nu_v$ , without communication, according to Lemma 5.19.

The runtime of the algorithm  $\mathcal{B}$  for  $\Pi^{\text{linearizable}}$  is  $O(T'(n^3)/\log n) = o(T(n))$ , and its failure probability is upper bounded by  $1/n^3 \leq 1/n$ , proving the claim.  $\square$

## 5.8 Instantiating the construction

In [BBC+25], the authors introduced a family of problems  $\{\text{GHZ}(\Delta) \mid \Delta \geq 3\}$ . While this set can be seen as a family of problems parametrized by the maximum degree  $\Delta$  of the graph, for the ease of presentation we will present this family as a single problem that we call *iterated GHZ*. We now formally define this problem, show that it can be defined as a linearizable problem, and then state its quantum and classical complexities.

**The iterated GHZ problem.** An instance of the problem is a hypergraph of rank 3. However, for the ease of presentation, in the following, we will describe an input instance as a bipartite graph

$(U, V, E)$ , where nodes in  $V$  have degree exactly 3 (equivalently, in order to remove this assumption, if a node in  $V$  has degree different from 3, then it is unconstrained). Nodes in  $U$  are called *white* nodes, or *players*, while nodes in  $V$  are called *black* nodes, or *games*. Each node  $u \in U$  receives as input an ordering on its incident edges. Let  $e_{u,i}$  be the  $i$ -th edge incident to  $u$  according to this order, where  $1 \leq i \leq d_u$ , and  $d_u$  is the degree of  $u$ . The problem is defined as follows. Each node  $u \in U$  must output two bits  $x(u, i)$  and  $y(u, i)$  on each incident edge  $e_{u,i}$ . The following must hold.

- For each white node  $u$ , it must hold that  $x(u, 1) = 0$ .
- For each white node  $u$ , it must hold that  $y(u, i) = x(u, i + 1)$ , for all  $i$ .
- Let  $v$  be a black node, and let  $u_1, u_2, u_3$  be its white neighbors. Let  $i_j$  be the position of the edge  $\{u_j, v\}$  according to the order of the edges incident to  $u_j$ , for  $1 \leq j \leq 3$ . Then, the following must hold.
  - If  $i_1 = i_2 = i_3 = 1$ , then it must hold that the multiset  $\{y(u_1, 1), y(u_2, 1), y(u_3, 1)\}$  is equal to  $\{0, 0, 1\}$ .
  - Otherwise, if  $x(u_1, i_1) + x(u_2, i_2) + x(u_3, i_3)$  is even, then it must hold that  $y(u_1, i_1) \oplus y(u_2, i_2) \oplus y(u_3, i_3) = x(u_1, i_1) \vee x(u_2, i_2) \vee x(u_3, i_3)$ .
  - Otherwise, node  $v$  is unconstrained.

The original definition of  $\text{GHZ}(\Delta)$  of [BBC+25] is equivalent to the definition that we have provided, when restricted to the case in which the maximum degree of the graph is  $\Delta$ .

**Lower bound for classical randomized algorithms.** In [BBC+25], the following lower bound is shown.

**Theorem 5.22** ([BBC+25]). *Let  $\Delta \geq 3$  be an integer. Any classical randomized algorithm that solves  $\text{GHZ}(\Delta)$  with high probability requires  $\Omega(\min\{\Delta, \log \Delta \log n\})$  rounds.*

In order to obtain the best possible lower bound stated solely as a function of  $n$ , we take  $\Delta = \Theta\left(\frac{\log \log n}{\log \log \log n}\right)$ , and we obtain the following.

**Corollary 5.23** ([BBC+25]). *Any classical randomized algorithm that solves iterated  $\text{GHZ}$  with high probability requires  $\Omega\left(\frac{\log \log n}{\log \log \log n}\right)$  rounds.*

**Upper bound for quantum algorithms.** In [BBC+25], the lower and upper bounds for the problem  $\text{GHZ}(\Delta)$  are proved in a setting in which the input instances are  $\Delta$ -edge colored. In the construction described in this section, however, we would need to provide a different color to each port gadget belonging to the same octopus gadget. This would require  $\omega(1)$  input labels, which is not allowed in a proper LCL.

Since the lower bound of [BBC+25] holds in a  $\Delta$ -edge colored graph, this lower bound clearly holds in graphs that are not  $\Delta$ -edge colored (having such a coloring can only make the problem potentially easier). However, for an upper bound, we need to adapt the algorithm of [BBC+25] to the case in which the edge coloring is not provided. Moreover, in order to use Lemma 5.20, we need an algorithm that works also in the case of parallel edges.

**Lemma 5.24.** *The iterated  $\text{GHZ}$  problem can be solved in  $O(1)$  rounds with a quantum algorithm, even if parallel edges are present.*

*Proof.* The algorithm works in two rounds of communications, which are described in the following two points.

1. Each white node  $w$  does the following. Let  $b_i$  be the  $i$ -th neighbor of  $w$ . Node  $w$  sends  $i$  to  $b_i$ , for all  $1 \leq i \leq d_w$ , where  $d_w$  is the degree of  $w$ .
2. Each black node  $b$  has now received three port numbers  $p_v, p_u, p_z$  from its three white neighbors  $v, u, z$ . It then prepares three qubits  $q_v, q_u, q_z$  in the GHZ state, i.e.,

$$|0\rangle_{q_v} |0\rangle_{q_u} |0\rangle_{q_z} \rightarrow \frac{1}{\sqrt{2}} \left( |0\rangle_{q_v} |0\rangle_{q_u} |0\rangle_{q_z} + |1\rangle_{q_v} |1\rangle_{q_u} |1\rangle_{q_z} \right).$$

If  $p_v = p_u = p_z = 1$ , then  $b$  sends the pair  $(q_v, 0)$  to  $v$ , the pair  $(q_u, 0)$  to  $u$ , and the pair  $(q_z, 1)$  to  $z$ . Otherwise, it just sends the qubit  $q_v$  to  $v$ , the qubit  $q_u$  to  $u$ , and the qubit  $q_z$  to  $z$ .

Now, all white nodes decide on their outputs without any further communication. First, all white nodes  $w$  set  $x(w, 1) = 0$ . Now, observe that each white node  $w$  has received a qubit  $q_{w,i}$  through its  $i$ -th incident edge. Furthermore, some white nodes  $w$  have received a pair  $(q_{w,1}, j_w)$ , where  $q_{w,1}$  is a qubit and  $j_w \in \{0, 1\}$ : we call such nodes *lucky*. Each lucky white node  $w$  sets  $y(w, 1) = j_w$  and  $x(w, 2) = j_w$  (if  $d_w \geq 2$ ). Now we describe what each white node  $w$  outputs for the  $j$ -th incident edge. If a node is lucky, it does the following for each  $j \geq 2$  iteratively, while if a node is unlucky it does the following for each  $j \geq 1$  iteratively. Each node  $w$  measures the qubit  $q_{w,j}$  according to the well-known GHZ strategy (with input  $x(w, j)$ ) that wins the game with probability 1 [BBT05], and sets the output  $y(w, j)$  accordingly. Then, node  $w$  sets  $x(w, j+1) = y(w, j)$  (if  $j+1 \leq d_w$ ).

We now argue about the correctness of the algorithm. White constraints are satisfied by construction of the algorithm. Suppose now that the constraints of some black node  $b$  is not satisfied. This means that the three white neighbors  $v, u, z$  of  $b$  have some outputs  $(x_v, y_v), (x_u, y_u), (z_u, y_u)$  that do not satisfy the constraints of the problem. We have three cases: If the three port numbers  $p_v, p_u, p_z$  are such that  $p_v = p_u = p_z = 1$ , then, by point 2, the white nodes are lucky and have set the outputs to be  $(0, 0), (0, 0), (0, 1)$ , which is a valid solution. Suppose that at least a port number between  $p_v, p_u, p_z$  is different from 1. If  $x_v + x_u + x_z$  is odd, then any solution is valid. Suppose now that  $x_v + x_u + x_z$  is even. Then, the outputs of the white nodes are constructed by measuring the three entangled qubits  $q_v, q_u, q_z$  in the GHZ state prepared by  $b$ . By the well-known GHZ winning strategy [BBT05], the white nodes' outputs are such that  $y_v \oplus y_u \oplus y_z = x_v \vee x_u \vee x_z$ , which is a valid solution.  $\square$

**The iterated GHZ problem as a linearizable problem.** We define a problem  $\Pi^{\text{linearizable}} = (\Sigma, (F, L, P), B)$  that encodes iterated GHZ as a linearizable problem. On a high level, each label will encode a pair  $(x(v, i), y(v, i))$  of bits, and we will use special labels for the case in which  $i = 1$ . We define the set of possible first labels as  $F = \{(\text{first}, y) \mid y \in \{0, 1\}\}$ . Then, we define the set of possible output labels as  $\Sigma = F \cup \{(\text{other}, x, y) \mid x, y \in \{0, 1\}\}$ , and we define  $L = \Sigma$ . The allowed pairs  $P$  are defined as all the pairs satisfying the following.

- All pairs  $((\text{first}, y_1), (\text{other}, x_2, y_2))$  where  $y_1 = x_2$  and  $y_1, x_2, y_2 \in \{0, 1\}$ .
- All pairs  $((\text{other}, x_1, y_1), (\text{other}, x_2, y_2))$  where  $y_1 = x_2$  and  $x_1, y_1, x_2, y_2 \in \{0, 1\}$ .

Let  $\{L_1, L_2, L_3\}$  be a multiset containing three elements from  $\Sigma$ . If  $L_i = (\text{first}, y_i)$  for some  $y_i \in \{0, 1\}$ , then let  $\ell_i = \text{first}$  and let  $x_i = 0$ . If  $L_i = (\text{other}, x_i, y_i)$  for some  $x_i, y_i \in \{0, 1\}$ , then let  $\ell_i = \text{other}$ . Observe that now, for all  $1 \leq i \leq 3$ ,  $\ell_i, x_i$  and  $y_i$  are defined. The set  $B$  contains the multisets  $\{L_1, L_2, L_3\}$  satisfying the following.

- All the multisets satisfying  $\ell_1 = \ell_2 = \ell_3 = \text{first}$  such that the multiset  $\{y_1, y_2, y_3\}$  is equal to  $\{0, 0, 1\}$ .
- All the multisets where, if  $\text{other} \in \{\ell_1, \ell_2, \ell_3\}$ , and  $x_1 + x_2 + x_3$  is even, then  $y_1 \oplus y_2 \oplus y_3 = x_1 \vee x_2 \vee x_3$ .
- All the multisets where  $\text{other} \in \{\ell_1, \ell_2, \ell_3\}$  and  $x_1 + x_2 + x_3$  is odd.

It is clear that  $\Pi^{\text{linearizable}}$  and iterated GHZ are equivalent:

- both problems require outputting two bits per edge;
- the black constraint of iterated GHZ has the same requirements of the set  $B$  on the pairs incident to a black node;
- both the white constraint of iterated GHZ and the triple  $(F, L, P)$  requires that the second bit of a port is the same as the first bit of the next one.

**Putting things together.** By combining the problem  $\Pi^{\text{linearizable}}$  obtained as a function of iterated GHZ with Corollary 5.23 and Lemmas 5.20, 5.21 and 5.24, we obtain the following result.

**Theorem 5.25.** *There exists an LCL problem  $\Pi$  with quantum complexity  $O(\log n)$  and classical randomized complexity  $\Omega\left(\log n \cdot \frac{\log \log n}{\log \log \log n}\right)$ .*

## Acknowledgements

This work was supported in part by the Research Council of Finland, Grants 359104 and 363558, by the MUR (Italy) Department of Excellence 2023–2027 for GSSI. Francesco d’Amore is supported by the project Decreto MUR n. 47/2025, CUP: D13C25000750001. This project was initiated at the Research Workshop on Distributed Algorithms (RW-DIST 2025) in Freiburg, Germany; we would like to thank all workshop participants and organizers for inspiring discussions. We would also like to thank Sebastian Brandt, Xavier Coiteux-Roy, François Le Gall, Augusto Modanese, Marc-Olivier Renou, Ronja Stimpert, Lucas Tendick, and Isadora Veeren for discussions that led to this project.

## References

- [ACd+25] Amirreza Akbari, Xavier Coiteux-Roy, Francesco d’Amore, François Le Gall, Henrik Lievonen, Darya Melnyk, Augusto Modanese, Shreyas Pai, Marc-Olivier Renou, Václav Rozhoň, and Jukka Suomela. “Online locality meets distributed quantum computing”. In: *Proc. 57th ACM Symposium on Theory of Computing (STOC 2025)*. ACM Press, 2025. URL: <https://doi.org/10.1145/3717823.3718211>.
- [AEL+23] Amirreza Akbari, Navid Eslami, Henrik Lievonen, Darya Melnyk, Joonas Särkijärvi, and Jukka Suomela. “Locality in Online, Dynamic, Sequential, and Distributed Graph Algorithms”. In: *50th International Colloquium on Automata, Languages, and Programming, ICALP 2023, July 10–14, 2023, Paderborn, Germany*. Ed. by Kousha Etessami, Uriel Feige, and Gabriele Puppis. Vol. 261. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 10:1–10:20. DOI: 10.4230/LIPICS.ICALP.2023.10.

- [BBC+25] Alkida Balliu, Sebastian Brandt, Xavier Coiteux-Roy, Francesco d’Amore, Massimo Equi, François Le Gall, Henrik Lievonen, Augusto Modanese, Dennis Olivetti, Marc-Olivier Renou, Jukka Suomela, Lucas Tendick, and Isadora Veeren. “Distributed quantum advantage for local problems”. In: *Proc. 57th ACM Symposium on Theory of Computing (STOC 2025)*. ACM Press, 2025. URL: <https://doi.org/10.1145/3717823.3718233>.
- [BBO+20] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. “How much does randomness help with locally checkable problems?” In: *PODC ’20: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, August 3-7, 2020*. Ed. by Yuval Emek and Christian Cachin. ACM, 2020, pp. 299–308. DOI: 10.1145/3382734.3405715.
- [BBO+21] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. “Almost global problems in the LOCAL model”. In: *Distributed Computing* 34.4 (2021), pp. 259–281. DOI: 10.1007/S00446-020-00375-2.
- [BCd+26] Alkida Balliu, Filippo Casagrande, Francesco d’Amore, Massimo Equi, Barbara Keller, Henrik Lievonen, Dennis Olivetti, Gustav Schmid, and Jukka Suomela. “Distributed Quantum Advantage in Locally Checkable Labeling Problems”. In: *Proceedings of the 2026 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2026, pp. 1268–1308. DOI: 10.1137/1.9781611978971.49. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611978971.49>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9781611978971.49>.
- [BCM+21] Alkida Balliu, Keren Censor-Hillel, Yannic Maus, Dennis Olivetti, and Jukka Suomela. “Locally Checkable Labelings with Small Messages”. In: *35th International Symposium on Distributed Computing, DISC 2021, October 4-8, 2021, Freiburg, Germany (Virtual Conference)*. Ed. by Seth Gilbert. Vol. 209. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, 8:1–8:18. DOI: 10.4230/LIPICS.DISC.2021.8.
- [BCC+25] Alkida Balliu, Corinna Coupette, Antonio Cruciani, Francesco d’Amore, Massimo Equi, Henrik Lievonen, Augusto Modanese, Dennis Olivetti, and Jukka Suomela. “New Limits on Distributed Quantum Advantage: Dequantizing Linear Programs”. In: *39th International Symposium on Distributed Computing, DISC 2025, October 27-31, 2025, Berlin, Germany*. Ed. by Dariusz Kowalski. Vol. 356. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.
- [BGK+25] Alkida Balliu, Mohsen Ghaffari, Fabian Kuhn, Augusto Modanese, Dennis Olivetti, Mikael Rabie, Jukka Suomela, and Jara Uitto. “Shared Randomness Helps with Local Distributed Problems”. In: *52nd International Colloquium on Automata, Languages, and Programming, ICALP 2025, Aarhus, Denmark, July 8-11, 2025*. Ed. by Keren Censor-Hillel, Fabrizio Grandoni, Joël Ouaknine, and Gabriele Puppis. Vol. 334. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, 16:1–16:18. DOI: 10.4230/LIPICS.ICALP.2025.16. URL: <https://doi.org/10.4230/LIPICS.ICALP.2025.16>.
- [BHK+18] Alkida Balliu, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Dennis Olivetti, and Jukka Suomela. “New classes of distributed time complexity”. In: *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2018, Los Angeles, CA, USA, June 25-29, 2018*. Ed. by Ilias Diakonikolas, David Kempe, and Monika Henzinger. ACM, 2018, pp. 1307–1318. DOI: 10.1145/3188745.3188860.

- [Bra19] Sebastian Brandt. “An Automatic Speedup Theorem for Distributed Problems”. In: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*. Ed. by Peter Robinson and Faith Ellen. ACM, 2019, pp. 379–388. DOI: 10.1145/3293611.3331611.
- [BFH+16] Sebastian Brandt, Orr Fischer, Juho Hirvonen, Barbara Keller, Tuomo Lempiäinen, Joel Rybicki, Jukka Suomela, and Jara Uitto. “A lower bound for the distributed Lovász local lemma”. In: *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2016, Cambridge, MA, USA, June 18-21, 2016*. Ed. by Daniel Wichs and Yishay Mansour. ACM, 2016, pp. 479–488. DOI: 10.1145/2897518.2897570.
- [BHK+17] Sebastian Brandt, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Patric R. J. Östergård, Christopher Purcell, Joel Rybicki, Jukka Suomela, and Przemysław Uznanski. “LCL Problems on Grids”. In: *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2017, Washington, DC, USA, July 25-27, 2017*. Ed. by Elad Michael Schiller and Alexander A. Schwarzmann. ACM, 2017, pp. 101–110. DOI: 10.1145/3087801.3087833.
- [BBT05] Gilles Brassard, Anne Broadbent, and Alain Tapp. “Quantum Pseudo-Telepathy”. In: *Foundations of Physics* 35.11 (2005), pp. 1877–1907. DOI: 10.1007/s10701-005-7353-4.
- [CKP19] Yi-Jun Chang, Tsvi Kopelowitz, and Seth Pettie. “An Exponential Separation between Randomized and Deterministic Complexity in the LOCAL Model”. In: *SIAM Journal on Computing* 48.1 (2019), pp. 122–143. DOI: 10.1137/17M1117537.
- [CMN+23] Yi-Jun Chang, Gopinath Mishra, Hung Thuan Nguyen, Mingyang Yang, and Yu-Cheng Yeh. “A Tight Lower Bound for 3-Coloring Grids in the Online-LOCAL Model”. In: *CoRR* abs/2312.01384 (2023). DOI: 10.48550/ARXIV.2312.01384. arXiv: 2312.01384.
- [CP19] Yi-Jun Chang and Seth Pettie. “A Time Hierarchy Theorem for the LOCAL Model”. In: *SIAM Journal on Computing* 48.1 (2019), pp. 33–69. DOI: 10.1137/17M1157957.
- [CDG+24] Xavier Coiteux-Roy, Francesco D’Amore, Rishikesh Gajjala, Fabian Kuhn, François Le Gall, Henrik Lievonen, Augusto Modanese, Marc-Olivier Renou, Gustav Schmid, and Jukka Suomela. “No Distributed Quantum Advantage for Approximate Graph Coloring”. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*. Ed. by Bojan Mohar, Igor Shinkar, and Ryan O’Donnell. ACM, 2024, pp. 1901–1910. DOI: 10.1145/3618260.3649679.
- [dAm25] Francesco d’Amore. “On the limits of distributed quantum computing”. In: *Bulletin of the EATCS* 145 (2025), pp. 51–91. URL: <http://bulletin.eatcs.org/index.php/beatcs/article/view/829>.
- [DCP16] Giacomo Mauro D’Ariano, Giulio Chiribella, and Paolo Perinotti. *Quantum Theory from First Principles: An Informational Approach*. Cambridge University Press, 2016. DOI: 10.1017/9781107338340.
- [DDL+23] Sameep Dahal, Francesco D’Amore, Henrik Lievonen, Timothé Picavet, and Jukka Suomela. “Brief Announcement: Distributed Derandomization Revisited”. In: *37th International Symposium on Distributed Computing, DISC 2023, October 10-12, 2023, L’Aquila, Italy*. Ed. by Rotem Oshman. Vol. 281. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 40:1–40:5. DOI: 10.4230/LIPICS.DISC.2023.40.

- [DKL+24] Anubhav Dhar, Eli Kujawa, Henrik Lievonon, Augusto Modanese, Mikail Muftuoglu, Jan Studený, and Jukka Suomela. “Local Problems in Trees Across a Wide Range of Distributed Models”. In: *28th International Conference on Principles of Distributed Systems, OPODIS 2024, December 11-13, 2024, Lucca, Italy*. Ed. by Silvia Bonomi, Letterio Galletta, Etienne Rivière, and Valerio Schiavoni. Vol. 324. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, 27:1–27:17. DOI: 10.4230/LIPICS.OPODIS.2024.27.
- [GKM09] Cyril Gavoille, Adrian Kosowski, and Marcin Markiewicz. “What Can Be Observed Locally?”. In: *Distributed Computing, 23rd International Symposium, DISC 2009, Elche, Spain, September 23-25, 2009. Proceedings*. Ed. by Idit Keidar. Vol. 5805. Lecture Notes in Computer Science. Springer, 2009, pp. 243–257. DOI: 10.1007/978-3-642-04355-0\_26.
- [GGH+23] Mohsen Ghaffari, Christoph Grunau, Bernhard Haeupler, Saeed Ilchi, and Václav Rozhon. “Improved Distributed Network Decomposition, Hitting Sets, and Spanners, via Derandomization”. In: *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*. Ed. by Nikhil Bansal and Viswanath Nagarajan. SIAM, 2023, pp. 2532–2566. DOI: 10.1137/1.9781611977554.CH97.
- [GHK18] Mohsen Ghaffari, David G. Harris, and Fabian Kuhn. “On Derandomizing Local Distributed Algorithms”. In: *59th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2018, Paris, France, October 7-9, 2018*. Ed. by Mikkel Thorup. IEEE Computer Society, 2018, pp. 662–673. DOI: 10.1109/FOCS.2018.00069.
- [GKM17] Mohsen Ghaffari, Fabian Kuhn, and Yannic Maus. “On the complexity of local distributed graph problems”. In: *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2017, Montreal, QC, Canada, June 19-23, 2017*. Ed. by Hamed Hatami, Pierre McKenzie, and Valerie King. ACM, 2017, pp. 784–797. DOI: 10.1145/3055399.3055471.
- [GHZ89] Daniel M. Greenberger, Michael A. Horne, and Anton Zeilinger. “Going Beyond Bell’s Theorem”. In: *Bell’s Theorem, Quantum Theory and Conceptions of the Universe*. Springer Netherlands, 1989, pp. 69–72. DOI: 10.1007/978-94-017-0849-4\_10.
- [Hol24] Alexander E. Holroyd. “Symmetrization for finitely dependent colouring”. In: *Electronic Communications in Probability* 29 (2024). DOI: 10.1214/24-ecp600.
- [HHL18] Alexander E. Holroyd, Tom Hutchcroft, and Avi Levy. “Finitely dependent cycle coloring”. In: *Electronic Communications in Probability* 23 (2018). DOI: 10.1214/18-ecp118.
- [HL16] Alexander E. Holroyd and Thomas M. Liggett. “Finitely Dependent Coloring”. In: *Forum of Mathematics, Pi* 4, e9 (2016). DOI: 10.1017/fmp.2016.7.
- [LNR19] François Le Gall, Harumichi Nishimura, and Ansis Rosmanis. “Quantum Advantage for the LOCAL Model in Distributed Computing”. In: *36th International Symposium on Theoretical Aspects of Computer Science, STACS 2019, March 13-16, 2019, Berlin, Germany*. Ed. by Rolf Niedermeier and Christophe Paul. Vol. 126. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, 49:1–49:14. DOI: 10.4230/LIPICS.STACS.2019.49.
- [Mer90] N. David Mermin. “Quantum mysteries revisited”. In: *American Journal of Physics* 58.8 (1990), pp. 731–734. DOI: 10.1119/1.16503.

- [NS95] Moni Naor and Larry J. Stockmeyer. “What Can be Computed Locally?” In: *SIAM Journal on Computing* 24.6 (1995), pp. 1259–1277. DOI: 10.1137/S0097539793254571.
- [Suo20] Jukka Suomela. “Landscape of Locality (Invited Talk)”. In: *17th Scandinavian Symposium and Workshops on Algorithm Theory, SWAT 2020, June 22-24, 2020, Tórshavn, Faroe Islands*. Ed. by Susanne Albers. Vol. 162. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020, 2:1–2:1. DOI: 10.4230/LIPICS.SWAT.2020.2.
- [Suo24] Jukka Suomela. *Open problems related to locality in distributed graph algorithms*. 2024. URL: <https://jukkasuomela.fi/open/>.

## A Simulation of component-wise online-LOCAL in LOCAL

In this section, we extend the result of Section 4 to hold also for the *component-wise randomized online-LOCAL model*. We start by briefly discussing the state of the art and motivating the study of yet another variant of the LOCAL model. We then define formally what we mean by component-wise randomized online-LOCAL. Finally, we extend the proofs of Section 4 to hold also in this new model.

### A.1 Motivation

Informally, component-wise randomized online-LOCAL is just like the component-wise deterministic online-LOCAL model (see [ACd+25, dAm25]), but generalized for the randomized case. In particular, a deterministic component-wise online-LOCAL algorithm is also a randomized one.

The component-wise deterministic online-LOCAL model was originally introduced because it allows simulation of randomized online-LOCAL in weaker models like SLOCAL in rooted trees and forests [ACd+25]. In particular, randomized online-LOCAL captures the power of non-signaling distributions [ACd+25], SLOCAL [GKM17] and dynamic-LOCAL [AEL+23]. Hence, proving lower bounds for randomized online-LOCAL allows us to provide strong lower bounds that span across a wide variety of models.

One way to prove these lower bounds is by providing a simulation of stronger models in weaker ones. One such model is the component-wise randomized online-LOCAL, which is stronger than both the deterministic component-wise online-LOCAL and the bounded-dependence model.

Extending the simulation result of Section 4 to component-wise randomized online-LOCAL model improves the state of the art in two major ways:

1. So far the only result known for component-wise deterministic online-LOCAL is the simulation result of Akbari et al. [ACd+25] in rooted trees and forests. Our result works in all graphs, albeit with a worse locality.
2. We provide a direct reduction from the bounded-dependence model to the component-wise randomized online-LOCAL. Previously, the reduction had to go through the non-signaling model, randomized online-LOCAL and component-wise deterministic online-LOCAL. This round-trip through randomized online-LOCAL incurs a doubly exponential cost in the locality. For localities in the  $\Omega(\log \log n)$  region, our result is asymptotically tighter.

### A.2 Definitions

We now formally define what we mean by component-wise randomized online-LOCAL. To do this, we first define the notion of *partial online-LOCAL run of length  $\ell$* , as introduced by Akbari et al. [ACd+25]:

**Definition A.1** (Partial online-LOCAL run of length  $\ell$ ). Let  $G$  be a graph with an order of nodes  $v_1, v_2, \dots, v_n$ , and let  $x$  encode the input data on each vertex. Consider the subgraph  $G_\ell \subseteq G$  induced by the radius- $T$  neighborhoods of the first  $\ell$  nodes  $v_1, \dots, v_\ell$ , i.e.,  $G_\ell = \mathring{G}[\mathcal{N}_T[\{v_1, \dots, v_\ell\}]]$ . We call  $(G_\ell, (v_1, \dots, v_\ell), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_\ell\}]})$  the *partial  $T$ -rounds online-LOCAL run of length  $\ell$  of  $G$* .

We denote by  $(G_\ell, (v_1, \dots, v_\ell), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_\ell\}]})[v_\ell]$  the tuple  $(\bar{G}_\ell, (w_1, \dots, w_k), x \upharpoonright_{\mathcal{N}_T[\{w_1, \dots, w_k\}]})$  where  $\bar{G}_\ell \subseteq G_\ell$  is the connected component containing  $v_\ell$ ,  $(w_1, \dots, w_k)$  is the maximal subsequence of  $(v_1, \dots, v_\ell)$  of nodes that belong to  $\bar{G}_\ell$ , and  $x \upharpoonright_{\mathcal{N}_T[\{w_1, \dots, w_k\}]}$  is the restriction of the local inputs to this component. In such case,  $k$  is the length of  $(G_\ell, (v_1, \dots, v_\ell), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_\ell\}]})[v_\ell]$ . Notice that  $w_k = v_\ell$ .

Now we are ready to introduce the online-LOCAL model. We loosely follow the definitions given in by Akbari et al. [ACd+25].

**The online-LOCAL model.** The deterministic online-LOCAL model was first introduced in [AEL+23]. It is a centralized model of computing where the algorithm initially knows only the number of nodes  $n$  of the input graph  $G$ . The nodes are processed with respect to an adversarial input ordering sequence  $\sigma = v_1, v_2, \dots, v_n$ . Like in the LOCAL model, each node  $v$  has some local state variable  $x(v)$  which encodes input data. In an algorithm with locality  $T$ , the output of node  $v_i$  depends on the partial  $T$ -rounds online-LOCAL run of length  $i$  of  $G$ , that is,  $(G_i, (v_1, \dots, v_i), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_i\}]})$  (see Definition A.1). In the *randomized* online-LOCAL model [ACd+25], every node has access to a private infinite random bit string, which is independent of the random bit strings of other nodes. Trivially, thanks to global memory, node  $v_i$  knows the random bit strings of  $\mathcal{N}_T[\{v_1, \dots, v_i\}]$ . In the randomized case, the solution to a problem should be correct with probability at least  $1 - 1/\text{poly}(n)$ .

We now give a formal definition of the component-wise online-LOCAL model.

**The component-wise online-LOCAL model.** The component-wise online-LOCAL model is similar to the online-LOCAL model above, but with some further restrictions. More specifically, let  $(G, (v_1, \dots, v_n), x_G)$  and  $(H, (u_1, \dots, u_n), x_H)$  be two  $n$ -node input instances. Let  $i, j \in [n]$  be two indices such that

$$\begin{aligned} (G_i, (v_1, \dots, v_i), x_G \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_i\}]})[v_i] &= (\bar{G}_i, (v'_1, \dots, v'_k), x_G \upharpoonright_{\mathcal{N}_T[\{v'_1, \dots, v'_k\}]}) \quad \text{and} \\ (H_j, (u_1, \dots, u_j), x_H \upharpoonright_{\mathcal{N}_T[\{u_1, \dots, u_j\}]})[u_j] &= (\bar{H}_j, (u'_1, \dots, u'_k), x_H \upharpoonright_{\mathcal{N}_T[\{u'_1, \dots, u'_k\}]}) \end{aligned}$$

are isomorphic with the following properties: the isomorphism preserves the order, i.e., it brings  $v'_h$  to  $u'_h$  for every  $h \in [n]$ , and the isomorphism also preserves the inputs  $x_G$  and  $x_H$  in these neighborhoods. Then, the algorithm must produce the same output for  $v_i$  and  $u_j$ .

In the *randomized* component-wise online-LOCAL model, each node is given access to a private infinite random bit string, which is independent of the random bit strings of other nodes. When the algorithm needs to output a label for node  $v_i$ , it can only use the random bit strings of nodes in  $\bar{G}_i$ . Again, in the randomized model a solution to a problem must be correct with probability at least  $1 - 1/\text{poly}(n)$ .

To see why this matches the intuition of an algorithm being *component-wise*, imagine that  $G_i$  is the real input and that  $H_j$  consists of only a single component, that is

$$(H_j, (u_1, \dots, u_j), x_H \upharpoonright_{\mathcal{N}_T[\{u_1, \dots, u_j\}]})[u_j] = (H_j, (u_1, \dots, u_j), x_H \upharpoonright_{\mathcal{N}_T[\{u_1, \dots, u_j\}]})$$

By the above definition, the algorithm must produce the same output for  $G_i$  and  $H_j$ . Then, effectively what the algorithm does on  $G_i$  must depend only on what the algorithm has seen in  $H_j$ , that is only the current component.

### A.3 Simulation of component-wise randomized online-LOCAL in randomized LOCAL

It is intuitive that a bounded-dependence outcome implies the existence of a component-wise online-LOCAL algorithm with the same locality: we formalize this intuition through the following lemma.

**Lemma A.2.** *Let  $\Pi$  be any labeling problem with output node label set  $\Sigma$ , and let  $\mathcal{O}$  be a bounded-dependent outcome solving  $\Pi$  with probability  $p > 0$  and locality  $T(n)$  on graphs of  $n$  nodes. Then*

there exists a component-wise randomized online-LOCAL algorithm solving  $\Pi$  with probability  $p$  with the same locality  $T(n)$ .

*Proof.* Consider any graph  $G$  of  $n$  nodes, and any adversarial processing order of the nodes  $(v_1, \dots, v_n)$ . We now construct a randomized component-wise online-LOCAL algorithm  $\mathcal{A}$  with the desired properties. When the adversary reveals  $v_1$  and  $(G_1, (v_1), x \upharpoonright_{\mathcal{N}_T[v_1]})$  to  $\mathcal{A}$ ,  $\mathcal{A}$  just samples an output  $\text{out}(v_1) = \lambda_{v_1}$  for  $v_1$  from  $O(G, x)[\{v_1\}]$ . Now, when the adversary reveals  $v_i$  and  $(G_i, (v_1, \dots, v_i), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_i\}]})[v_i] = (\bar{G}_i, (v'_1, \dots, v'_k), x \upharpoonright_{\mathcal{N}_T[\{v'_1, \dots, v'_k\}]})$  to  $\mathcal{A}$ , it samples an output  $\text{out}(v_i)$  for  $v_i$  from the distribution  $O(G, x)[\mathcal{N}_T[\{v'_1, \dots, v'_k\}]]$  conditional on the fact that the outputs of  $v'_1, \dots, v'_{k-1}$  have already been sampled and are equal to  $\text{out}(v'_1) = \lambda_{v'_1}, \dots, \text{out}(v'_{k-1}) = \lambda_{v'_{k-1}}$ . Let  $\mathcal{P}_{\mathcal{A}}$  denote the probability measure induced by the algorithm  $\mathcal{A}$  on the outputs of the nodes, and let  $\mathcal{P}_O$  denote the probability measure induced by the bounded-dependence outcome  $O$  on the outputs of the nodes. Fix any  $k \in [n]$ , and suppose

$$(G_k, (v_1, \dots, v_k), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_k\}]})[v_k] = (\bar{G}_k, (w_1, \dots, w_h), x \upharpoonright_{\mathcal{N}_T[\{w_1, \dots, w_h\}]})$$

with  $w_h = v_k$ . We first show that, for all  $(\lambda_{w_1}, \dots, \lambda_{w_h}) \in \Sigma^h$ , we have that

$$\mathcal{P}_{\mathcal{A}}(\cap_{i=1}^h \{\text{out}(w_i) = \lambda_{w_i}\}) = \mathcal{P}_O(\cap_{i=1}^h \{\text{out}(w_i) = \lambda_{w_i}\}).$$

Let us start by induction on  $h$ . The base case is  $h = 1$ . Since  $w_1$  is the only node in  $\bar{G}_1$ , then it means that there are no nodes in the radius- $(2T)$  neighborhood of  $w_1$  which have been previously labeled (otherwise the partial online-LOCAL run would contain them). By construction of  $\mathcal{A}$  we are freely sampling from  $O(G, x)[\mathcal{N}_T[w_1]]$  and the thesis is trivial. Suppose now  $h > 1$ . We have that

$$\begin{aligned} & \mathcal{P}_{\mathcal{A}}(\cap_{i=1}^h \{\text{out}(w_i) = \lambda_{w_i}\}) \\ &= \mathcal{P}_{\mathcal{A}}\left(\text{out}(w_h) = \lambda_{w_h} \mid \cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}\right) \mathcal{P}_{\mathcal{A}}(\cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}). \end{aligned}$$

By the inductive hypothesis,  $\mathcal{P}_{\mathcal{A}}(\cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}) = \mathcal{P}_O(\cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\})$  and, by construction of  $\mathcal{A}$ ,  $\mathcal{P}_{\mathcal{A}}\left(\text{out}(w_h) = \lambda_{w_h} \mid \cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}\right) = \mathcal{P}_O\left(\text{out}(w_h) = \lambda_{w_h} \mid \cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}\right)$ . Hence,

$$\begin{aligned} & \mathcal{P}_{\mathcal{A}}(\cap_{i=1}^h \{\text{out}(w_i) = \lambda_{w_i}\}) \\ &= \mathcal{P}_{\mathcal{A}}\left(\text{out}(w_h) = \lambda_{w_h} \mid \cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}\right) \mathcal{P}_{\mathcal{A}}(\cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}) \\ &= \mathcal{P}_O\left(\text{out}(w_h) = \lambda_{w_h} \mid \cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}\right) \mathcal{P}_O(\cap_{i=1}^{h-1} \{\text{out}(w_i) = \lambda_{w_i}\}) \\ &= \mathcal{P}_O(\cap_{i=1}^h \{\text{out}(w_i) = \lambda_{w_i}\}), \end{aligned}$$

and the induction is complete. Let  $S \subseteq V(G)$  with  $S = \{v_{j_1}, \dots, v_{j_h}\}$  where  $j_1, \dots, j_h$  are induced by the ordering  $v_1, \dots, v_n$ . The above argument implies that, for all  $(\lambda_{v_{j_1}}, \dots, \lambda_{v_{j_h}}) \in \Sigma^h$  it holds that

$$\mathcal{P}_{\mathcal{A}}(\cap_{i=1}^h \{\text{out}(v_{j_i}) = \lambda_{v_{j_i}}\}) = \mathcal{P}_O(\cap_{i=1}^h \{\text{out}(v_{j_i}) = \lambda_{v_{j_i}}\}).$$

In fact, consider the partial online-LOCAL runs  $\{(G_{j_i}, (v_1, \dots, v_{j_i}), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_{j_i}\}]})[v_{j_i}]\}_{i \in [h]}$ . Consider the maximal subsequence of partial online-LOCAL runs  $\{(G_{j_{i_k}}, (v_1, \dots, v_{j_{i_k}}), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_{j_{i_k}}\}]})[v_{j_{i_k}}]\}_{i_k \in [h^*]}$  such that  $G_{j_{i_k}}$  has no intersection with  $G_{j_i}$  for all  $i > i_k$ ,  $i \in [h]$ . We have two properties: First, it trivially holds that  $\cup_{k=1}^{h^*} G_{j_{i_k}} = \cup_{i=1}^h G_{j_i}$ . As for the second property, for any two  $k \neq k' \in [h^*]$ , let

$$(\bar{G}_{j_{i_k}}, (w_1, \dots, w_s), x \upharpoonright_{\mathcal{N}_T[\{w_1, \dots, w_s\}]}) = (G_{j_{i_k}}, (v_1, \dots, v_{j_{i_k}}), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_{j_{i_k}}\}]})[v_{j_{i_k}}]$$

and

$$(\bar{G}_{j_{i_{k'}}}, (w'_1, \dots, w'_{s'}), x \upharpoonright_{\mathcal{N}_T[\{w'_1, \dots, w'_{s'}\}]}) = (G_{j_{i_{k'}}}, (v_1, \dots, v_{j_{i_{k'}}}), x \upharpoonright_{\mathcal{N}_T[\{v_1, \dots, v_{j_{i_{k'}}}\}]})(v_{j_{i_{k'}}}).$$

Then,  $\text{dist}_G(\{w_1, \dots, w_S\}, \{w'_1, \dots, w'_{s'}\}) > 2T$ . Hence, we can treat the two partial online-LOCAL runs as independent, and we can apply the inductive hypothesis separately to each of them to conclude the proof and get that the overall success probability is at least  $p$ .  $\square$

*Remark A.3.* The non-signaling model (see [ACd+25] for a definition) cannot be simulated in the component-wise online-LOCAL model because non-signaling can make use of globally shared resources, such as shared randomness, while component-wise online-LOCAL cannot.

*Remark A.4.* Theorem 4.5 is almost-tight for the randomized component-wise online-LOCAL model. This is because [AEL+23] provides a deterministic  $O(\log n)$ -round component-wise online-LOCAL algorithm for 3-coloring bipartite graphs, and Theorem 4.5 would give a  $\tilde{O}(\sqrt{n})$ -round deterministic LOCAL algorithm for the same problem. However, we know that  $\Omega(\sqrt{n})$  is a lower bound for the problem in the LOCAL model, as well as in the bounded-dependence model (and even in the non-signaling model).